

UNIVERSITÀ DEGLI STUDI DI VERONA

Corso di Laurea in Bioinformatica

Algoritmi per Bioinformatica

e Laboratorio di Programmazione II

I seguenti appunti sono preparati tramite i lucidi del professore e il libro di riferimento

Per la parte di laboratorio il professore prende gli esercizi da:

<https://leetcode.com/problemset/algorithms/>

Anno Accademico 2025/2026

Indice

1	Fondamenti	9
1.1	Introduzione al corso	9
1.1.1	Requisiti	9
1.1.2	Programma del corso	9
1.1.3	Struttura del modulo	10
1.1.4	Modalità di esame	10
1.2	Il ruolo degli algoritmi nell'informatica	10
1.2.1	Cos'è un algoritmo?	10
1.3	Esempio di algoritmo ricorsivo	11
1.4	Esempi di problemi computazionali da risolvere in aula	16
2	InsertionSort e MergeSort	24
2.0.1	InsertionSort	24
2.1	MergeSort	32
2.1.1	Metodo Divide et Impera	32
2.1.2	Descrizione dell'algoritmo	32
2.1.3	Pseudocodice	32
2.1.4	Correttezza: Invariante di ciclo di Merge	34
2.1.5	Complessità	34
2.1.6	Confronto con InsertionSort	35
3	Notazione Asintotica	37
3.1	Introduzione	37
3.2	Notazione O — Limite superiore	37
3.3	Notazione Ω — Limite inferiore	38
3.4	Notazione Θ — Limite stretto	39
3.5	Uso nelle equazioni e disequazioni	40
3.6	Proprietà di confronto tra funzioni	40
3.7	Limiti asintotici delle funzioni elementari	40
3.8	Equazioni e disequazioni asintotiche	41
3.9	Proprietà delle notazioni asintotiche	42
3.10	Limiti asintotici delle funzioni elementari	42
4	Divide et Impera	43
4.1	Definizione	43
4.2	Intuizione	44
4.3	Motivazione	44
4.4	Algoritmo	44
4.5	Esempio guidato	44
4.6	Grafico o diagramma	45

4.7	Dimostrazione	46
4.8	Analisi	46
4.9	Errori tipici d'esame	46
4.10	Possibili domande d'esame	47
5	Analisi Probabilistica e Algoritmi Randomizzati	47
5.1	Preliminari: variabili casuali indicatrici	47
5.2	Analisi probabilistica: il Problema delle Assunzioni	48
5.3	Algoritmi randomizzati	49
5.4	Permutazione casuale: <code>RandomizeInPlace</code>	49
5.5	QuickSort	50
5.6	Idea Generale	50
5.7	Partition(A, p, r)	51
5.7.1	Invariante di ciclo di Partition	52
5.8	Analisi di Complessità di QuickSort	53
5.8.1	Caso peggiore	53
5.8.2	Caso migliore	54
5.8.3	Verso il caso medio	54
5.9	QuickSort Randomizzato	55
5.10	Analisi del Caso Medio di QuickSort	56
6	Statistiche d'Ordine e Problema della Mediana	57
6.1	Definizione	57
6.2	Intuizione	57
6.3	Motivazione	58
6.4	Algoritmo	58
6.5	Esempio guidato	58
6.6	Grafico o diagramma	59
6.7	Dimostrazione	59
6.8	Analisi	60
6.9	Errori tipici d'esame	60
6.10	Possibili domande d'esame	60
6.11	Contenuto richiesto ma non nel materiale ufficiale	60
7	HeapSort	60
7.1	Idea Generale	61
7.2	Struttura Dati Heap	61
7.2.1	Navigazione dell'albero tramite indici	62
7.2.2	Tipi di heap e proprietà	62
7.2.3	Altezza dello heap	62
7.2.4	Operazioni fondamentali sullo heap	63
7.3	MaxHeapify	63

7.3.1	Analisi di complessità di MaxHeapify	63
7.4	BuildMaxHeap	64
7.4.1	Correttezza di BuildMaxHeap	64
7.4.2	Complessità di BuildMaxHeap	65
7.5	Algoritmo HeapSort	65
7.5.1	Correttezza di HeapSort	66
7.5.2	Complessità di HeapSort	66
7.6	Code di Priorità	66
7.6.1	Operazioni fondamentali	67
7.6.2	Implementazione delle operazioni	67
8	Ordinamento in Tempo Lineare	68
8.1	Definizione	68
8.2	Intuizione	69
8.3	Motivazione	69
8.4	Algoritmo	69
8.5	Esempio guidato	70
8.6	Grafico o diagramma	70
8.7	Dimostrazione	71
8.8	Analisi	71
8.9	Errori tipici d'esame	72
8.10	Possibili domande d'esame	72
8.11	Contenuto richiesto ma non nel materiale ufficiale	72
9	Hashing e Tabelle Hash	72
9.1	Tavole a indirizzamento diretto	72
9.2	Tabelle hash	73
9.3	Funzioni hash: scelta e qualità	73
9.3.1	Conversione verso gli interi	73
9.4	Hashing statico	73
9.4.1	Metodo della divisione	74
9.4.2	Metodo della moltiplicazione	74
9.4.3	Metodo moltiplica-e-scorri	74
9.5	Hashing randomizzato	75
9.6	Gestione delle collisioni	76
9.6.1	Risoluzione mediante concatenamento	76
9.6.2	Risoluzione mediante indirizzamento aperto	77
9.6.3	Ispezione lineare	79
9.6.4	Doppio hashing	79
10	Alberi Binari di Ricerca	80
10.1	Definizione di ABR	80

10.2	Attraversamento	80
10.2.1	Attraversamento Simmetrico (in-order)	81
10.2.2	Pre-ordine e Post-ordine	81
10.3	Interrogazione di un ABR	82
10.3.1	Search	82
10.3.2	Minimum e Maximum	82
10.3.3	Successor e Predecessor	83
10.4	Inserimento e Cancellazione	84
10.4.1	Inserimento	84
10.4.2	Cancellazione	84
10.5	Costruzione di un ABR	86
10.6	Cenno agli ABR Rosso-Neri	87
10.7	Cenno ai Radix Tree	87
10.8	Esercizi	88
11	Programmazione Dinamica	88
11.1	Idea generale	88
11.1.1	Confronto con Divide et Impera	89
11.2	Esempi di applicazione: confronto di sequenze DNA	89
11.3	Massima Sottosequenza Comune (LCS)	90
11.3.1	Fase 1 — Sottostruttura ottima	90
11.3.2	Fase 2 — Definizione ricorsiva del valore	91
11.3.3	Fase 3 — Calcolo bottom-up: algoritmo LCS-LENGTH	91
11.3.4	Fase 4 — Ricostruzione: algoritmo PRINT-LCS	92
11.4	Massima Sottosequenza Crescente	92
11.4.1	Riduzione a cammino massimo in un DAG	92
11.4.2	Definizione ricorsiva	92
11.4.3	Algoritmo bottom-up (versione valutativa)	93
11.5	String Matching Approssimato	93
11.5.1	Definizione ricorsiva	94
11.5.2	Algoritmo SMA	94
11.6	Problema dello Zaino (Knapsack 0-1)	95
11.6.1	Sottostruttura ottima	95
11.6.2	Definizione ricorsiva	95
11.6.3	Algoritmo ZAINO	96
11.7	Casi comuni di sottoproblemi	96
11.8	Memoization	97
11.8.1	Esempio: Zaino con memoization	97
12	Algoritmi Avidi (Greedy)	97
12.1	Paradigma greedy	97

12.1.1	Definizione	98
12.1.2	Intuizione	98
12.1.3	Motivazione	98
12.1.4	Proprieta' teoriche	98
12.1.5	Algoritmo	99
12.1.6	Esempio guidato	99
12.1.7	Grafico o diagramma	100
12.1.8	Dimostrazione	100
12.1.9	Analisi della complessita'	100
12.1.10	Errori tipici d'esame	101
12.1.11	Possibili domande d'esame	101
12.1.12	Collegamenti teorici	101
12.2	Albero di ricoprimento minimo e algoritmo di Prim	101
12.2.1	Definizione	101
12.2.2	Intuizione	102
12.2.3	Motivazione	102
12.2.4	Proprieta' teoriche	102
12.2.5	Algoritmo	103
12.2.6	Esempio guidato	103
12.2.7	Grafico o diagramma	104
12.2.8	Dimostrazione	105
12.2.9	Analisi della complessita'	106
12.2.10	Errori tipici d'esame	107
12.2.11	Possibili domande d'esame	107
12.2.12	Collegamenti teorici	107
12.3	Cammini minimi da singola sorgente e algoritmo di Dijkstra	108
12.3.1	Definizione	108
12.3.2	Intuizione	108
12.3.3	Motivazione	108
12.3.4	Proprieta' teoriche	109
12.3.5	Algoritmo	110
12.3.6	Esempio guidato	110
12.3.7	Grafico o diagramma	111
12.3.8	Dimostrazione	112
12.3.9	Analisi della complessita'	113
12.3.10	Errori tipici d'esame	114
12.3.11	Possibili domande d'esame	114
12.3.12	Collegamenti teorici	114
12.4	Matroidi	114
12.4.1	Definizione	114
12.4.2	Intuizione	115

12.4.3	Motivazione	115
12.4.4	Proprieta' teoriche	115
12.4.5	Algoritmo	116
12.4.6	Esempio guidato	117
12.4.7	Grafico o diagramma	117
12.4.8	Dimostrazione	118
12.4.9	Analisi della complessita'	120
12.4.10	Errori tipici d'esame	120
12.4.11	Possibili domande d'esame	121
12.4.12	Collegamenti teorici	121
12.5	Programmazione di attivita' unitarie con penalita'	121
12.5.1	Definizione	121
12.5.2	Intuizione	122
12.5.3	Motivazione	122
12.5.4	Proprieta' teoriche	122
12.5.5	Algoritmo	123
12.5.6	Esempio guidato	123
12.5.7	Grafico o diagramma	124
12.5.8	Dimostrazione	124
12.5.9	Analisi della complessita'	125
12.5.10	Errori tipici d'esame	126
12.5.11	Possibili domande d'esame	126
12.5.12	Collegamenti teorici	127
12.6	Altri problemi greedy citati nelle slide	127
12.6.1	Definizione	127
12.6.2	Intuizione	127
12.6.3	Motivazione	127
12.6.4	Proprieta' teoriche	127
12.6.5	Algoritmo	127
12.6.6	Esempio guidato	128
12.6.7	Grafico o diagramma	128
12.6.8	Dimostrazione	128
12.6.9	Analisi della complessita'	128
12.6.10	Errori tipici d'esame	129
12.6.11	Possibili domande d'esame	129
12.6.12	Collegamenti teorici	129
12.7	Esercizi ufficiali della lezione	129
12.7.1	Definizione	129
12.7.2	Intuizione	129
12.7.3	Motivazione	130
12.7.4	Proprieta' teoriche	130

12.7.5	Algoritmo	130
12.7.6	Esempio guidato	131
12.7.7	Grafico o diagramma	132
12.7.8	Dimostrazione	132
12.7.9	Analisi della complessita'	133
12.7.10	Errori tipici d'esame	133
12.7.11	Possibili domande d'esame	133
12.7.12	Collegamenti teorici	134
12.8	Contenuti richiesti ma non presenti nella slide Greedy	134
12.8.1	Definizione	134
12.8.2	Intuizione	134
12.8.3	Motivazione	134
12.8.4	Proprieta' teoriche	134
12.8.5	Algoritmo	135
12.8.6	Esempio guidato	135
12.8.7	Grafico o diagramma	135
12.8.8	Dimostrazione	135
12.8.9	Analisi della complessita'	135
12.8.10	Errori tipici d'esame	135
12.8.11	Possibili domande d'esame	135
12.8.12	Collegamenti teorici	136
13	Può tornare utile	136
13.1	Appendice A (Sommatorie)	136
13.1.1	Formule fondamentali	136
13.1.2	Serie geometrica	136
13.1.3	Serie armonica	137
13.1.4	Tecniche di manipolazione	137
13.1.5	Approssimazione con integrali	138
13.2	Appendice B (Insiemi, relazioni, funzioni, grafi, alberi)	138
13.2.1	Insiemi	138
13.2.2	Relazioni	138
13.2.3	Funzioni	139
13.2.4	Grafi	139
13.2.5	Alberi	140
13.3	Appendice C (Calcolo combinatorio e probabilità)	141
13.3.1	Principi di conteggio	141
13.3.2	Permutazioni e combinazioni	141
13.3.3	Probabilità	142
13.3.4	Probabilità condizionata e indipendenza	143
13.3.5	Variabili casuali e valore atteso	143

13.4	Appendice D (Matrici)	144
13.4.1	Definizioni base	144
13.4.2	Operazioni fondamentali	144
13.4.3	Proprietà del prodotto	145
13.4.4	Determinante e matrice inversa	145
13.4.5	Applicazioni agli algoritmi	146
13.5	Esercizi proposti in aula	146
13.5.1	Confronto tra BST e Min-Heap	146
13.6	Elencare ordinatamente le chiavi: BST vs Min-Heap	146
13.6.1	BST: visita in-order in $O(n)$	146
13.6.2	Min-Heap: impossibile in $O(n)$	147
13.7	Domande d'esame (Appello 2)	148

1. Fondamenti

Questa sezione introduce i concetti fondamentali necessari per comprendere il resto del corso. Non preoccuparti se all'inizio alcune definizioni sembrano astratte: diventeranno chiare con gli esempi concreti nelle sezioni successive.

1.1 Introduzione al corso

1.1.1 Requisiti

- Fondamenti di matematica discreta e grafi
- Calcolo delle probabilità
- Basi di C $\rightarrow$ si studia Java (Classi e Oggetti)¹
- Le procedure ricorsive sono come l'Ave Maria (STUDIA: c'è nell'esame)

Nota

La ricorsione è **fondamentale** in questo corso. Se non ti senti sicuro, ripassa prima di procedere: fibonacci, fattoriale, e i concetti base di chiamate ricorsive.

1.1.2 Programma del corso

- Divide et Impera
- Programmazione dinamica
- Algoritmi Greedy
- Grafi $\rightarrow$ Nodi e archi

Questi quattro argomenti sono le **tecniche algoritmiche** principali. Ogni tecnica è un "modo di pensare" per risolvere problemi complessi:

- **Divide et Impera:** spezza il problema in pezzi piccoli
- **Programmazione Dinamica:** memorizza soluzioni parziali per evitare ricalcoli
- **Greedy:** fai sempre la scelta localmente migliore
- **Grafi:** rappresenta problemi come reti di connessioni

¹Java è il male

1.1.3 Struttura del modulo

48 ore di lezione: 4 ore a settimana + 9 ore di studio personale.² Non usare ChatGPT o altri strumenti simili.

I lucidi non bastano (purtroppo).

Se proprio devi, usa **Anthropic CLAUDE**.

1.1.4 Modalità di esame

- **Prima parte:** 15 domande a risposta multipla (almeno 4 su codice Java – CODE RUNNER)
- **Seconda parte:** Progettare una soluzione algoritmica
- **Se va male:** Può fare orale se ha dubbi (Ave o Maria...)

1.2 Il ruolo degli algoritmi nell'informatica

Prima di tuffarci nei dettagli tecnici, capiamo **perché** gli algoritmi sono importanti. Un algoritmo è semplicemente una "ricetta" per risolvere un problema al computer.

1.2.1 Cos'è un algoritmo?

Definizione 1.1 (Algoritmo). Procedura di calcolo ben definita: dato un input (anche nullo), produce un output in tempo finito.

Nota

In parole semplici: un algoritmo è una sequenza di passi precisi che trasforma dei dati di partenza (input) in un risultato (output). Deve sempre finire in tempo ragionevole.

Esempio quotidiano: una ricetta di cucina è un algoritmo:

- **Input:** ingredienti (farina, uova, zucchero)
- **Passi:** mescola, cuoci a 180°C per 30 minuti
- **Output:** torta

Definizione 1.2 (Problema computazionale). Relazione input/output desiderata. L'algoritmo specifica *come* ottenerla.

Differenza chiave:

- **Problema:** “Voglio ordinare una lista di numeri” (COSA)
- **Algoritmo:** “Confronta elementi a coppie e scambiali se necessario” (COME)

Il problema dice *cosa* vogliamo ottenere, l'algoritmo dice *come* farlo.

²Si, lo so, è una noia, ma serve.

Definizione 1.3 (Algoritmo corretto). Termina producendo il risultato corretto per ogni istanza valida.

Osservazione 1. Non sempre esiste un algoritmo: il problema della fermata è indecidibile.

Nota

Cosa significa “ind decidibile”? Esistono problemi per cui è matematicamente impossibile scrivere un algoritmo generale che funzioni sempre. Il famoso “problema della fermata” chiede: “Dato un programma qualsiasi, terminerà mai?” È stato dimostrato che non esiste un algoritmo che possa rispondere a questa domanda per tutti i programmi possibili.

1.3 Esempio di algoritmo ricorsivo

Ora vediamo un esempio concreto: calcolare i numeri di Fibonacci. Questo ci permetterà di capire la differenza tra un algoritmo **corretto** (che funziona) e uno **efficiente** (che funziona velocemente).

Nota

Cosa sono i numeri di Fibonacci?

È una sequenza di numeri dove ogni numero è la somma dei due precedenti:

$$0, 1, 1, 2, 3, 5, 8, 13, 21, 34, \dots$$

Definizione formale:

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \text{ per } n \geq 2$$

Esempio: $F_5 = F_4 + F_3 = 3 + 2 = 5$

Nota

Le convenzioni usate per il pseudocodice sono a pag. 19 del [CLRS].

Algorithm 1: FibonacciR

Input: $n \geq 0$

Output: F_n

```
1 if  $n = 0$  then
2   return 0;
3 if  $n = 1$  then
4   return 1;
5 return FibonacciR( $n - 1$ ) + FibonacciR( $n - 2$ );
```

Come funziona questo algoritmo?

Traduce direttamente la definizione matematica in codice:

1. Se $n = 0$ restituisci 0 (caso base)
2. Se $n = 1$ restituisci 1 (caso base)
3. Altrimenti, calcola F_{n-1} e F_{n-2} ricorsivamente e sommal

Esempio per $n = 4$:

- Calcola $F_3 + F_2$
- $F_3 = F_2 + F_1$, quindi serve calcolare F_2 due volte!
- $F_2 = F_1 + F_0 = 1 + 0 = 1$
- Risultato finale: $F_4 = 3$

L'algoritmo è corretto? **Sì!**

Per ogni input valido, termina? **Sì, assumendo che input sia sempre un intero $n \geq 0$**

Se sì, calcola il valore giusto? **Sì, perché l'algoritmo codifica la definizione ricorsiva.**

Quanto **tempo** richiede per terminare?

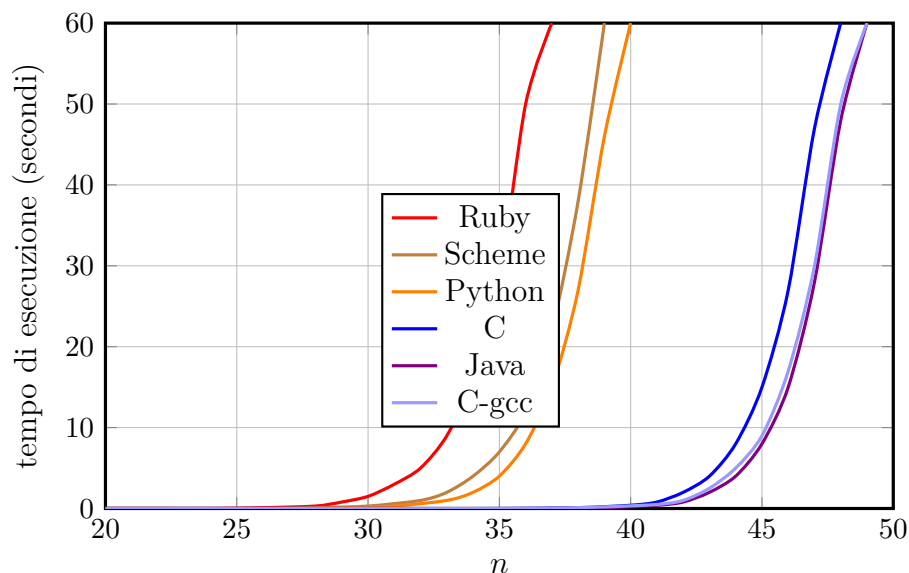


Figura 1. Confronto dei tempi di esecuzione dell'algoritmo ricorsivo `FibonacciR` in diversi linguaggi. I linguaggi interpretati (Ruby, Scheme, Python) saturano a valori di n molto più bassi rispetto ai linguaggi compilati (C, Java, C-gcc).

Problema grave: anche se l'algoritmo è corretto, è **terribilmente lento**.

Per $n = 50$ impiega diversi minuti, mentre un algoritmo migliore lo calcola istantaneamente.

Il problema è che ricalcola gli stessi valori moltissime volte.

Calcolo della complessità computazionale

La **complessità** misura quanto “costa” eseguire un algoritmo. Non misuriamo i secondi (dipendono dal computer), ma il numero di operazioni in funzione della dimensione dell’input.

Definizione 1.4 (Efficienza). Risorse (tempo e spazio) richieste in funzione della dimensione n dell’input.

Definizione 1.5 (Algoritmo vs. Implementazione). L’algoritmo è un concetto **astratto** ($\neq$ implementazione): descrive *come* risolvere un problema indipendentemente dal linguaggio. L’implementazione è un **programma concreto**, ovvero la traduzione dell’algoritmo in un linguaggio comprensibile dalla macchina.

Nota

Analogia:

- **Algoritmo** = ricetta scritta in italiano (“mescola gli ingredienti”)
- **Implementazione** = ricetta tradotta in giapponese per un cuoco giapponese

L’idea (algoritmo) è la stessa, cambia solo la forma (implementazione).

Nozione 1.1 (Algoritmo efficiente). Andiamo a cercare l’algoritmo efficiente:

- **Efficiente:** usa poche risorse
- **Risorse:** tempo e spazio richiesti dall’algoritmo per produrre l’output

Teorema 1.1 (Algoritmo vs. hardware). Con $n = 10^7$ interi da ordinare:

$$t_A = \frac{2(10^7)^2}{10^{10}} \approx 5.5 \text{ ore} \quad (\text{InsSort}, 10^9 \text{ istr/s})$$

$$t_B = \frac{50 \cdot 10^7 \log_2 10^7}{10^6} \approx 20 \text{ min} \quad (\text{MergeSort}, 10^6 \text{ istr/s})$$

L’algoritmo migliore batte l’hardware più veloce.

Lezione fondamentale: un algoritmo efficiente su un computer lento batte un algoritmo inefficiente su un computer veloce.

Nell’esempio:

- Computer A: 10× più veloce, ma usa InsertionSort (algoritmo lento)
- Computer B: 10× più lento, ma usa MergeSort (algoritmo veloce)

Risultato: B finisce in 20 minuti, A impiega 5.5 ore!

Possiamo fare meglio?

Ottimizzazione: FibonacciI**Algorithm 2:** FibonacciI

Input: Intero positivo n
Output: Il valore F_n

```

1 if  $n = 0$  then
2   return 0;
3 if  $n = 1$  then
4   return 1;
5  $pprev = 0$ ;
6  $prev = 1$ ;
7 for  $i = 2$  to  $n$  do
8    $f = prev + pprev$ ;
9    $pprev = prev$ ;
10   $prev = f$ ;
11 return  $f$ ;

```

Come funziona FibonacciI (iterativo)?

Invece di chiamare ricorsivamente la funzione, mantiene in memoria solo gli ultimi due valori e li aggiorna ad ogni passo:

- Inizia con $F_0 = 0$ e $F_1 = 1$
- Calcola $F_2 = F_1 + F_0 = 1$
- Calcola $F_3 = F_2 + F_1 = 2$
- Calcola $F_4 = F_3 + F_2 = 3$
- E così via fino a F_n

Vantaggio: ogni numero viene calcolato **una sola volta**, non milioni di volte come nella versione ricorsiva!

La complessità è:

$$T(0) = 2, \quad T(1) = 3, \quad T(n) = 4 + 2n + 4(n-1) + 1 = 6n + 1$$

La crescita è **lineare in n** .

Algoritmo	Complessità	$n = 50$
Ricorsivo	$(1.4)^n$	minuti
Iterativo	$\Theta(n)$	istantaneo

Confronto drammatico:

- FibonacciR (ricorsivo): per $n = 50$ fa circa $2^{25} \approx 33$ milioni di chiamate $\rightarrow$ minuti
- FibonacciI (iterativo): per $n = 50$ fa esattamente 50 iterazioni $\rightarrow$ istantaneo

Questo dimostra che due algoritmi che risolvono lo stesso problema possono avere prestazioni **radicalmente diverse**.

Modello RAM: ogni operazione elementare ha costo 1; parola di memoria a dimensione fissa.

Nota

Il **modello RAM** (Random Access Machine) è una semplificazione che assumiamo per analizzare gli algoritmi:

- Ogni operazione base (somma, confronto, assegnazione) costa 1
- La memoria è infinita e accedervi costa 1
- Un solo processore esegue un'istruzione alla volta

Non è realistico (nella realtà ci sono cache, processori multipli, ecc.), ma è utile per confrontare algoritmi in modo uniforme.

1.4 Esempi di problemi computazionali da risolvere in aula

Ora vediamo alcuni problemi classici che ti aiuteranno a capire come si progettano e analizzano gli algoritmi. Studia bene questi esempi: sono fondamentali per l'esame!

Indici dei due numeri che sommati danno N

Problema: Data una matrice di interi `nums` e un intero `target`, restituisci gli indici dei due numeri tali che la loro somma sia uguale a `target`. Puoi assumere che ogni input abbia esattamente una soluzione, e non puoi usare lo stesso elemento due volte. Puoi restituire la risposta in qualsiasi ordine.

Nota

In parole semplici: hai un array di numeri e un numero obiettivo. Devi trovare due numeri nell'array che sommati danno il numero obiettivo, e restituire le loro posizioni.

Esempio:

- Input: `nums = [2, 7, 11, 15]`, `target = 9`
- Output: `[0, 1]`
- Spiegazione: `nums[0] + nums[1] = 2 + 7 = 9`

Algorithm 3: twoSum (Brute Force)

Input: `nums[0...n-1]`, `target`
Output: `{i, j}` tali che `nums[i] + nums[j] = target`, $i \neq j$

```

1 for  $i = 0$  to  $n - 2$  do
2   for  $j = i + 1$  to  $n - 1$  do
3     if nums[i] + nums[j] == target then
4       return {i, j};

```

Come funziona l'algoritmo?

Prova tutte le possibili coppie di numeri:

1. Fisso il primo numero (indice i)
2. Controllo tutti i numeri successivi (indice $j > i$)
3. Se `nums[i] + nums[j] = target`, ho trovato la coppia!

Si chiama “brute force” (forza bruta) perché prova semplicemente tutte le combinazioni possibili.

Invariante: Dopo ogni iterazione esterna, tutti gli elementi da 0 a $i - 1$ sono stati confrontati con tutti gli elementi successivi.

Nota

L'**invariante di ciclo** è una proprietà che rimane vera ad ogni iterazione. Serve per dimostrare che l'algoritmo è corretto. In questo caso: “alla fine dell'iterazione i , abbiamo controllato tutte le coppie che iniziano con indici da 0 a i .”

Complessità:

- Caso migliore: $\Theta(1)$ (se la coppia è trovata subito)
- Caso peggiore: $\Theta(n^2)$

Calcolo della complessità temporale (caso peggiore):

L'algoritmo esegue due cicli annidati. Il ciclo esterno itera $n - 1$ volte, e per ogni i il ciclo interno itera $n - 1 - i$ volte. Quindi:

$$T(n) = \sum_{i=0}^{n-2} (n - 1 - i) = \sum_{k=1}^{n-1} k = \frac{(n-1)n}{2} \in \Theta(n^2)$$

Perché $\Theta(n^2)$?

Il ciclo esterno fa n passi, e per ognuno il ciclo interno fa in media $n/2$ passi. In totale: $n \times n/2 = n^2/2$ operazioni. Siccome ignoriamo le costanti, rimane n^2 .

Esempio concreto: con $n = 100$ elementi, l'algoritmo fa circa $100 \times 100/2 = 5000$ confronti.

La complessità spaziale è $\Theta(1)$, poiché usiamo solo variabili costanti.

Codice JAVA

```
class twoSum {
    public int[] twoSum(int[] nums, int target) {
        for (int i = 0; i < nums.length; i++) {
            for (int j = i + 1; j < nums.length; j++) {
                if (nums[i] + nums[j] == target) {
                    return new int[]{i, j};
                }
            }
        }
        return new int[]{}; // nessuna soluzione trovata
    }
}
```

Nota**Nota sul codice Java:**

- `nums.length` restituisce la lunghezza dell'array
- `new int[]{i, j}` crea un nuovo array con due elementi
- Il `return` interrompe immediatamente l'esecuzione quando trova la soluzione

Problema dell'inversione numerica

Questo problema insegna a manipolare le cifre di un numero usando aritmetica modulare. È un classico per imparare operazioni su interi.

Algorithm 4: Inversione numerica**Input:** Intero x rappresentato con segno a 32 bit**Output:** Intero y con le cifre di x invertite

```

1  $y = 0;$  // Inizializza risultato
2  $sign = 1;$  // Memorizza segno
3  $sign = (x < 0) ? -1 : 1;$  // Memorizza segno di x
4  $x = |x|;$  // Rende x positivo
5 while  $x > 0$  do
6    $digit = x \bmod 10;$  // Estrae ultima cifra
7    $y = y \times 10 + digit;$  // Aggiunge a y
8    $x = \lfloor x/10 \rfloor;$  // Rimuove cifra
9 return  $y \times sign;$  // Restituisce con segno

```

Come funziona l'inversione?**Esempio:** inverti 1234

1. $digit = 1234 \bmod 10 = 4$, $y = 0 \times 10 + 4 = 4$, $x = 123$
2. $digit = 123 \bmod 10 = 3$, $y = 4 \times 10 + 3 = 43$, $x = 12$
3. $digit = 12 \bmod 10 = 2$, $y = 43 \times 10 + 2 = 432$, $x = 1$
4. $digit = 1 \bmod 10 = 1$, $y = 432 \times 10 + 1 = 4321$, $x = 0$

Trucchi usati:

- $n \bmod 10$ estrae l'ultima cifra
- $\lfloor n/10 \rfloor$ rimuove l'ultima cifra
- Moltiplicare per 10 e sommare "costruisce" il numero invertito

Complessità. Sia k il numero di cifre decimali di $|x|$. Allora

$$T(k) = \begin{cases} 6 & \text{se } x = 0, \\ 5k + 6 & \text{se } x > 0, \\ 5k + 8 & \text{se } x < 0, \end{cases} \Rightarrow T(k) = \mathcal{O}(k).$$

La complessità dell'algoritmo è lineare nel numero di cifre.

Nota

Se il numero ha k cifre, il ciclo while viene eseguito esattamente k volte (una per ogni cifra). Quindi la complessità è $\mathcal{O}(k)$, dove $k = \log_{10}(n)$ per un numero n .

Esempio: il numero 12345 ha 5 cifre, quindi $k = 5$ e il ciclo fa 5 iterazioni.

Nota

L'**operatore ternario** è definito come:

$x = (\text{espressione booleana}) ? v1 : v2$

- se l'espressione booleana è vera viene usata $v1$
- se l'espressione booleana è falsa viene usata $v2$

Esempio: $\max = (a > b) ? a : b$ assegna a $\max$ il valore più grande tra a e b .

Container con più acqua

Questo è un problema di ottimizzazione: dobbiamo trovare il rettangolo di area massima. Introduciamo la tecnica dei “due puntatori” che è molto utile per problemi su array.

Si consideri un array di interi non negativi $height$ di lunghezza n . Per ogni i , la retta verticale ha estremi $(i, 0)$ e $(i, height[i])$. Si vogliono trovare due rette che, insieme all'asse x , formino un contenitore che possa contenere la massima quantità d'acqua.

Nel nostro esempio poniamo

$$A = [4, 6, 3, 5].$$

L'area del contenitore formato dalle rette in posizione i e j ($i < j$) è data da

$$\text{area}(i, j) = (j - i) \min(A[i], A[j]).$$

Nota**Perché questa formula?**

L'area di un rettangolo è: $\text{base} \times \text{altezza}$

- **Base** = distanza orizzontale = $(j - i)$
- **Altezza** = il lato più basso = $\min(A[i], A[j])$

Usiamo il minimo perché l'acqua “trabocca” dal lato più basso, quindi l'altezza effettiva è limitata dalla barra più corta.

Per $A = [4, 6, 3, 5]$ otteniamo:

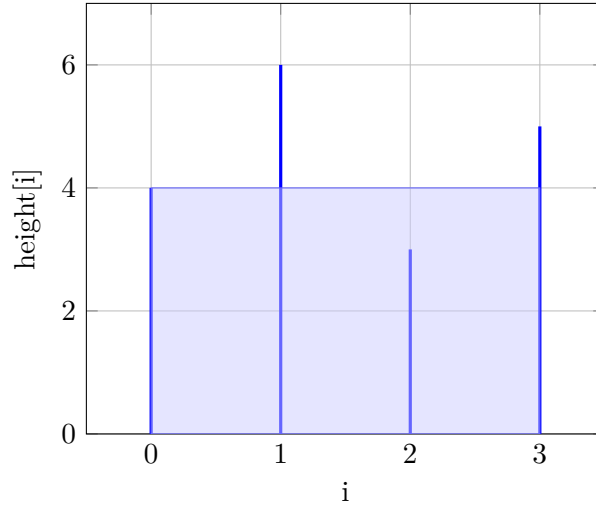


Figura 2. Container con area massima per $A = [4, 6, 3, 5]$.

Il valore massimo è quindi

$$\max_{0 \leq i < j \leq 3} \text{area}(i, j) = 12,$$

ottenuto per la coppia di indici $(i, j) = (0, 3)$.

Nell'esempio: $\text{area}(0, 3) = (3 - 0) \times \min(4, 5) = 3 \times 4 = 12$ è la massima possibile.

Quindi il pseudocodice per questo problema computazionale è il seguente:

Algorithm 5: Container With Most Water

Input: Array $\text{height}[0 \dots n - 1]$ di interi non negativi

Output: Massima area d'acqua contenibile

```

1  $left \leftarrow 0;$                                      // Puntatore sinistro
2  $right \leftarrow n - 1;$                              // Puntatore destro
3  $maxArea \leftarrow 0;$                                // Area massima trovata finora
4 while  $left < right$  do
5    $width \leftarrow right - left;$                      // Larghezza del contenitore
6    $h \leftarrow \min(\text{height}[left], \text{height}[right]);$  // Altezza = lato più basso
7    $area \leftarrow width \cdot h;$                      // Area del contenitore corrente
8    $maxArea \leftarrow \max(maxArea, area);$              // Aggiorna massimo
9   if  $\text{height}[left] < \text{height}[right]$  then
10     $left \leftarrow left + 1;$                        // Sposta il lato più basso verso il centro
11  else
12     $right \leftarrow right - 1;$                      // Sposta il lato più basso verso il centro
13 return  $maxArea;$                                    // Restituisce l'area massima

```

Tecnica dei due puntatori:

1. Partiamo dai due estremi dell'array (left=0, right=n-1)
2. Calcoliamo l'area del contenitore corrente
3. Spostiamo verso il centro il puntatore che punta alla barra **più bassa**
4. Ripetiamo fino a quando i puntatori si incontrano

Perché muoviamo il lato più basso? Se muovessimo il lato più alto, la larghezza diminuirebbe e l'altezza resterebbe limitata dal lato basso: l'area peggiorerebbe sicuramente. Muovendo il lato basso abbiamo la *possibilità* di trovare un lato più alto che migliori l'area.

Complessità. Ad ogni iterazione del ciclo **while** viene spostato solo uno dei due puntatori, che possono muoversi ciascuno al più $n - 1$ volte. Pertanto il numero di iterazioni è al più $n - 1$ e il costo è

$$T(n) = c_1 + c_2(n - 1) = \Theta(n).$$

Lo spazio aggiuntivo utilizzato è costante, quindi $S(n) = \Theta(1)$.

Nota

Complessità $\Theta(n)$ significa che l'algoritmo è **lineare**: raddoppiando la dimensione dell'input, il tempo raddoppia. Molto meglio di $\Theta(n^2)$ dove raddoppiando l'input il tempo quadruplica!

Prefisso più lungo comune fra più sequenze

Questo problema è utile per capire come elaborare stringhe. L'idea è trovare la parte iniziale condivisa da tutte le stringhe in input.

Algorithm 6: Prefisso comune più lungo

Input: Array $S[1 \dots n]$ di stringhe
Output: Prefisso più lungo comune a tutte le stringhe
// Assumo che le stringhe di S siano tutte diverse da null

```

1  $p \leftarrow S[1];$                                 // Prefisso corrente
2 for  $i \leftarrow 2$  to  $n$  do
3    $p \leftarrow \text{PrefissoComune}(p, S[i]);$           // Prefisso tra  $p$  e  $S[i]$ 
4   if  $p = \text{stringa vuota}$  then
5     return  $p;$                                 // Nessun prefisso comune
6 return  $p;$                                     // Prefisso comune più lungo
```

Come funziona?

Esempio: $S = [\text{"fiore"}, \text{"fiorire"}, \text{"fioraio"}]$

1. $p = \text{"fiore"}$ (prima stringa)
2. $\text{PrefissoComune}(\text{"fiore"}, \text{"fiorire"}) = \text{"fiori"}$
3. $\text{PrefissoComune}(\text{"fiori"}, \text{"fioraio"}) = \text{"fior"}$
4. Risultato: "fior"

L'idea è semplice: confrontiamo il prefisso corrente con ogni stringa successiva, accorciandolo se necessario.

Invariante: Dopo ogni iterazione del ciclo, p è il prefisso comune più lungo delle prime i stringhe.

Complessità. Sia n il numero di stringhe in S e sia m la lunghezza della stringa più corta. Ad ogni passo del ciclo la funzione **PrefissoComune** confronta al più m caratteri, quindi

$$T(n, m) = c_1 + c_2(n - 1)m = \Theta(nm).$$

Lo spazio aggiuntivo utilizzato è costante, dunque $S(n, m) = \Theta(1)$.

Nota

La complessità $\Theta(nm)$ significa che nel caso peggiore confrontiamo m caratteri per ognuna delle n stringhe. Se abbiamo 100 stringhe di 50 caratteri, facciamo al massimo $100 \times 50 = 5000$ confronti.

2. InsertionSort e MergeSort

Ora studiamo due algoritmi di ordinamento fondamentali. InsertionSort è semplice ma lento, MergeSort è più complesso ma veloce. Capirai la differenza tra approccio **incrementale** e **divide et impera**.

2.0.1 InsertionSort

Inserisce ogni elemento nella posizione corretta tra quelli già ordinati (come le carte da gioco).

Nota

Analogia con le carte: quando ordini una mano di carte, prendi una carta alla volta e la inserisci nella posizione giusta tra quelle già ordinate. Questo è esattamente quello che fa InsertionSort!

Algorithm 7: InsertionSort

Input: $A = \langle a_1, \dots, a_n \rangle$, array di elementi confrontabili con $\leq$

Output: A ordinato rispetto a $\leq$

```
1 for  $i = 2$  to  $n$  do
2    $key = A[i]$  ;           // Inserisce  $A[i]$  nella parte ordinata  $a_1, \dots, a_{i-1}$ 
3    $j = i - 1$ ;
4   while  $j > 0$  and  $A[j] > key$  do
5      $A[j + 1] = A[j]$ ;
6      $j = j - 1$ ;
7    $A[j + 1] = key$ ;
8 return  $A$ 
```

Come funziona InsertionSort passo p

asso:

1. Parte dall'elemento in posizione 2 (il primo è già "ordinato")
2. Prende l'elemento corrente (**key**)
3. Lo confronta con gli elementi precedenti (da destra a sinistra)
4. Sposta gli elementi più grandi verso destra
5. Inserisce **key** nella posizione corretta
6. Ripete per tutti gli elementi

Nota

Ordina gli elementi **sul posto** (in-place), cioè non usa array aggiuntivi ma modifica direttamente l'array di input.

Ciclo Algoritmo

$$\begin{bmatrix} 5 & 2 & 4 & 6 & 1 \end{bmatrix} = A$$

Ogni blocco rappresenta un ciclo for (0 o più cicli while)

1	2	3	4	5	Commento
5 j	2 i	4	6	1	Seq. ord.: 5 Scambio 5→2
2 ←	5 j	4 i	6	1	Seq. ord.: 2,5 Scambio 5→4
2	4 ←	5 j	6 i	1	Seq. ord.: 2,4,5 Nessun scambio
2	4	5 ←	6 j	1 i	Seq. ord.: 2,4,5,6 1 → pos. 1
1	2	4	5	6 i	Seq. ordinata

Nella tabella:

- **Rosso** = elemento che stiamo inserendo (**key**)
- i = indice elemento corrente
- j = indice di confronto (parte da $i - 1$ e va a ritroso)
- Le frecce ← mostrano il movimento degli elementi verso destra

Invariante di ciclo

Definizione 2.1 (Invariante di ciclo). Un'asserzione (proprietà booleana) sempre vera all'inizio di ogni iterazione di un ciclo.

Nota

L'invariante di ciclo è come una **promessa** che l'algoritmo mantiene: “prima di ogni giro del ciclo, questa proprietà è vera”. Serve per dimostrare matematicamente che l'algoritmo funziona correttamente.

La validità si dimostra in tre passi:

1. **Inizializzazione:** è vera prima della prima iterazione.
2. **Conservazione:** se è vera prima di un'iterazione, rimane vera prima di quella successiva.
3. **Conclusione:** quando il ciclo termina, l'invariante fornisce una proprietà utile a dimostrare la correttezza.

Esempio 2.1 (Invariante di InsertionSort). “All'inizio del ciclo *for*, il sottoarray $A[1 : i - 1]$ è ordinato e formato dagli stessi elementi che erano in $A[1 : i - 1]$.”

- **Inizializzazione:** $i = 2$, quindi $A[1 : 1]$ è un solo elemento $\Rightarrow$ ordinato. ✓
- **Conservazione:** $\text{key} = A[i]$, il ciclo **while** sposta $A[i - 1], A[i - 2], \dots$ a destra finché non trova la posizione corretta per **key**. Quindi $A[1 : i]$ diventa ordinato con gli stessi elementi. All'iterazione successiva i aumenta di 1, l'invariante vale su $A[1 : i - 1]$. ✓
- **Conclusione:** il ciclo termina con $i = n + 1$. Per l'invariante, $A[1 : n]$ è ordinato con gli stessi elementi $\Rightarrow$ l'algoritmo è corretto. ✓

In parole semplici:

- **Inizializzazione:** all'inizio funziona (un elemento è sempre ordinato)
- **Conservazione:** se funziona al passo i , funziona anche al passo $i + 1$
- **Conclusione:** alla fine, l'intero array è ordinato

È come l'effetto domino: se il primo pezzo cade e ogni pezzo fa cadere il successivo, alla fine cadono tutti!

Modello RAM

Per analizzare la complessità si usa un modello astratto di computer: la **Random Access Machine (RAM)**.

- Un solo processore con memoria ad accesso casuale.
- Numeri interi e a virgola mobile con dimensione fissa (es. 64 bit per parola).
- Istruzioni elementari (lettura, scrittura, somma, sottrazione, moltiplicazione, divisione, confronto, if, salto, chiamata a procedura).
- Ogni istruzione ha **costo costante** = 1.

Nota

È una semplificazione: nella realtà cache, pipeline e parallelismo influenzano i tempi reali. In questo corso si ignorano questi dettagli.

Perché usiamo il modello RAM? Perché ci permette di confrontare algoritmi in modo uniforme, indipendentemente dall'hardware specifico su cui girano.

Dimensione dell'input e tempo di esecuzione

Definizione 2.2 (Dimensione dell'input). Misura della quantità di bit/word necessari per rappresentare l'input. Un indicatore è **corretto** se la dimensione reale è in relazione polinomiale con esso.

Definizione 2.3 (Tempo di esecuzione). Numero totale di operazioni elementari eseguite dall'algoritmo nel modello RAM, in funzione della dimensione n dell'input.

Nota

Indicatore corretto: una misura semplificata della dimensione dell'input che mantiene una relazione ragionevole con la dimensione reale.

Esempi:

- Per un array di n numeri: indicatore = n (numero di elementi)
- Per una matrice $n \times n$: indicatore = n (anche se contiene n^2 elementi)
- Per un numero intero: indicatore = numero di bit necessari a rappresentarlo

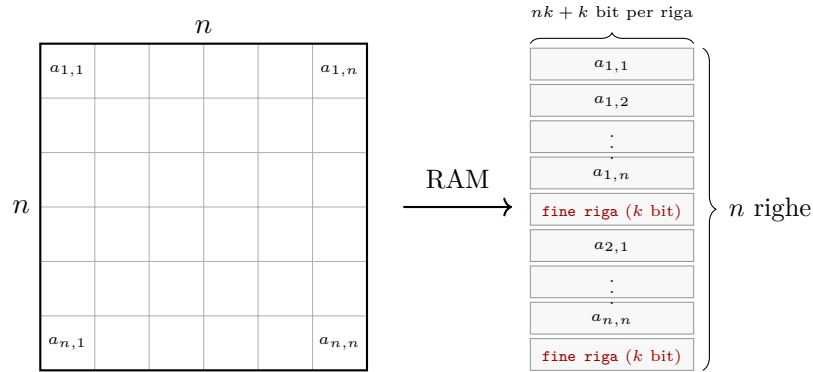
Esempi di indicatori corretti:

- **Array di n elementi:** indicatore = n . ✓
- **Matrice $n \times n$:** indicatore = n (la dimensione reale $n \cdot k(n+1)$ è polinomiale in n). ✓
- **Intero n :** indicatore = $\lceil \lg n \rceil + 1$ (numero di bit). **Non** si può usare n come indicatore perché $2^{|I|} \approx n$, relazione esponenziale. ✓

Esempio 2.2 (Matrice $n \times n$). Sia A una matrice $n \times n$ di interi, dove ogni intero richiede k bit. La RAM memorizza la matrice riga per riga.

Ogni riga occupa nk bit per gli interi, più k bit per il marcatore di fine riga. L'intera matrice richiede:

$$|I| = n(n+1)k \text{ bit} = n(kn+k) \text{ bit} = \boxed{kn^2 + kn \text{ bit}} = \Theta(n)$$



Si usa come indicatore $|I| = n$. È **corretto** perché la dimensione reale $kn^2 + kn$ è un **polinomio** in n .

Nota

L'errore comune è usare il *valore* di un intero n come dimensione dell'input invece del numero di bit necessari a rappresentarlo. Questo porta a sottostimare la complessità.

Esempio: il numero 1000 ha valore 1000, ma occupa solo $\lceil \log_2(1000) \rceil = 10$ bit. La dimensione dell'input è 10, non 1000!

Analisi di InsertionSort

c_k = costo esecuzione istruzione alla riga k . $t_i - 1 = \#$ cicli **while** nel i -esimo ciclo **for**.

INSERTION-SORT(A, n)	<i>cost</i>	<i>times</i>
1 for $i = 2$ to n	c_1	n
2 $key = A[i]$	c_2	$n - 1$
3 // Insert $A[i]$ into the sorted subarray $A[1 : i - 1]$	0	$n - 1$
4 $j = i - 1$	c_4	$n - 1$
5 while $j > 0$ and $A[j] > key$	c_5	$\sum_{i=2}^n t_i$
6 $A[j + 1] = A[j]$	c_6	$\sum_{i=2}^n (t_i - 1)$
7 $j = j - 1$	c_7	$\sum_{i=2}^n (t_i - 1)$
8 $A[j + 1] = key$	c_8	$n - 1$

Contare le iterazioni:

for $i = \ell$ **to** n $\Rightarrow$ #Cicli = $n - \ell + 1$

Esempio: **for** $i = 2$ **to** n $\Rightarrow$ $n - 1$ iterazioni

$$T(n) = c_1 n + c_2(n - 1) + c_4(n - 1) + c_5 \sum_{i=2}^n t_i + c_6 \sum_{i=2}^n (t_i - 1) + c_7 \sum_{i=2}^n (t_i - 1) + c_8(n - 1)$$

Indichiamo con c_k il costo dell'istruzione alla riga k , e con t_i il numero di volte che il ciclo **while** viene eseguito durante l' i -esimo ciclo **for**.

Nota

t_i dipende dai dati:

- Se $A[i]$ è già al posto giusto: $t_i = 1$ (un solo confronto)
- Se $A[i]$ deve andare all'inizio: $t_i = i$ (confronta con tutti)

Caso migliore: array già ordinato

Se l'array è già ordinato, il corpo del **while** non viene mai eseguito: $t_i = 1$ per ogni i . Le somme $\sum(t_i - 1) = 0$, quindi:

$$\begin{aligned} T(n) &= c_1n + c_2(n-1) + c_4(n-1) + c_5(n-1) + c_8(n-1) \\ &= \underbrace{(c_1 + c_2 + c_4 + c_5 + c_8)}_a n - \underbrace{(c_2 + c_4 + c_5 + c_8)}_b \\ &= an - b \quad \Rightarrow \quad \boxed{T(n) = \Theta(n)} \end{aligned}$$

Nota

Nel caso migliore la crescita è **lineare**: ogni elemento viene confrontato una sola volta con il suo predecessore e nessuno scambio viene effettuato.

Esempio pratico: se l'array è $[1, 2, 3, 4, 5]$, l'algoritmo fa solo $n - 1$ confronti e nessuno spostamento. Velocissimo!

Caso peggiore: array ordinato al contrario

Se l'array è in ordine decrescente, ogni $A[i]$ deve essere confrontato con **tutti** gli elementi in $A[1 : i - 1]$, quindi $t_i = i$. Usando $\sum_{i=2}^n i = \frac{n(n+1)}{2} - 1$:

$$T(n) = c_1n + c_2(n-1) + c_4(n-1) + c_5\left(\frac{n(n+1)}{2} - 1\right) + (c_6 + c_7)\frac{n(n-1)}{2} + c_8(n-1)$$

Raccogliendo i termini per grado:

$$\begin{aligned} T(n) &= \underbrace{\frac{c_5 + c_6 + c_7}{2}}_a n^2 + \underbrace{\left(c_1 + c_2 + c_4 + \frac{c_5 - c_6 - c_7}{2} + c_8\right)}_b n - \underbrace{(c_2 + c_4 + c_5 + c_8)}_c \\ &\Rightarrow \quad \boxed{T(n) = \Theta(n^2)} \end{aligned}$$

Nota

Nel caso peggiore la crescita è **quadratica**: ogni nuovo elemento deve “scendere” fino in fondo alla parte già ordinata, confrontandosi con tutti gli elementi precedenti.

Esempio pratico: se l'array è $[5, 4, 3, 2, 1]$:

- Il 4 fa 1 confronto
- Il 3 fa 2 confronti
- Il 2 fa 3 confronti

- L'1 fa 4 confronti

Totale: $1 + 2 + 3 + 4 = 10 = \frac{n(n-1)}{2}$ confronti. Molto lento!

Caso	Input	Complessità
Migliore	Array già ordinato	$\Theta(n)$
Peggior	Array ordinato inverso	$\Theta(n^2)$

Solitamente si considera come complessità di un algoritmo il tempo di esecuzione con input che rappresenta il caso peggiore.

1. È una garanzia sulla descrizione del comportamento dell'algoritmo.
2. In alcuni casi, il caso peggiore è il più frequente.
3. Il caso medio spesso è simile al caso peggiore (vedi InsertionSort).

Per InsertionSort: il caso medio è $\Theta(n^2)$ come il caso peggiore, perché in media metà degli elementi vanno spostati.

2.1 MergeSort

MergeSort è un algoritmo di ordinamento basato sul metodo **divide et impera**, più efficiente di InsertionSort nel caso peggiore: $\Theta(n \lg n)$ contro $\Theta(n^2)$.

2.1.1 Metodo Divide et Impera

Il metodo divide et impera è un paradigma di progettazione degli algoritmi che si articola in tre fasi ricorsive:

- **Divide:** il problema di dimensione n viene suddiviso in a sotto-problemi di dimensione n/b .
- **Impera:** ogni sotto-problema viene risolto ricorsivamente. La ricorsione si ferma quando il problema è sufficientemente piccolo da essere risolto direttamente (caso base).
- **Combina:** le soluzioni dei sotto-problemi vengono unite per ottenere la soluzione del problema originale.

Nota

InsertionSort usa un approccio **incrementale**: estende la soluzione di $A[1 \dots i-1]$ inserendo $A[i]$ nella posizione corretta. MergeSort usa invece **divide et impera**: risolve due metà indipendentemente e le fonde.

2.1.2 Descrizione dell'algoritmo

Applicando divide et impera al problema dell'ordinamento:

- **Divide:** l'array $A[p \dots r]$ viene spezzato in due sotto-array $A[p \dots q]$ e $A[q+1 \dots r]$, dove $q = \lfloor (p+r)/2 \rfloor$.
- **Impera:** i due sotto-array vengono ordinati ricorsivamente con MergeSort. Quando $p = r$ (sotto-array di un solo elemento), la ricorsione termina.
- **Combina:** i due sotto-array ordinati vengono fusi dall'operazione $\text{MERGE}(A, p, q, r)$.

L'operazione chiave è MERGE: dati due sotto-array adiacenti già ordinati, li fonde in un unico sotto-array ordinato.

2.1.3 Pseudocodice

Dato $A[1 \dots n]$, MergeSort si invoca con $\text{MERGE-SORT}(A, 1, n)$.

Algorithm 8: MERGE-SORT(A, p, r)

Input: Array A , indici $p \leq r$ **Output:** $A[p \dots r]$ ordinato in loco

```
1 if  $p < r$  then
2    $q = \lfloor (p + r)/2 \rfloor$ ;
3   MERGE-SORT( $A, p, q$ );
4   MERGE-SORT( $A, q + 1, r$ );
5   MERGE( $A, p, q, r$ );
```

Algorithm 9: MERGE(A, p, q, r)

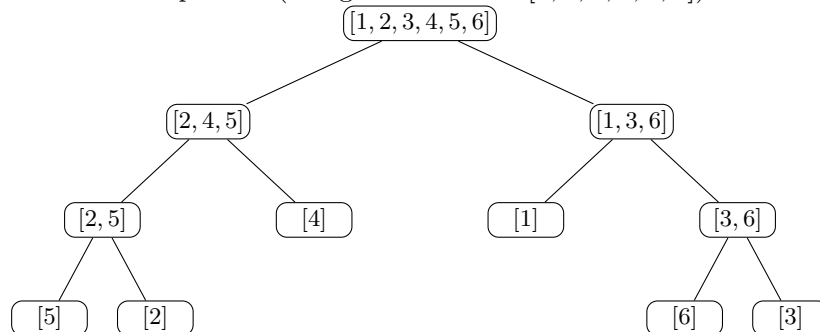
Input: $A[p \dots q]$ e $A[q + 1 \dots r]$ già ordinati**Output:** $A[p \dots r]$ ordinato

```
1  $n_L = q - p + 1$ ;
2  $n_R = r - q$ ;
3 crea array  $L[0 \dots n_L - 1]$  e  $R[0 \dots n_R - 1]$ ;
4 for  $i = 0$  to  $n_L - 1$  do
5    $L[i] = A[p + i]$ ;
6 for  $j = 0$  to  $n_R - 1$  do
7    $R[j] = A[q + 1 + j]$ ;
8  $i = 0$ ;  $j = 0$ ;  $k = p$ ;
9 while  $i < n_L$  and  $j < n_R$  do
10  if  $L[i] \leq R[j]$  then
11     $A[k] = L[i]$ ;
12     $i = i + 1$ ;
13  else
14     $A[k] = R[j]$ ;
15     $j = j + 1$ ;
16   $k = k + 1$ ;
17 while  $i < n_L$  do
18   $A[k] = L[i]$ ;
19   $i = i + 1$ ;
20   $k = k + 1$ ;
21 while  $j < n_R$  do
22   $A[k] = R[j]$ ;
23   $j = j + 1$ ;
24   $k = k + 1$ ;
```

Nota

Questa è la versione più semplice di MERGE. Nel Cormen (ed. IV) e in rete esistono versioni più compatte che usano la tecnica della **sentinella**: si aggiunge $+\infty$ in fondo a L e R , eliminando i due cicli **while** finali.

Esempio 2.3 (MergeSort su $A = [5, 2, 4, 6, 1, 3]$).



Le foglie sono i sotto-array di dimensione 1 (già ordinati). La fase di *merge* risale l'albero dal basso verso la radice, fondendo i fratelli a ogni livello. Con $n = 6$ ci sono $\lceil \lg 6 \rceil = 3$ livelli di merge, ognuno di costo $\Theta(n)$.

2.1.4 Correttezza: Invariante di ciclo di Merge

Definizione 2.4 (Invariante di MERGE). All'inizio di ogni iterazione del ciclo **while** principale, $A[p \dots k-1]$ contiene, ordinati, gli $i+j$ elementi minori di tutti gli elementi in $L[0 \dots n_L-1]$ e $R[0 \dots n_R-1]$. $L[i]$ e $R[j]$ sono gli elementi più piccoli dei rispettivi array non ancora copiati in A .

Dimostrazione. 1. **Inizializzazione:** $k = p$, quindi $A[p \dots k-1]$ è vuoto (0 elementi copiati).

Con $i = j = 0$, $L[0]$ e $R[0]$ sono i minimi non ancora copiati. L'invariante è vera. ✓

2. **Conservazione:** supponiamo $L[i] \leq R[j]$ (il caso $L[i] > R[j]$ è simmetrico). $L[i]$ è il minimo tra tutti gli elementi non ancora copiati, quindi viene copiato in $A[k]$. Incrementando k e i , l'array $A[p \dots k-1]$ contiene ora $i+j$ elementi ordinati e $L[i]$ è il nuovo minimo non copiato di L . L'invariante è mantenuta. ✓

3. **Conclusione:** il ciclo termina quando $i = n_L$ oppure $j = n_R$, cioè quando uno dei due array è esaurito. I rimanenti elementi dell'altro array (già ordinati e tutti maggiori di quelli copiati) vengono aggiunti dai due cicli **while** finali. Al termine, $A[p \dots r]$ è ordinato e contiene gli stessi elementi originali. ✓

□

2.1.5 Complessità

Complessità di Merge. Sia $n = r - p + 1$ la dimensione del sotto-array da fondere:

- Copiare in L e R : $n_L + n_R = n$ operazioni

- Ciclo **while** principale: al più n iterazioni
- Cicli **while** finali: al più n iterazioni totali

$$T_{\text{MERGE}}(n) = \Theta(n)$$

Equazione di ricorrenza di MergeSort. Con $a = 2$ sotto-problemi di dimensione $n/2$, costo di divisione $D(n) = \Theta(1)$ e costo di fusione $C(n) = \Theta(n)$:

$$T(n) = \begin{cases} \Theta(1) & \text{se } n = 1 \\ 2T(n/2) + \Theta(n) & \text{altrimenti} \end{cases}$$

Applicando il **Teorema dell'Esperto**:

$$\boxed{T(n) = \Theta(n \lg n)} \quad \text{dove } \lg = \log_2$$

Soluzione per albero di ricorsione.

Nota

Ad ogni livello i dell'albero vi sono 2^i sotto-problemi di dimensione $n/2^i$, ciascuno con costo di fusione $\Theta(n/2^i)$. Il costo totale per livello è:

$$2^i \cdot \Theta\left(\frac{n}{2^i}\right) = \Theta(n)$$

Il numero di livelli è $\lg n + 1$ (le foglie hanno dimensione 1, con $n/2^{\lg n} = 1$). Sommando su tutti i livelli:

$$T(n) = (\lg n + 1) \cdot \Theta(n) = \Theta(n \lg n)$$

2.1.6 Confronto con InsertionSort

Nota

I due algoritmi rappresentano due filosofie opposte: InsertionSort usa un approccio **incrementale**, MergeSort usa **divide et impera**. La scelta dipende dal contesto.

Proprietà	InsertionSort	MergeSort
Paradigma	Incrementale	Divide et impera
Caso migliore	$\Theta(n)$	$\Theta(n \lg n)$
Caso peggiore	$\Theta(n^2)$	$\Theta(n \lg n)$
Caso medio	$\Theta(n^2)$	$\Theta(n \lg n)$
Memoria extra	$\Theta(1)$ (in-place)	$\Theta(n)$ (array L, R)
Stabile	Sì	Sì
Adatto a input piccoli	Sì	No

Quando usare InsertionSort?

- Array piccoli ($n \leq 20-30$): la costante nascosta in $\Theta(n^2)$ è talmente piccola che batte MergeSort in pratica.
- Array **quasi ordinati**: con pochi elementi fuori posto, InsertionSort è quasi $\Theta(n)$.
- Ambienti con memoria limitata: nessun array aggiuntivo.

Quando usare MergeSort?

- Array grandi: $\Theta(n \lg n)$ diventa rapidamente più conveniente di $\Theta(n^2)$.
- Quando si vuole una **garanzia** sul caso peggiore (non dipende dall'ordine dell'input).
- Ordinamento esterno (dati su disco): MergeSort si adatta naturalmente alla fusione di blocchi.

Perché $\Theta(n \lg n)$ è asintoticamente ottimale?

Qualsiasi algoritmo di ordinamento **basato su confronti** richiede nel caso peggiore almeno $\Omega(n \lg n)$ confronti (dimostrazione tramite alberi di decisione, cap. 8 CLRS). MergeSort è quindi **asintoticamente ottimale** per questa classe di algoritmi.

InsertionSort, con il suo $\Theta(n^2)$, è lontano dall'ottimo per input grandi, ma rimane utile grazie alle sue costanti moltiplicative molto piccole e alla semplicità implementativa.

Esempio 2.4 (Crescita a confronto). Per $n = 10^6$ elementi:

- InsertionSort (caso peggiore): $\sim 10^{12}$ operazioni
- MergeSort: $\sim 2 \times 10^7$ operazioni ($n \lg_2 n \approx 2 \times 10^7$)

La differenza è di **cinque ordini di grandezza**. Su un processore da 10^9 operazioni/secondo, InsertionSort impiegherebbe ~ 16 minuti, MergeSort $\sim 0,02$ secondi.

3. Notazione Asintotica

Questa sezione introduce gli strumenti matematici per confrontare la **velocità di crescita** degli algoritmi. Non ci interessa il tempo esatto in secondi (dipende dall'hardware), ma *come* cresce il tempo all'aumentare della dimensione dell'input n .

3.1 Introduzione

Quando si analizza il tempo di computazione di un algoritmo, per input sufficientemente grandi, è rilevante soltanto il **tasso di crescita**. Studiare il tasso di crescita significa studiare l'**efficienza asintotica** degli algoritmi.

Nota

Perché “asintotica”? Ignoriamo le costanti e i termini di ordine inferiore, che contano poco per input grandi.

Esempio:

- $T(n) = 3n^2 + 100n + 5$ funzione precisa
- $T(n) \in \Theta(n^2)$ classe asintotica

Per $n = 10000$, il termine $3n^2 = 3 \cdot 10^8$ domina completamente $100n = 10^6$. La costante e i termini minori diventano irrilevanti.

Anziché dare una funzione precisa per il tempo di esecuzione, si preferisce dare una **classe di funzioni**: la notazione asintotica.

3.2 Notazione O — Limite superiore

O -grande risponde a: “*al massimo quanto cresce questo algoritmo?*”. È la notazione più usata perché fornisce una **garanzia sul caso peggiore**.

Definizione 3.1 (O -grande). Data una funzione $g(n)$:

$$O(g(n)) = \{ f(n) \mid \exists c > 0, n_0 > 0 : 0 \leq f(n) \leq c g(n) \forall n \geq n_0 \}$$

Si scrive $f(n) = O(g(n))$ (abuso di notazione per $f(n) \in O(g(n))$).

Nota

In parole semplici: f cresce *al più* come g , a meno di una costante moltiplicativa, per n abbastanza grande.

Analogia: O è come il prezzo massimo di un menù fisso. Se so che costa $O(20\text{€})$, non pagherò mai di più (a meno di una costante).

Esempio 3.1. Dimostriamo che $\frac{1}{2}n^2 + 3n \in O(n^2)$.

Dobbiamo trovare $c > 0$, $n_0 > 0$ tali che $\frac{1}{2}n^2 + 3n \leq cn^2$ per ogni $n \geq n_0$.

Dividendo per n^2 : $\frac{1}{2} + 3/n \leq c$.

Per $n \geq 7$: $\frac{1}{2} + 3/7 < 1$. Quindi $c = 1$, $n_0 = 7$. ✓

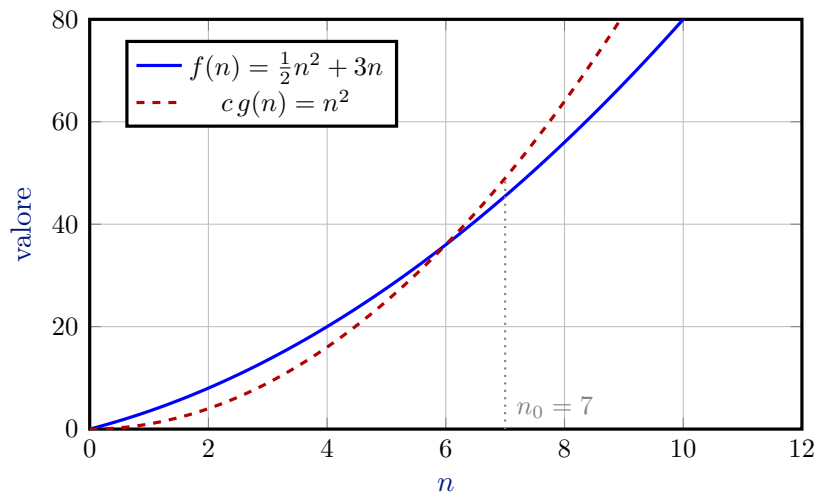


Figura 3. Da $n_0 = 7$ la curva rossa (n^2) domina quella blu ($\frac{1}{2}n^2 + 3n$), confermando $\frac{1}{2}n^2 + 3n = O(n^2)$.

O viene usata quasi sempre per i tempi di esecuzione perché è **facile da determinare**: basta trovare *un* valore di c e n_0 che funzionino — non è necessario trovare i valori ottimali.

3.3 Notazione Ω — Limite inferiore

Ω risponde a: “*almeno quanto cresce questo algoritmo?*”. Fornisce un **lower bound** sulla complessità.

Definizione 3.2 (Ω -grande). Data una funzione $g(n)$:

$$\Omega(g(n)) = \{ f(n) \mid \exists c > 0, n_0 > 0 : 0 \leq cg(n) \leq f(n) \forall n \geq n_0 \}$$

Nota

In parole semplici: f cresce *almeno* come g .

Attenzione: quando si caratterizza un tempo di esecuzione con Ω significativo (diverso da $\Omega(n)$), si deve specificare il tipo di analisi (caso peggiore o migliore), perché non esiste una prassi consolidata.

3.4 Notazione Θ — Limite stretto

Θ è la notazione più **precisa**: la funzione è “incastrata” tra due costanti di $g(n)$. Si usa quando si conosce sia il limite superiore che inferiore.

Definizione 3.3 (Θ). Data una funzione $g(n)$:

$$\Theta(g(n)) = \{ f(n) \mid \exists c_1 > 0, c_2 > 0, n_0 > 0 : 0 \leq c_1 g(n) \leq f(n) \leq c_2 g(n) \forall n \geq n_0 \}$$

Teorema 3.1. Date $f(n), g(n)$:

$$f(n) \in \Theta(g(n)) \iff f(n) \in \Omega(g(n)) \text{ e } f(n) \in O(g(n))$$

Esempio 3.2. Dimostriamo che $\frac{1}{2}n^2 - 3n \in \Theta(n^2)$.

Si devono trovare c_1, c_2, n_0 tali che $c_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq c_2 n^2$.

Dividendo per n^2 : $c_1 \leq \frac{1}{2} - 3/n \leq c_2$.

Per $n \geq 7$: scegliendo $c_1 = \frac{1}{14}$ e $c_2 = \frac{1}{2}$, la disuguaglianza è soddisfatta. ✓

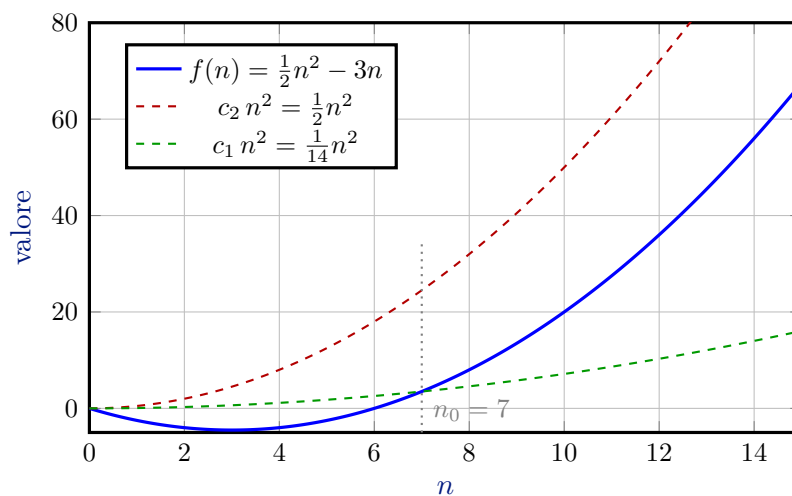


Figura 4. Per $n \geq 7$, $f(n)$ è compresa tra le due curve tratteggiate: $\frac{1}{14}n^2 \leq \frac{1}{2}n^2 - 3n \leq \frac{1}{2}n^2$, confermando $\Theta(n^2)$.

Nota

Esempio pratico — InsertionSort:

- caso migliore (array già ordinato): $\Omega(n)$
- caso peggiore (array inverso): $O(n^2)$
- se consideriamo *solo* il caso peggiore: $\Theta(n^2)$

3.5 Uso nelle equazioni e disequazioni

La notazione asintotica può comparire dentro espressioni algebriche. La convenzione è che il **lato destro** sia sempre meno preciso del sinistro: $\Theta(n)$ a destra significa “una qualche funzione in $\Theta(n)$ ”.

Osservazione 2. Si scrive $2n^2 + \Theta(n) = \Theta(n^2)$ con la seguente lettura: per ogni funzione $f(n) \in \Theta(n)$, la funzione $2n^2 + f(n)$ appartiene a $\Theta(n^2)$. Il lato sinistro è più preciso del destro: $\Theta(n)$ a destra indica “una qualche funzione in quella classe”, non un’identità tra insiemi.

Nota

Regola pratica: quando la notazione appare a sinistra dell’uguale, indica un membro anonimo della classe; a destra indica la classe complessiva. Usare $=$ è un abuso di notazione comodo ma non simmetrico: $\Theta(n) = \Theta(n^2)$ sarebbe privo di senso.

3.6 Proprietà di confronto tra funzioni

Le relazioni asintotiche si comportano in modo analogo ai confronti tra numeri reali ($\leq$, $\geq$, $=$), ma **non tutte le proprietà valgono**.

Siano f, g, h funzioni asintoticamente positive.

Teorema 3.2 (Proprietà). 1. **Transitività:** $f = O(g)$ e $g = O(h) \Rightarrow f = O(h)$ (vale anche per $\Omega, \Theta, o, \omega$).

2. **Riflessività:** $f = \Theta(f)$, $f = O(f)$, $f = \Omega(f)$.

3. **Simmetria:** $f = \Theta(g) \Leftrightarrow g = \Theta(f)$.

4. **Proprietà trasposta:** $f = O(g) \Leftrightarrow g = \Omega(f)$; $f = o(g) \Leftrightarrow g = \omega(f)$.

Nota

La **proprietà trasposta** è la più utile: dimostrare un limite superiore su g equivale a dimostrare un limite inferiore su f , e viceversa.

Osservazione 3 (Funzioni non confrontabili). Non tutte le coppie di funzioni sono confrontabili asintoticamente. Ad esempio, $f(n) = n$ e $g(n) = n^{1+\sin n}$ non soddisfano né $f = O(g)$ né $f = \Omega(g)$, perché l’esponente $1 + \sin n$ oscilla tra 0 e 2 senza stabilizzarsi.

3.7 Limiti asintotici delle funzioni elementari

I tre teoremi seguenti forniscono la **gerarchia fondamentale** tra classi di funzioni: polinomi, logaritmi e esponenziali. Sono usati continuamente nell’analisi degli algoritmi.

Teorema 3.3 (Polinomi). Sia $f(n) = a_d n^d + a_{d-1} n^{d-1} + \dots + a_0$ un polinomio di grado d con $a_d > 0$. Allora $f(n) = \Theta(n^d)$.

Dimostrazione. Basta mostrare $f(n) = O(n^d)$ e $f(n) = \Omega(n^d)$. Per il limite superiore: $f(n) \leq (|a_d| + \dots + |a_0|) n^d$ per ogni $n \geq 1$. Per il limite inferiore: $f(n) \geq \frac{a_d}{2} n^d$ per n sufficientemente grande, poiché i termini minori crescono più lentamente di $a_d n^d$. $\square$

Teorema 3.4 (Logaritmi e polinomi). Per ogni costante $a > 0$, $m \geq 0$ e $k \geq 0$:

$$(\log n^m)^k = (m \log n)^k = \Theta((\log n)^k) = O(n^a).$$

In particolare, $\log n = O(n)$.

Dimostrazione. Si dimostra $\log n = O(n^a)$ per ogni $a > 0$ usando i limiti: $\lim_{n \rightarrow \infty} \frac{\log n}{n^a} = 0$ (per L'Hôpital o per la definizione di esponenziale). $\square$

Teorema 3.5 (Esponenziali e polinomi). Per ogni costante $a > 1$ e $k \geq 0$:

$$n^k = O(a^n).$$

Dimostrazione. $\lim_{n \rightarrow \infty} n^k / a^n = 0$ per ogni $a > 1$ e k fissato, poiché gli esponenziali crescono più velocemente di qualunque polinomio. $\square$

Corollario 3.6. Per ogni $a > 1$:

$$(\log n)^k = O(n^a), \quad n^k = O(a^n), \quad a^n = O(n!).$$

Gerarchia da ricordare (dal più lento al più veloce):

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^k) \subset O(a^n) \subset O(n!)$$

Queste inclusioni derivano direttamente dai tre teoremi sopra.

3.8 Equazioni e disequazioni asintotiche

Le notazioni asintotiche servono per **eliminare dettagli non importanti**. Si trovano spesso a destra di un “=” che sostituisce $\in$:

$$2n^2 + \Theta(n) = \Theta(n^2)$$

Nota

Regola fondamentale: il lato destro di un'equazione asintotica è sempre *meno preciso* del lato sinistro. Si può leggere come: “il lato sinistro appartiene all'insieme descritto dal lato

destro”.

Esempio 3.3. Verificare le seguenti:

- $2n^2 + 3n + 1 = 2n^2 + \Theta(n) = \Theta(n^2)$ ✓
- $n^2 + 4n + 1 = n^2 + \Theta(n)?$ ✓
- $n^2 + \Theta(n) = O(n^2)?$ ✓

3.9 Proprietà delle notazioni asintotiche

Siano $f(n)$ e $g(n)$ asintoticamente positive ($f(n), g(n) > 0$ per n sufficientemente grande).

Proprietà	Θ	O / Ω
Transitiva	$f = \Theta(g), g = \Theta(h) \Rightarrow f = \Theta(h)$	vale anche per O e Ω
Riflessiva	$f = \Theta(f)$	$f = O(f), f = \Omega(f)$
Simmetrica	$f = \Theta(g) \Leftrightarrow g = \Theta(f)$	—
Trasposta	—	$f = O(g) \Leftrightarrow g = \Omega(f)$

Non tutte le funzioni sono confrontabili!

Ad esempio n e $n^{1+\sin n}$: poiché $\sin n$ oscilla tra -1 e $+1$, l'esponente oscilla tra 0 e 2 . Non si può affermare né $n = O(n^{1+\sin n})$ né il contrario.

3.10 Limiti asintotici delle funzioni elementari

Questa sezione stabilisce la **gerarchia di crescita** tra le funzioni più comuni. Da memorizzare: più si è a destra nella gerarchia, più velocemente cresce la funzione.

$$1 \prec \log n \prec n^a \prec n^b \prec c^n \prec n! \quad (0 < a < b, c > 1)$$

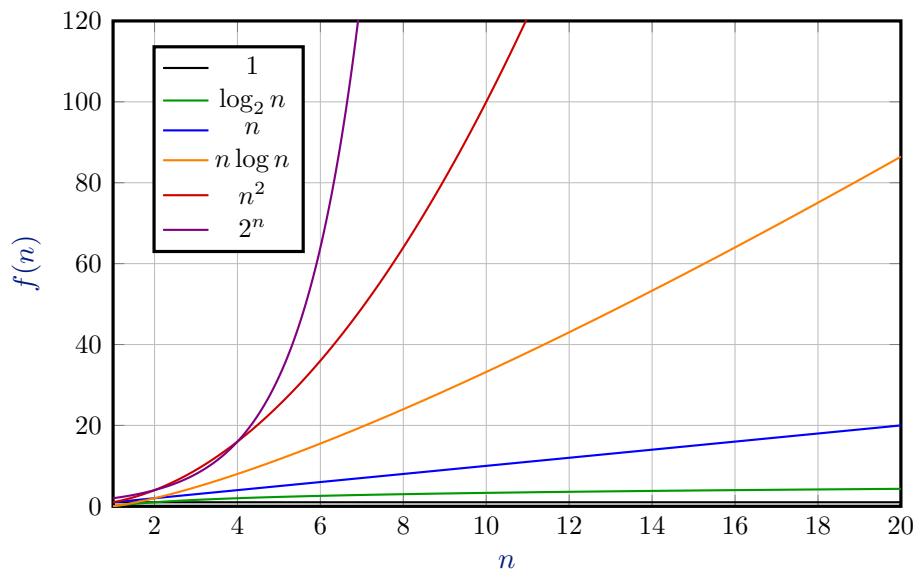


Figura 5. Confronto delle principali classi di complessità.

Teorema 3.7 (Polinomi). Sia $f(n) = a_0 + a_1n + \dots + a_dn^d$ con $a_d > 0$. Allora $f(n) \in \Theta(n^d)$.

Nota

Conseguenza pratica: per determinare la classe Θ di un polinomio basta guardare il termine di grado massimo.

Es.: $5n^3 - 200n^2 + 100n - 9 \in \Theta(n^3)$.

Teorema 3.8 (Logaritmi e polinomi). Per ogni $k, m, a > 0$: $\log^m n^k = O(n^a)$.

Qualsiasi funzione polilogaritmica è dominata da qualsiasi polinomio positivo in n .

Esempio 3.4. $\log^{107} n = O(n^{0,00001})$

Teorema 3.9 (Esponenziali e polinomi). Per ogni $a > 1, k > 0$: $n^k = O(a^n)$.

Qualsiasi funzione esponenziale con base > 1 cresce più rapidamente di qualsiasi polinomio.

Esempio 3.5. $n^{1000} = O(1,01^n)$

Gerarchia da memorizzare:

$$\log n \ll n \ll n \log n \ll n^2 \ll n^3 \ll 2^n \ll n!$$

4. Divide et Impera

4.1 Definizione

Il paradigma **divide et impera** risolve un problema tramite tre passi:

1. **Divide:** suddividere l'istanza in sottoistanze più piccole.

2. **Impera**: risolvere ricorsivamente le sottoistanze.
3. **Combina**: comporre le soluzioni parziali nella soluzione finale.

Algorithm 10: DivideEtImpera(X)

Input: X istanza di dimensione $|X| = n$

Output: Y soluzione

```

1 if  $|X| \leq n_0$  then
2   return  $\text{ad hoc}(X)$ ;
3 Dividi  $X$  in sottoistanze  $X_1, \dots, X_a$ ;
4 for  $i \leftarrow 1$  to  $a$  do
5    $Y_i \leftarrow \text{DivideEtImpera}(X_i)$ ;
6 Combina  $Y_1, \dots, Y_a$  in  $Y$ ;
7 return  $Y$ ;

```

Schema generale (coerente con le slide):

4.2 Intuizione

La tecnica è efficace quando i sottoproblemi sono il più possibile disgiunti e il costo di combinazione non domina il costo ricorsivo.

4.3 Motivazione

Molti problemi importanti (prodotto di interi grandi, MergeSort, massimo sottoarray) ammettono una decomposizione naturale che rende il costo asintotico inferiore rispetto ad approcci diretti.

4.4 Algoritmo

Il costo si modella tramite una ricorrenza:

$$T(n) = \begin{cases} g(n) & n \leq n_0, \\ aT(n/b) + f(n) & \text{altrimenti,} \end{cases}$$

dove:

- $g(n)$ è il costo del caso base (**ad hoc**),
- $f(n)$ è il costo non ricorsivo (divide + combina),
- a è il numero di sottoistanze,
- n/b è la dimensione tipica di ciascuna sottoistanza.

4.5 Esempio guidato

Prodotto di interi (Karatsuba/Gauss). Siano $x = 2^{n/2}x_L + x_R$ e $y = 2^{n/2}y_L + y_R$. Formula base:

$$xy = 2^n(x_L y_L) + 2^{n/2}(x_L y_R + x_R y_L) + x_R y_R.$$

Con il trucco di Gauss:

$$P_1 = x_L y_L, \quad P_2 = x_R y_R, \quad P_3 = (x_L + x_R)(y_L + y_R),$$

$$xy = 2^n P_1 + 2^{n/2} (P_3 - P_1 - P_2) + P_2.$$

Quindi le moltiplicazioni ricorsive diventano 3 (non 4):

$$T(n) = 3T(n/2) + \Theta(n) = \Theta(n^{\log_2 3}).$$

MergeSort.

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

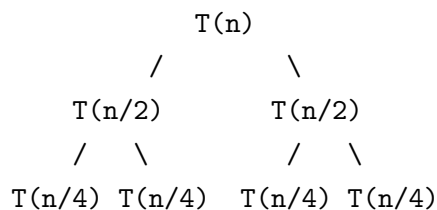
Massimo sottoarray (versione divide et impera). La soluzione su $A[low : high]$ è il massimo tra:

1. massimo interamente a sinistra,
2. massimo interamente a destra,
3. massimo che attraversa *mid*.

Ricorrenza:

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

4.6 Grafico o diagramma



Esempio di albero di ricorsione per

$$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2) :$$

Livello i :

- numero nodi: 3^i
- costo per nodo: $c (n/4^i)^2$
- costo livello: $(3/16)^i c n^2$

4.7 Dimostrazione

Metodo dell'esperto (teorema nelle slide). Per

$$T(n) = aT(n/b) + f(n), \quad a > 0, b > 1,$$

con $f(n)$ asintoticamente positiva:

1. Se $f(n) = O(n^{\log_b a - \varepsilon})$, allora

$$T(n) = \Theta(n^{\log_b a}).$$

2. Se $f(n) = \Theta(n^{\log_b a} \log^k n)$, allora

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n).$$

3. Se $f(n) = \Omega(n^{\log_b a + \varepsilon})$ e

$$af(n/b) \leq cf(n) \quad \text{con } c < 1,$$

allora

$$T(n) = \Theta(f(n)).$$

Esempi espliciti dalle slide:

- $T(n) = 9T(n/3) + n \Rightarrow \Theta(n^2)$.
- $T(n) = 2T(n/2) + n \log n \Rightarrow \Theta(n \log^2 n)$.

4.8 Analisi

Metodi di risoluzione ricorrenze richiamati nelle slide:

1. **Sostituzione:** ipotesi + induzione.
2. **Albero di ricorsione:** somma dei costi per livello.
3. **Metodo dell'esperto.**

Nel caso

$$T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2),$$

si ottiene costo totale $O(n^2)$ dalla somma geometrica dei livelli.

4.9 Errori tipici d'esame

Errore 1: confondere costo di un livello con costo totale dell'albero di ricorsione.

Errore 2: nel metodo di sostituzione, concludere erroneamente da $T(n) \leq cn + n = (c+1)n$ che $T(n) = O(n)$.

Errore 3: applicare il metodo dell'esperto senza verificare la forma $aT(n/b) + f(n)$.

4.10 Possibili domande d'esame

- Derivare la ricorrenza di MergeSort e risolverla.
- Spiegare perché il trucco di Gauss riduce da 4 a 3 moltiplicazioni ricorsive.
- Applicare il metodo dell'esperto a $T(n) = 2T(n/2) + n \log n$.
- Costruire l'albero di ricorsione di $T(n) = 3T(n/4) + \Theta(n^2)$.
- Motivare i pro/contro della trasformazione ricorsione $\rightarrow$ iterazione.

5. Analisi Probabilistica e Algoritmi Randomizzati

Finora abbiamo analizzato gli algoritmi nel caso peggiore. Questa sezione introduce un'alternativa: l'analisi del **caso medio**, e gli **algoritmi randomizzati** che rendono l'analisi media valida per qualunque input.

5.1 Preliminari: variabili casuali indicatrici

Nota

Si assume che lo studente conosca il concetto di variabile casuale. Chi non lo conosce può studiare l'Appendice C di [CLRS] come riferimento sufficiente per questa sezione.

Definizione 5.1 (Variabile casuale indicatrice). Dato uno spazio campionario S e un evento A , la **variabile casuale indicatrice** $I\{A\}$ è:

$$I\{A\} = \begin{cases} 1 & \text{se } A \text{ si verifica} \\ 0 & \text{altrimenti} \end{cases}$$

Lemma 5.1. Dato uno spazio campionario S e un evento A , posto $X_A = I\{A\}$, si ha:

$$E[X_A] = \Pr\{A\}.$$

Dimostrazione. $E[X_A] = 1 \cdot \Pr\{A\} + 0 \cdot \Pr\{\bar{A}\} = \Pr\{A\}.$

□

Nota

Perché sono utili? Le v.c. indicatrici trasformano il calcolo di medie complesse in somme di probabilità semplici, grazie alla **linearità del valore atteso**: $E[\sum_i X_i] = \sum_i E[X_i]$, che vale anche quando le X_i sono *dependenti*.

Esempio 5.1. Numero atteso di teste in n lanci di moneta. Sia $X_i = I\{\text{testa all}'i\text{-esimo lancio}\}$, e $X = \sum_{i=1}^n X_i$.

$$E[X] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n \frac{1}{2} = \frac{n}{2}.$$

Notare che si evita il calcolo diretto della distribuzione binomiale.

5.2 Analisi probabilistica: il Problema delle Assunzioni

Definizione 5.2 (Problema delle Assunzioni). **Input:** n candidati ordinabili per bravura; costo di colloquio $c_c > 0$, costo di assunzione $c_a \gg c_c$.

Output: assumere il candidato migliore con il minimo costo.

Strategia: si assume il primo candidato; si sostituisce il corrente ogni volta che si incontra uno migliore.

Se m è il numero di assunzioni effettuate, il costo totale è:

$$n \cdot c_c + m \cdot c_a.$$

Il costo $n \cdot c_c$ è fisso; quello che ci interessa è $m \cdot c_a$. Nel caso peggiore $m = n$, nel caso migliore $m = 1$. Vogliamo calcolare il **costo medio**, che dipende dalla distribuzione dei candidati.

Calcolo del costo medio. Si assuma che le $n!$ permutazioni dei candidati siano equiprobabili. Sia $X_i = I\{\text{il candidato } i\text{-esimo viene assunto}\}$, $X = \sum_{i=1}^n X_i$.

Il candidato in posizione i viene assunto se e solo se è il migliore tra i primi i . Poiché ogni permutazione dei primi i candidati è equiprobabile:

$$\Pr\{X_i = 1\} = \frac{1}{i}.$$

Quindi:

$$E[X] = \sum_{i=1}^n E[X_i] = \sum_{i=1}^n \frac{1}{i} \leq \ln n + 1.$$

Teorema 5.2 (Costo medio del Problema delle Assunzioni).

$$\text{Costo medio} \leq c_a(\ln n + 1) = O(c_a \log n),$$

contro il costo nel caso peggiore $O(c_a n)$.

Nota

$\sum_{i=1}^n 1/i$ è la **serie armonica**. Vale l'approssimazione: $\ln(n+1) \leq \sum_{i=1}^n \frac{1}{i} \leq \ln n + 1$. Ad esempio, per $n = 100$: $4.615 \leq 5.177 \leq 5.605$.

5.3 Algoritmi randomizzati

Assumere una distribuzione uniforme dell'input non è sempre realistico. Gli algoritmi randomizzati risolvono questo problema imponendo **essi stessi** la distribuzione, indipendentemente dall'input.

Analisi probabilistica	Algoritmo randomizzato
Algoritmo deterministico	Algoritmo non deterministico
Stesso input $\rightarrow$ stesso risultato	Stesso input $\rightarrow$ esecuzione diversa
Ipotesi sulla distribuzione dell'input	Nessuna ipotesi necessaria
Caso peggiore dipende dall'input	Il comportamento non dipende da un input specifico

Osservazione 4. Nel Problema delle Assunzioni, l'algoritmo randomizzato permuta casualmente i candidati *prima* di iniziare. In questo modo il costo medio $O(c_a \log n)$ è garantito **per qualunque input**, non solo per quelli ben distribuiti.

5.4 Permutazione casuale: RandomizeInPlace**Algorithm 11:** RandomizeInPlace(A)

Input: Array $A[1 : n]$

Output: A con una permutazione casuale uniforme

```

1  $n \leftarrow |A|$ ;
2 for  $i = 1$  to  $n$  do
3   └─ scambia  $A[i]$  con  $A[\text{RANDOM}(i, n)]$ ;
4 return  $A$ ;
```

Complessità in tempo: $O(n)$ — n iterazioni, ciascuna con un numero costante di operazioni e una chiamata a $\text{RANDOM}(i, n)$, assunta a costo costante.

Complessità in spazio: $O(1)$ — nessun array ausiliario.

Lemma 5.3. *RandomizeInPlace genera una permutazione casuale uniforme (ogni delle $n!$ permutazioni ha probabilità $1/n!$).*

Dimostrazione. Si dimostra tramite invariante di ciclo.

Invariante: prima della i -esima iterazione, per ogni possibile $(i-1)$ -permutazione, il sottoarray $A[1 : i-1]$ la contiene con probabilità $\frac{(n-i+1)!}{n!}$.

Base ($i = 1$): il sottoarray $A[1 : 0]$ è vuoto; l'unica 0-permutazione (vuota) vi compare con probabilità $1 = n!/n!$. ✓

Conservazione: si assume che ogni $(i-1)$ -permutazione $\langle x_1, \dots, x_{i-1} \rangle$ compaia in $A[1 : i-1]$ con probabilità $(n-i+1)!/n!$. La probabilità che l'algoritmo scelga x_i come i -esimo elemento è $1/(n-i+1)$. Quindi la probabilità di ottenere $\langle x_1, \dots, x_{i-1}, x_i \rangle$ è:

$$\frac{(n-i+1)!}{n!} \cdot \frac{1}{n-i+1} = \frac{(n-i)!}{n!}.$$

Questo è esattamente l'invariante per $i+1$. ✓

Conclusione: alla fine ($i = n+1$) ogni n -permutazione compare in $A[1 : n]$ con probabilità $\frac{(n-n)!}{n!} = \frac{1}{n!}$. □

Da ricordare all'esame: la correttezza di `RandomizeInPlace` si dimostra con un invariante di ciclo, non semplicemente contando le permutazioni. Il punto chiave è che a ogni passo la scelta uniforme in $A[i : n]$ garantisce che tutte le possibili estensioni siano equiprobabili.

5.5 QuickSort

QuickSort è uno degli algoritmi di ordinamento più usati in pratica. A differenza di MergeSort, ordina **sul posto** e, pur avendo caso peggiore $\Theta(n^2)$, nel caso medio è $\Theta(n \log n)$ con costanti moltiplicative molto piccole.

5.6 Idea Generale

QuickSort è un algoritmo che:

- ordina **sul posto** come InsertionSort;
- è basato sulla tecnica **divide et impera** come MergeSort;
- ha complessità $\Theta(n^2)$ nel caso peggiore e $\Theta(n \log n)$ nel caso medio (qui si dimostra il caso medio con l'ulteriore assunzione che i valori siano tutti distinti);
- nel caso medio le costanti moltiplicative nascoste da Θ sono abbastanza piccole $\Rightarrow$ molto efficiente in pratica, tra i più usati per l'ordinamento interno.

Nota

In Java viene usato, nella sua versione parallela, quando si devono ordinare ≥ 44 elementi.

Schema divide et impera.

- **Divide:** partiziona l'array $A[p : r]$ in due sotto-array $A[p : q - 1]$ e $A[q + 1 : r]$ tali che ogni elemento del primo sia $\leq A[q]$ (il *pivot*) e ogni elemento del secondo sia $> A[q]$.
- **Impera:** ordina i due sotto-array in modo ricorsivo fino a quando la dimensione è 0 o 1 (quando $p \geq r$).
- **Combina:** dato che i due sotto-array sono già ordinati al ritorno della chiamata ricorsiva, non serve alcuna ulteriore fase di combinazione.

Algorithm 12: QuickSort(A, p, r)**Input:** Array A , indici p, r **Output:** $A[p : r]$ ordinato in loco

```

1 if  $p < r$  then
2    $q \leftarrow \text{Partition}(A, p, r)$ ;
3   QuickSort( $A, p, q - 1$ );
4   QuickSort( $A, q + 1, r$ );

```

Nota

Per ordinare un intero array A di n elementi si invoca QuickSort($A, 1, n$).

5.7 Partition(A, p, r)

Partition è la componente fondamentale di QuickSort. Sceglie un valore *pivot* (l'ultimo elemento $A[r]$), riorganizza l'array e restituisce l'indice q della posizione finale del pivot.

Algorithm 13: Partition(A, p, r)**Input:** Array A , indici p, r **Output:** q e A modificato con $A[p : q - 1] \leq A[q] < A[q + 1 : r]$

```

1  $x \leftarrow A[r]$ ;           // Pivot: scegliere l'ultimo valore è utile per il for
2  $i \leftarrow p - 1$ ;         // Indice ultimo elemento sotto-array sinistro
3 for  $j \leftarrow p$  to  $r - 1$  do
4   if  $A[j] \leq x$ ;           // Se l'elemento deve andare nel sotto-array sx
5   then
6      $i \leftarrow i + 1$ ;      // Posizione per l'elemento da spostare
7     scambia  $A[i]$  con  $A[j]$ ;
8  $q \leftarrow i + 1$ ;         // Posizione per il pivot
9 scambia  $A[q]$  con  $A[r]$ ;
10 return  $q$ ;

```

Esempio 5.2 (Partition su $A = [2, 8, 7, 1, 3, 5, 6, 4]$, pivot = $A[8] = 4$). Traccia di PARTITION($A, 1, 8$): il pivot è $p = 4$, i parte da 0 e j scorre da 1 a 7.

Passo	Array	Azione
Inizio	[2 , 8, 7, 1, 3, 5, 6 <u>4</u>]	$i = 0$
$j = 1$	[2 , 8, 7, 1, 3, 5, 6 4]	$A[1] = 2 \leq 4$: $i++$, swap(1, 1)
$j = 2$	[2, 8 , 7, 1, 3, 5, 6 4]	$A[2] = 8 > 4$: nulla
$j = 3$	[2, 8, 7 , 1, 3, 5, 6 4]	$A[3] = 7 > 4$: nulla
$j = 4$	[2, 8, 7, 1 , 3, 5, 6 4]	$A[4] = 1 \leq 4$: $i++$, swap(2, 4)
$j = 5$	[2, 1, 7, 3 , 8, 5, 6 4]	wait... ricordare: $A[4] \leftrightarrow A[2]$
$j = 5$	[2, 1, 7, 8, 3 , 5, 6 4]	$A[5] = 3 \leq 4$: $i++$, swap(3, 5)
$j = 6$	[2, 1, 3, 8, 7 , 5, 6 4]	—
$j = 7$	[2, 1, 3, 8, 7, 5 , 6 4]	$A[6] = 5 > 4$: nulla
Fine	[2, 1, 3, 4 , 7, 5, 6, 8]	swap($i+1, 8$): pivot in pos. 4

Risultato: $\underbrace{[2, 1, 3]}_{\leq 4}, \underbrace{[4, 7, 5, 6, 8]}_{> 4}$. Il pivot è nella posizione corretta; i due sotto-array vengono ordinati ricorsivamente.

Dopo PARTITION, il pivot si trova nella sua posizione *definitiva*: non verrà mai spostato nelle chiamate ricorsive.

5.7.1 Invariante di ciclo di Partition

Per dimostrare la correttezza di **Partition**(A, p, r), si considera la seguente invariante di ciclo:

Definizione 5.3 (Invariante di Partition). All'inizio di ogni iterazione del ciclo **for**, per qualsiasi indice k dell'array vale almeno una delle seguenti proprietà:

1. Se $p \leq k \leq i$, allora $A[k] \leq x$;
2. Se $i + 1 \leq k \leq j - 1$, allora $A[k] > x$;
3. Se $k = r$, allora $A[k] = x$ (il pivot).

Nota

Non si considerano i valori con indice *t.c.* $j \leq k \leq r - 1$ perché sono valori non ancora considerati e quindi non ancora posizionati correttamente.

Dimostrazione. 1. **Inizializzazione** (prima della I iterazione, $i = p - 1$, $j = p$): non ci sono valori tra p e $i + 1 = p$ né tra $i + 1 = p$ e $j - 1 = p - 1$. Le prime due proprietà dell'invariante sono banalmente soddisfatte. L'istruzione $x \leftarrow A[r]$ soddisfa la III proprietà. ✓

2. **Conservazione:** ci sono due possibilità.

- Se $A[j] > x$ (riga 4 non eseguita), allora non si fa nulla e $j' = j + 1$: la proprietà 2 dell'invariante è soddisfatta all'inizio del prossimo giro perché $j' - 1 = j$.
- Se invece $A[j] \leq x$, allora $i' = i + 1$, vengono scambiati $A[i']$ e $A[j]$, poi $j' = j + 1$. In seguito allo scambio, $A[i'] \leq x$ e la proprietà 1 è soddisfatta. Anche $A[j' + 1] > x$ perché era l'elemento $A[i]$ prima dello scambio. Quindi anche la proprietà 2 è soddisfatta.

3. **Conclusione:** alla fine del ciclo $j = r$. Ogni posizione dell'array si trova in uno dei tre insiemi descritti nell'invariante. L'istruzione alla riga 8 posiziona l'elemento pivot nella giusta posizione: **Partition** fa esattamente quello che ci si aspetta, riordinando l'array rispetto all'elemento pivot.

□

Complessità di Partition. Il tempo di computazione ha sempre lo stesso ordine di grandezza per qualsiasi input di dimensione $n = r - p + 1$:

$$T_{\text{Partition}}(n) = \Theta(n).$$

5.8 Analisi di Complessità di QuickSort

Il tempo di esecuzione di QuickSort dipende, a parità di n , da quanto il partizionamento è bilanciato.

5.8.1 Caso peggiore

Analisi elementare. **Partition** restituisce sempre un sotto-array di dimensione $n - 1$ e uno di dimensione 0. In questo caso:

$$T(n) = T(n - 1) + T(0) + \Theta(n) = T(n - 1) + \Theta(n),$$

che è la somma aritmetica dei costi di **Partition**:

$$T(n) = \Theta(n^2). \quad (\text{caso peggiore})$$

Questo caso si verifica anche quando l'array è **completamente ordinato**! QuickSort è peggiore di InsertionSort in questo scenario.

Analisi formale. **Partition** restituisce sempre due sotto-array, uno di dimensione d e l'altro di dimensione $n - d - 1$, dove $0 \leq d \leq n - 1$:

$$T(n) = \max_{0 \leq d \leq n-1} [T(d) + T(n - d - 1)] + \Theta(n).$$

Si usa il metodo di sostituzione ipotizzando $T(n) \leq cn^2$:

$$T(n) \leq \max_{0 \leq d \leq n-1} [cd^2 + c(n - d - 1)^2] + \Theta(n).$$

La quantità $d^2 + (n - d - 1)^2$ ha il suo massimo quando $d = 0$ o $d = n - 1$:

$$\max_{0 \leq d \leq n-1} [d^2 + (n - d - 1)^2] = (n - 1)^2 \leq n^2 - 2n + 1.$$

Quindi $T(n) \leq c(n^2 - 2n + 1) + \Theta(n) \leq cn^2$, confermando $T(n) = O(n^2)$.

5.8.2 Caso migliore

Partition restituisce sempre due sotto-array con dimensione $\lfloor n/2 \rfloor$ e $\lceil n/2 \rceil - 1$ rispettivamente. In questo caso (omettendo troncamenti e arrotondamenti):

$$T(n) = 2T(n/2) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n). \quad (\text{caso migliore})$$

5.8.3 Verso il caso medio

Caso con proporzionalità fissa. Si supponga che **Partition** restituisca sempre due sotto-array con dimensione $\frac{9}{10}n$ e $\frac{1}{10}n$ rispettivamente (caso sbilanciato):

$$T(n) = T\left(\frac{9}{10}n\right) + T\left(\frac{1}{10}n\right) + \Theta(n).$$

Sembrerebbe un caso simile al peggiore perché sbilanciato... ma analizzando l'albero di ricorsione si vede che ogni livello ha costo $\Theta(n)$ finché il ramo $T(\frac{1}{10}n)$ non raggiunge la dimensione 1 (dopo $\log_{10} n$ livelli). Dopo tale livello ogni livello costa ancora $\Theta(n)$. La ricorsione termina alla profondità $\log_{10/9} n = \Theta(\log n)$, quindi il costo totale è $\Theta(n \log n)$.

Analisi verso il caso medio generale. In generale non si può supporre che tutte le chiamate ricorsive siano sempre caso migliore, peggiore o con proporzionalità fissa. Assumendo che gli input siano distribuiti in modo equiprobabile, durante una computazione alcune ricorsioni divideranno bene, altre no.

Si definisce l'elemento pivot $A[q]$ trovato da **Partition** **buono** se $A[q]$ risiede tra il 25° e il 75° percentile:

$$A[q] \text{ buono} \Leftrightarrow |A[p : q - 1]|, |A[q + 1 : r]| \leq \frac{3}{4}|A[p : r]|.$$

Il 50% degli elementi sono buoni. Sia $X_q = I\{A[q] \text{ è un elemento buono}\}$:

$$\Pr\{X_q = 1\} = E[X_q] = \frac{1}{2}.$$

Dati due pivot $A[q]$ e $A[q']$ scelti casualmente uno dopo l'altro, il valore atteso del numero di pivot buoni nella sequenza è:

$$E[X_q + X_{q'}] = E[X_q] + E[X_{q'}] = \frac{1}{2} + \frac{1}{2} = 1.$$

Quindi, mediamente ogni 2 scelte casuali del pivot si ottiene un valore buono. Due livelli consecutivi della ricorsione costano complessivamente $\Theta(n)$. Dopo una partizione buona, la dimensione del sotto-problema più grande è al più $\frac{3}{4}n$. Raggruppando i livelli due a due, la ricorrenza si comporta come:

$$T(n) = T\left(\frac{3}{4}n\right) + T\left(\frac{1}{4}n\right) + \Theta(n) \Rightarrow T(n) = \Theta(n \log n).$$

5.9 QuickSort Randomizzato

Assumere che l'input abbia gli elementi distribuiti in modo casuale uniforme è troppo limitante. È preferibile mescolare l'input usando un generatore di numeri casuali, oppure scegliere il pivot casualmente prima di invocare **Partition**.

Algorithm 14: RandQuickSort(A, p, r)

Input: Array A , indici p, r

Output: $A[p : r]$ ordinato in loco

```

1 if  $p < r$  then
2    $q \leftarrow \text{RandPartition}(A, p, r);$ 
3   RandQuickSort( $A, p, q - 1$ );
4   RandQuickSort( $A, q + 1, r$ );
```

Algorithm 15: RandPartition(A, p, r)

Input: Array A , indici p, r

Output: q e A modificato

```

1  $i \leftarrow \text{RANDOM}(p, r);$ 
2 scambia  $A[r]$  con  $A[i]$ ;
3 return Partition( $A, p, r$ );
```

5.10 Analisi del Caso Medio di QuickSort

RandQuickSort garantisce una scelta uniforme del pivot ad ogni chiamata ricorsiva. Di seguito si presenta l'analisi formale della complessità nel caso medio seguendo [CLRS cap. 7].

Il costo reale dell'algoritmo è proporzionale al numero totale di confronti (istruzione 4) eseguiti dalla procedura **Partition**. Si deve quindi determinare il numero totale di confronti fatti da **Partition** durante tutta una computazione di QuickSort.

Dato A , si rinominano tutti gli elementi come $z_1, z_2, \dots, z_n$ dove z_i è l' i -esimo elemento più piccolo di A . QuickSort confronta in **Partition** una coppia z_i, z_j **al più una volta** in tutta la sua esecuzione. Se uno dei due diventa pivot, può essere confrontato con l'altro; un elemento pivot non viene poi più confrontato con elementi appartenenti a chiamate ricorsive successive.

Si definisce la variabile casuale indicatrice:

$$X_{ij} = I\{z_i \text{ confrontato con } z_j\}.$$

Il numero totale di confronti è:

$$X = \sum_{i=1}^{n-1} \sum_{j=i+1}^n X_{ij}.$$

Il numero totale atteso di confronti:

$$E[X] = \sum_{i=1}^{n-1} \sum_{j=i+1}^n \Pr\{z_i \text{ confrontato con } z_j\}.$$

Calcolo di $\Pr\{z_i \text{ confrontato con } z_j\}$. Si consideri l'insieme $Z_{ij} = \{z_i, z_{i+1}, \dots, z_j\}$.

- Se **Partition** sceglie un elemento x tale che $z_i < x < z_j$, allora z_i e z_j andranno su due sotto-array distinti e **non** verranno mai confrontati.
- Se $x = z_i$ oppure $x = z_j$, allora z_i e z_j verranno confrontati **una volta**.

Gli elementi di Z_{ij} sono equi-probabili. La probabilità di scegliere un elemento particolare è quindi $\frac{1}{j-i+1}$ (quando gli elementi sono distinti):

$$\Pr\{z_i \text{ confrontato con } z_j\} = \Pr\{z_i \text{ è il primo pivot di } Z_{ij}\} + \Pr\{z_j \text{ è il primo pivot di } Z_{ij}\} = \frac{2}{j-i+1}.$$

Calcolo finale.

$$\begin{aligned} E[X] &= \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} \\ &= \sum_{i=1}^{n-1} \sum_{k=1}^{n-i} \frac{2}{k+1} \quad (\text{cambio variabile } k = j - i) \\ &\leq \sum_{i=1}^{n-1} \sum_{k=1}^n \frac{2}{k} = \sum_{i=1}^{n-1} O(\log n) = O(n \log n). \end{aligned}$$

Teorema 5.4 (Complessità media di QuickSort).

$$E[T(n)] = O(n \log n). \quad (\text{caso medio})$$

Caso	Input	Complessità
Peggior	Array ordinato (o inversamente ordinato)	$\Theta(n^2)$
Migliore	Pivot sempre mediano	$\Theta(n \log n)$
Medio	Input equiprobabili / pivot casuale	$\Theta(n \log n)$

Da ricordare all'esame: il caso peggiore di QuickSort è $\Theta(n^2)$ e si verifica su array *già ordinati* con il pivot fisso all'ultimo elemento. La versione randomizzata **RandQuickSort** rende tale caso molto improbabile e garantisce $O(n \log n)$ in media per qualsiasi input.

6. Statistiche d'Ordine e Problema della Mediana

6.1 Definizione

Definizione 6.1 (Statistica d'ordine). Dato un insieme di n elementi, la i -esima statistica d'ordine è l' i -esimo elemento più piccolo.

Definizione 6.2 (Mediana). Un valore v è mediana se almeno il 50% degli elementi è $\leq v$ e almeno il 50% è $\geq v$.

- Se n è dispari: mediana = statistica d'ordine $(n+1)/2$.
- Se n è pari: mediana inferiore = statistica d'ordine $\lfloor (n+1)/2 \rfloor$, mediana superiore = statistica d'ordine $\lceil (n+1)/2 \rceil$.

Nelle slide, quando si scrive "mediana" si intende la **mediana inferiore**.

6.2 Intuizione

Ordinare tutto l'array per trovare un solo rango è spesso inutile: il problema Selezione richiede un solo elemento, non la permutazione completa.

6.3 Motivazione

Selezione è un esempio didatticamente centrale di algoritmo divide et impera randomizzato con costo medio lineare.

6.4 Algoritmo

Problema Selezione. Input: insieme A di n numeri e intero $1 \leq i \leq n$.

Output: i -esimo elemento più piccolo di A .

Riduzione con pivot v .

$$A_L = \{x \in A \mid x < v\}, \quad A_v = \{x \in A \mid x = v\}, \quad A_R = \{x \in A \mid x > v\}.$$

$$\text{Selezione}(A, i) = \begin{cases} \text{Selezione}(A_L, i) & i \leq |A_L|, \\ v & |A_L| < i \leq |A_L| + |A_v|, \\ \text{Selezione}(A_R, i - |A_L| - |A_v|) & i > |A_L| + |A_v|. \end{cases}$$

Pseudocodice RandSelect (slide: CLRS 9.2).

Algorithm 16: RANDSELECT(A, p, r, i)

Input: Array A , indici p, r , rango locale i

Output: i -esimo elemento più piccolo in $A[p : r]$

1 **if** $p = r$ **then**

2 **return** $A[p]$;

3 $q \leftarrow \text{RANDPARTITION}(A, p, r)$;

4 $k \leftarrow q - p + 1$;

5 **if** $i = k$ **then**

6 **return** $A[q]$;

7 **if** $i < k$ **then**

8 **return** RANDSELECT($A, p, q - 1, i$);

9 **else**

10 **return** RANDSELECT($A, q + 1, r, i - k$);

6.5 Esempio guidato

Sia $A = \{3, 5, 9, 4, 6, 1\}$, $n = 6$, mediana inferiore: $i = 3$.

1. Scelta casuale $v = 3$:

$$A_L = \{1\}, \quad A_v = \{3\}, \quad A_R = \{5, 9, 4, 6\}.$$

Poiché $3 > |A_L| + |A_v| = 2$, si passa a

$$\text{Selezione}(A_R, 3 - 1 - 1) = \text{Selezione}(\{5, 9, 4, 6\}, 1).$$

2. Scelta casuale $v = 4$ nel nuovo insieme:

$$A_L = \emptyset, A_v = \{4\}, A_R = \{5, 9, 6\}.$$

Poiché $0 < 1 \leq 1$, la soluzione è 4.

6.6 Grafico o diagramma

```
A = {3,5,9,4,6,1}, i=3
pivot=3
AL={1} | Av={3} | AR={5,9,4,6}
--> recurse on AR with i'=1
```

```
AR={5,9,4,6}, i'=1
pivot=4
AL={} | Av={4} | AR={5,9,6}
--> answer = 4
```

6.7 Dimostrazione

Caso medio (come nelle slide). Definiamo pivot “buono” se è tra 25-esimo e 75-esimo percentile. Allora con probabilità $1/2$ il pivot è buono e il sottoproblema ha dimensione al più $3n/4$.

Indicando con $s(n) = \Theta(n)$ il costo di una RANDPARTITION e con $w(n)$ il costo totale fino al primo pivot buono, le slide assumono:

$$E[w(n)] = O(n).$$

Quindi:

$$E[T(n)] \leq E[T(3n/4)] + O(n).$$

Per sostituzione:

$$E[T(n)] = O(n).$$

Inoltre almeno una partizione completa è sempre necessaria, quindi:

$$E[T(n)] = \Omega(n).$$

Conclusione:

$$E[T(n)] = \Theta(n).$$

6.8 Analisi

Caso peggiore. Se il pivot è sempre massimo (o minimo):

$$T(n) = T(n-1) + \Theta(n) = \Theta(n^2).$$

Caso medio.

$$E[T(n)] = \Theta(n).$$

Nota implementativa dalle slide. RANDPARTITION non usa memoria ausiliaria ma modifica l'ordine dell'array di input.

6.9 Errori tipici d'esame

Confondere "tempo medio" con "tempo nel caso peggiore": per RANDSELECT restano distinti.

Dimenticare la correzione dell'indice nella chiamata su A_R : $i' = i - |A_L| - |A_v|$.

6.10 Possibili domande d'esame

- Dimostrare che minimo/massimo richiedono almeno $n - 1$ confronti.
- Completare MINMAX con confronto a coppie e motivare il limite $\left\lceil \frac{3n}{2} \right\rceil - 2$.
- Giustificare formalmente perché il caso medio di RANDSELECT è lineare.
- Riscrivere RANDSELECT con duplicati espliciti (partizione a tre insiemi).

6.11 Contenuto richiesto ma non nel materiale ufficiale

Selezione deterministica lineare (Median of Medians)

Contenuto non presente nel materiale ufficiale del corso.

7. HeapSort

HeapSort è l'ultimo algoritmo di ordinamento per confronti che studiamo. Combina il meglio di InsertionSort (ordina **sul posto**) e di MergeSort (complessità $O(n \lg n)$), ma introduce una tecnica nuova: si basa su una **struttura dati** dedicata, lo *heap*, che trova applicazione anche nelle code di priorità.

7.1 Idea Generale

HeapSort:

- ordina **sul posto** come InsertionSort;
- ha complessità $O(n \lg n)$ nel caso peggiore, come MergeSort;
- nel caso medio è **meno veloce** di QuickSort, perché le costanti nascoste in $O(n \lg n)$ sono più piccole in QuickSort che in HeapSort;
- è basato sulla struttura dati **heap**, usata anche in altri algoritmi importanti come le code di priorità.

7.2 Struttura Dati Heap

Definizione 7.1 (Heap binario). Uno **heap** è un array che rappresenta un albero binario *quasi completo*:

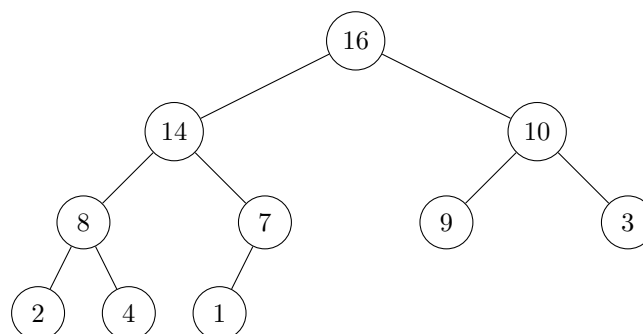
- ogni nodo dell'albero corrisponde a un elemento dell'array;
- tutti i livelli devono essere completamente riempiti tranne eventualmente l'ultimo, che può essere parziale ma sempre riempito da sinistra verso destra;
- se A è uno heap: $A.length$ è il numero di elementi dell'array, $A.heapSize$ è il numero di nodi dell'albero rappresentati ($0 \leq A.heapSize \leq A.length$);
- la radice dell'albero è in $A[1]$.

Nota

Albero binario completo vs. quasi completo:

- **Completo:** ciascun nodo interno ha esattamente due figli; le foglie sono tutte allo stesso livello.
- **Quasi completo:** tutti i livelli riempiti tranne eventualmente l'ultimo, sempre riempito da sinistra. (Se si numerano i nodi per livelli, la numerazione non ha “buchi”.)

Esempio 7.1 (Rappresentazione Heap: array e albero). Dato l'array $A = [16, 14, 10, 8, 7, 9, 3, 2, 4, 1]$, la corrispondente struttura ad albero è:



Le posizioni dell'array corrispondono ai nodi per livelli (BFS):

16	14	10	8	7	9	3	2	4	1
1	2	3	4	5	6	7	8	9	10

Regola di navigazione (indice 1-based):

$$\text{left}(i) = 2i, \quad \text{right}(i) = 2i + 1, \quad \text{parent}(i) = \lfloor i/2 \rfloor.$$

Il nodo $i = 2$ (valore 14) ha figlio sinistro in posizione 4 (valore 8) e figlio destro in posizione 5 (valore 7).

7.2.1 Navigazione dell'albero tramite indici

Dato il nodo in posizione i , genitore e figli si calcolano con:

$$\text{PARENT}(i) = \lfloor i/2 \rfloor, \quad \text{LEFT}(i) = 2i, \quad \text{RIGHT}(i) = 2i + 1.$$

Nota

Moltiplicare/dividere per 2 richiede solo di spostare di 1 bit a sinistra/destra: operazione estremamente efficiente a livello hardware.

7.2.2 Tipi di heap e proprietà

Definizione 7.2 (Max-heap e min-heap). • **Max-heap:** $A[\text{PARENT}(i)] \geq A[i]$ per ogni $i \neq 1$ (il genitore è sempre $\geq$ dei figli).

• **Min-heap:** $A[\text{PARENT}(i)] \leq A[i]$ per ogni $i \neq 1$ (il genitore è sempre $\leq$ dei figli).

HEAPSORT usa i **max-heap**; i min-heap sono usati per realizzare le code di priorità.

7.2.3 Altezza dello heap

Definizione 7.3 (Altezza di un nodo). L'**altezza** di un nodo è il numero di archi del cammino semplice più lungo che parte dal nodo e termina in una foglia (numero di livelli sottostanti al livello del nodo). L'**altezza dello heap** è l'altezza della radice.

Teorema 7.1 (Altezza di un heap di n nodi). *Un albero binario quasi completo di n nodi ha altezza $h = \lfloor \lg n \rfloor$.*

Dimostrazione. Vale $2^h \leq n < 2^{h+1}$. Applicando $\lg$: $h \leq \lg n < h + 1$, quindi $h = \lfloor \lg n \rfloor$. □

Esempio 7.2 (Altezza con $n = 10$). $h = \lfloor \lg 10 \rfloor = \lfloor 3,32 \rfloor = 3$.

Nota

Quasi tutte le operazioni fondamentali sugli heap sono eseguite in un tempo proporzionale all'altezza dell'albero, quindi $O(\lg n)$.

7.2.4 Operazioni fondamentali sullo heap

Sono 6 le operazioni fondamentali:

1. **BUILDMAXHEAP** — genera un max-heap da un array generico in tempo $O(n)$;
2. **MAXHEAPIFY** — ripristina la proprietà di max-heap quando a un elemento viene diminuito il valore; tempo $O(\lg n)$ (necessaria a **BUILDMAXHEAP**);
3. **MAXHEAPINSERT**, **HEAPEXTRACTMAX**, **HEAPINCREASEKEY** e **HEAPMAX** — consentono di usare uno heap per realizzare una coda di priorità; complessità $O(\lg n)$ tranne **HEAPMAX** che è $O(1)$.

7.3 MaxHeapify

MAXHEAPIFY è l'operazione di base: dato un nodo i il cui valore è stato diminuito, “fa scendere” $A[i]$ fino alla posizione corretta ripristinando la proprietà di max-heap nel sottoalbero radicato in i .

Precondizione. I sottoalberi radicati in $\text{LEFT}(i)$ e $\text{RIGHT}(i)$ sono già max-heap; la proprietà può essere violata solo nel nodo i .

Algorithm 17: MAXHEAPIFY(A, i)

Input: Array A , indice i ; sottoalberi di $\text{LEFT}(i)$ e $\text{RIGHT}(i)$ sono max-heap

Output: A con la proprietà di max-heap ristabilita nel sottoalbero radicato in i

```

1  $l \leftarrow \text{LEFT}(i)$ ;
2  $max \leftarrow (l \leq A.heapSize \text{ and } A[l] > A[i]) ? l : i$ ;           // Figlio sx vs. radice
3  $r \leftarrow \text{RIGHT}(i)$ ;
4 if  $r \leq A.heapSize$  and  $A[r] > A[max]$  then
5    $max \leftarrow r$ ;
6 if  $max \neq i$  then
7   scambia  $A[i]$  con  $A[max]$ ;
8   MAXHEAPIFY( $A, max$ );                                           // Ricorsione sul figlio

```

7.3.1 Analisi di complessità di MaxHeapify

Sia n il numero di nodi del sottoalbero da sistemare.

Ricorrenza. MAXHEAPIFY esegue $\Theta(1)$ operazioni più una sola chiamata ricorsiva sul sottoalbero più grande:

$$T(n) \leq T\left(\frac{2}{3}n\right) + \Theta(1).$$

Giustificazione del fattore $\frac{2}{3}$. Nel caso peggiore l'ultimo livello dell'albero è riempito esattamente a metà: il sottoalbero sinistro ha un livello in più del destro. Se h è l'altezza della radice, il sottoalbero sinistro (completo) ha $2^h - 1$ nodi, quello destro (completo) $2^{h-1} - 1$ nodi e il totale è $n = 3 \cdot 2^{h-1} - 1$. La proporzione del sottoalbero sinistro è:

$$\frac{2^h - 1}{3 \cdot 2^{h-1} - 1} \leq \frac{2^h}{3 \cdot 2^{h-1}} = \frac{2}{3}.$$

Soluzione. Applicando il Teorema dell'Esperto (caso 2), la ricorrenza $T(n) \leq T(\alpha n) + \Theta(1)$ ha soluzione $T(n) = O(\lg n)$ per qualsiasi $\alpha < 1$:

$$T_{\text{MAXHEAPIFY}}(n) = O(\lg n) = O(h).$$

Caso peggiore concreto: un elemento posto alla radice deve scendere fino a una foglia, eseguendo $\lfloor \lg n \rfloor$ chiamate ricorsive. Quindi la complessità è anche $\Omega(\lg n)$, cioè $\Theta(\lg n)$ nel caso peggiore.

7.4 BuildMaxHeap

Per trasformare un array generico in un max-heap si osserva che tutti gli elementi in $A[\lfloor n/2 \rfloor + 1 : n]$ sono **foglie**, quindi già heap di un elemento. Basta applicare MAXHEAPIFY a tutti i nodi interni, risalendo dall'ultimo verso la radice.

Algorithm 18: BUILDMAXHEAP(A)

Input: Array A di n elementi

Output: A riordinato come max-heap

```

1  $A.heapSize \leftarrow A.length;$ 
2 for  $i \leftarrow \lfloor A.length/2 \rfloor$  to 1 do
3    $\lfloor$  MAXHEAPIFY( $A, i$ );
```

7.4.1 Correttezza di BuildMaxHeap

Definizione 7.4 (Invariante di BuildMaxHeap). All'inizio di ogni iterazione del ciclo **for**, ogni nodo di indice $i + 1, \dots, n$ è la radice di un max-heap.

Dimostrazione. 1. **Inizializzazione:** prima del I ciclo, $i = \lfloor n/2 \rfloor$. Tutti i nodi successivi sono foglie, dunque radici di max-heap di un solo elemento. ✓

2. **Conservazione:** i figli del nodo i hanno indice maggiore di i , quindi per l'invariante sono radici di max-heap. $\text{MAXHEAPIFY}(A, i)$ ripristina la proprietà nel sottoalbero radicato in i , garantendo che all'iterazione $i - 1$ i nodi $i, \dots, n$ siano tutti radici di max-heap. ✓
3. **Conclusioni:** alla fine del ciclo $i = 0$, quindi il nodo 1 è la radice di un max-heap che comprende tutti i nodi da 1 a n : A è un max-heap. ✓

□

7.4.2 Complessità di BuildMaxHeap

Una stima immediata è $O(n \lg n)$ (si chiama MAXHEAPIFY $n/2$ volte, ciascuna $O(\lg n)$). Ma si può dimostrare un limite più stretto: $O(n)$.

Per ogni altezza h' con $0 \leq h' \leq \lfloor \lg n \rfloor$, il numero di nodi ad altezza h' è al più $n_{h'} = \lceil n/2^{h'+1} \rceil$.

Dimostrazione per induzione. Le foglie ($h' = 0$) sono $\lceil n/2 \rceil = \lceil n/2^{0+1} \rceil$. Se $n_{h'-1} \leq \lceil n/2^{h'} \rceil$, allora $n_{h'} = \lceil n_{h'-1}/2 \rceil \leq \lceil n/2^{h'+1} \rceil$. □

MAXHEAPIFY usa tempo $O(h')$ per ciascun nodo ad altezza h' . Il costo totale è:

$$\sum_{h'=1}^{\lfloor \lg n \rfloor} \left\lceil \frac{n}{2^{h'+1}} \right\rceil O(h') \leq O\left(n \sum_{h'=0}^{\lfloor \lg n \rfloor} \frac{h'}{2^{h'}}\right) = O\left(n \cdot \frac{1/2}{(1-1/2)^2}\right) = O(n).$$

Nota

Si usa la somma della serie $\sum_{h'=0}^{\infty} h' x^{h'} = x/(1-x)^2$ con $x = 1/2$, ottenendo il valore finito 2.

Teorema 7.2 (Complessità di BuildMaxHeap).

$$T_{\text{BUILDMAXHEAP}}(n) = O(n).$$

7.5 Algoritmo HeapSort

Una volta costruito il max-heap, l'elemento massimo si trova sempre in $A[1]$. L'idea è estrarlo iterativamente: si scambia la radice con l'ultimo elemento, si riduce l'*heapSize* di 1 e si ripristina la proprietà con MAXHEAPIFY . Al termine l'array è ordinato in modo crescente.

Algorithm 19: HEAPSORT(A)**Input:** Array A di n elementi**Output:** A ordinato in loco

```

1 BUILDMAXHEAP( $A$ );
2 for  $i \leftarrow A.length$  to 2 do
3   | scambia  $A[1]$  con  $A[i]$ ;
4   |  $A.heapSize \leftarrow A.heapSize - 1$ ;
5   | MAXHEAPIFY( $A, 1$ );

```

7.5.1 Correttezza di HeapSort

Definizione 7.5 (Invariante di HeapSort). All'inizio di ogni iterazione del ciclo **for**, il sottoarray $A[1 : i]$ è un max-heap contenente gli i elementi più piccoli di $A[1 : n]$, e il sottoarray $A[i + 1 : n]$ contiene gli $n - i$ elementi più grandi di $A[1 : n]$, ordinati in modo crescente.

7.5.2 Complessità di HeapSort

- BUILDMAXHEAP: $O(n)$.
- Ciclo **for**: $n - 1$ chiamate a MAXHEAPIFY, ciascuna $O(\lg n)$.

$$T_{\text{HEAPSORT}}(n) = O(n) + (n - 1) O(\lg n) = O(n \lg n).$$

Caso	Input	Complessità
Peggior	qualsiasi	$O(n \lg n)$
Medio	qualsiasi	$O(n \lg n)$
Miglior	qualsiasi	$O(n \lg n)$

Da ricordare all'esame: HeapSort è $O(n \lg n)$ in tutti i casi e ordina sul posto. Nel caso medio è più lento di QuickSort (costanti peggiori), ma non soffre del caso peggiore $\Theta(n^2)$ di QuickSort non randomizzato.

7.6 Code di Priorità

Lo heap non serve solo per ordinare: è la struttura dati ideale per realizzare una **coda di priorità**, che consente di estrarre sempre l'elemento con priorità massima (o minima) in modo efficiente.

Definizione 7.6 (Coda di max-priorità). Una **coda di max-priorità** è una struttura dati che gestisce un insieme S di elementi, ciascuno associato a un valore detto *chiave* o *priorità*, in modo da estrarre l'elemento di priorità massima o modificare la priorità di un elemento nel modo più efficiente possibile. Analogamente si definisce la coda di **min-priorità**.

7.6.1 Operazioni fondamentali

Se x è un elemento, $x.key$ è la sua chiave/priorità. Le operazioni fondamentali sono:

- **Insert**(S, x): inserisce x in S .
- **Max**(S): restituisce l'elemento di S con priorità massima.
- **ExtractMax**(S): rimuove e restituisce l'elemento con priorità massima.
- **IncreaseKey**(S, x, k): aumenta la chiave di x a k (con $k \geq x.key$).

Nota

Se si vuole **diminuire** il valore di un elemento, è sufficiente farlo direttamente e poi invocare MAXHEAPIFY per ripristinare la proprietà di max-heap.

7.6.2 Implementazione delle operazioni

Algorithm 20: MAX(S)

Input: S coda di max-priorità

Output: Elemento con priorità massima

1 **return** $S[1]$;

Complessità: $O(1)$.

Algorithm 21: EXTRACTMAX(S)

Input: S coda di max-priorità

Output: Elemento con priorità massima, rimosso da S

1 **if** $S.heapSize < 1$ **then**

2 **return** "Coda vuota!";

3 $max \leftarrow S[1]$;

4 $S[1] \leftarrow S[S.heapSize]$;

5 $S.heapSize \leftarrow S.heapSize - 1$;

6 MAXHEAPIFY($S, 1$);

7 **return** max ;

Complessità: $O(\lg n)$.

Algorithm 22: INCREASEKEY(S, i, key)

Input: S coda di max-priorità, posizione i , nuovo valore key

Output: Priorità $S[i].key$ aggiornata in S

```
1 if  $key < S[i].key$  then
2   return "Errore: nuova priorità minore dell'attuale";
3  $S[i].key \leftarrow key$ ;
4 while  $i > 1$  and  $S[PARENT(i)].key < S[i].key$  do
5   scambia  $S[i]$  con  $S[PARENT(i)]$ ;
6    $i \leftarrow PARENT(i)$ ;
```

Se la nuova chiave rompe la proprietà di max-heap, l'elemento viene scambiato con il genitore fino a trovare la posizione corretta (risalita verso la radice).

Complessità: $O(\lg n)$ (al più si sale dalla foglia più profonda alla radice).

Algorithm 23: INSERT(S, x)

Input: S coda di max-priorità, nuovo elemento x

Output: S aggiornato con x

```
1  $S.heapSize \leftarrow S.heapSize + 1$ ;
2  $key \leftarrow x.key$ ;
3  $x.key \leftarrow -\infty$ ; // Inserisce in fondo con priorità minima
4  $S[S.heapSize] \leftarrow x$ ;
5 INCREASEKEY( $S, S.heapSize, key$ ); // Risale alla posizione corretta
```

Complessità: $O(\lg n)$ (l'elemento potrebbe risalire dalla foglia più profonda alla radice).

Operazione	Complessità
MAX	$O(1)$
EXTRACTMAX	$O(\lg n)$
INCREASEKEY	$O(\lg n)$
INSERT	$O(\lg n)$
BUILDMAXHEAP	$O(n)$
MAXHEAPIFY	$O(\lg n)$

8. Ordinamento in Tempo Lineare

8.1 Definizione

In questa lezione si distinguono:

- algoritmi di ordinamento **basati sul confronto**,

- algoritmi **non** basati sul confronto (CountingSort, BucketSort).

Per gli algoritmi basati sul confronto vale il limite inferiore nel caso peggiore:

$$\Omega(n \log n).$$

8.2 Intuizione

Il limite inferiore discende dagli alberi di decisione: con n elementi distinti ci sono $n!$ permutazioni possibili, quindi l'altezza dell'albero deve soddisfare $2^h \geq n!$.

8.3 Motivazione

Per valori numerici con struttura nota (intervallo limitato, distribuzione uniforme) si possono superare i limiti degli ordinamenti per confronto nel modello generale.

8.4 Algoritmo

CountingSort (slide 12–16).

Algorithm 24: COUNTINGSORT(A, k)

Input: A array di interi in $[0, k]$

Output: B array ordinato

```
1  $B[1 : n]$  nuovo array;  
2  $C[0 : k]$  nuovo array inizializzato a 0;  
3 for  $j \leftarrow 1$  to  $A.length$  do  
4    $C[A[j]] \leftarrow C[A[j]] + 1;$   
5 for  $i \leftarrow 1$  to  $k$  do  
6    $C[i] \leftarrow C[i] + C[i - 1];$   
7 for  $j \leftarrow A.length$  to 1 do  
8    $B[C[A[j]]] \leftarrow A[j];$   
9    $C[A[j]] \leftarrow C[A[j]] - 1;$   
10 return  $B;$ 
```

Algorithm 25: BUCKETSORT(A)**Input:** A array di decimali positivi in $[0, 1)$ **Output:** C array ordinato

BucketSort (slide 17–28).

```

1  $n \leftarrow A.length$ ;
2  $B[0 : n - 1]$  array di liste vuote;
3  $C[1 : n]$  array vuoto;
4 for  $i \leftarrow 1$  to  $n$  do
5    $B[\lfloor n \cdot A[i] \rfloor].add(A[i])$ ;
6 for  $i \leftarrow 0$  to  $n - 1$  do
7   INSERTIONSORT( $B[i]$ );
8   Aggiungi a  $C$  tutti i valori ordinati di  $B[i]$ ;
9 return  $C$ ;
```

8.5 Esempio guidato**CountingSort.** Con $A = [2, 5, 3, 0, 2, 3, 0, 3]$, $k = 5$:

- conteggi: $C = [2, 0, 2, 3, 0, 1]$,
- cumulati: $C = [2, 2, 4, 7, 7, 8]$,
- output finale: $B = [0, 0, 2, 2, 3, 3, 3, 5]$.

BucketSort. Con $A = [0.78, 0.17, 0.39, 0.26, 0.72, 0.94, 0.21, 0.12, 0.23, 0.68]$ e $n = 10$:

- bucket 1: $\{0.17, 0.12\}$,
- bucket 2: $\{0.26, 0.21, 0.23\}$,
- bucket 7: $\{0.78, 0.72\}$,
- bucket 9: $\{0.94\}$,
- concatenazione finale dei bucket ordinati.

8.6 Grafico o diagrammaCountingSort: $A = [2, 5, 3, 0, 2, 3, 0, 3]$ C (conteggi): $[2, 0, 2, 3, 0, 1]$ C (cumulati): $[2, 2, 4, 7, 7, 8]$ B finale: $[0, 0, 2, 2, 3, 3, 3, 5]$ BucketSort ($n=10$): $[0.0, 0.1)$: -

$[0.1, 0.2): 0.17, 0.12$
 $[0.2, 0.3): 0.26, 0.21, 0.23$
 $[0.3, 0.4): 0.39$
 $\dots$
 $[0.9, 1.0): 0.94$

8.7 Dimostrazione

Lower bound per confronto. Se l è il numero di foglie dell'albero di decisione e h l'altezza:

$$n! \leq l \leq 2^h \Rightarrow h \geq \log_2(n!) = \Omega(n \log n).$$

Correttezza BucketSort (slide 21). Per due elementi $A[i] \leq A[j]$:

$$\lfloor nA[i] \rfloor \leq \lfloor nA[j] \rfloor.$$

Quindi $A[i]$ finisce nello stesso bucket di $A[j]$ o in uno precedente. Se nello stesso bucket, INSERTIONSORT li ordina; se in bucket diversi, la concatenazione per indice crescente mantiene l'ordine globale.

8.8 Analisi

CountingSort.

$$T(n) = \Theta(n) + \Theta(k) + \Theta(n) = \Theta(n + k).$$

Se $k = O(n)$, allora $T(n) = \Theta(n)$.

Stabilità di CountingSort. La stabilità deriva dalla scansione finale da destra verso sinistra.

BucketSort. Costo generale:

$$T(n) = \Theta(n) + \sum_{i=0}^{n-1} O(n_i^2),$$

dove n_i è la dimensione del bucket i .

Con input uniforme in $[0, 1)$:

$$E[T(n)] = \Theta(n).$$

Dettaglio chiave delle slide:

$$E[n_i^2] = 2 - \frac{1}{n} = O(1)$$

quindi

$$E[T(n)] = \Theta(n) + n O(1) = \Theta(n).$$

8.9 Errori tipici d'esame

Applicare CountingSort a chiavi non intere o fuori dall'intervallo $[0, k]$ senza trasformazione valida.

Dimenticare l'ipotesi di distribuzione uniforme quando si dichiara $\Theta(n)$ per BucketSort.

8.10 Possibili domande d'esame

- Dimostrare il lower bound $\Omega(n \log n)$ tramite alberi di decisione.
- Spiegare perché CountingSort è stabile.
- Derivare $E[T(n)]$ di BucketSort nel caso uniforme.
- Proporre una variante di BucketSort per decimali positivi non limitati a $[0, 1)$.

8.11 Contenuto richiesto ma non nel materiale ufficiale

RadixSort

Contenuto non presente nel materiale ufficiale del corso.

9. Hashing e Tabelle Hash

Le tabelle hash sono strutture dati efficienti per implementare dizionari che supportano le sole operazioni INSERT, SEARCH e DELETE. Analogamente a un array indicizzato da interi, una tabella hash indicizza gli elementi tramite la *chiave* dell'elemento stesso. Nel caso medio, sotto ipotesi ragionevoli, il tempo di ricerca è $O(1)$, anche se nel caso peggiore è $O(n)$.

9.1 Tavole a indirizzamento diretto

Definizione 9.1 (Tavola a indirizzamento diretto). Sia U l'insieme di tutti i possibili valori delle chiavi. Se $|U|$ non è troppo grande e nessun elemento ha chiave uguale a un altro, è possibile convertire ogni chiave in un intero e usare un array T di lunghezza $|U|$ per memorizzare i riferimenti agli elementi. La funzione di indicizzazione è:

$$I(k) = \sum_{i=0}^{n-1} (k[i] - 'A') \cdot 26^i$$

(nell'esempio con chiavi alfanumeriche di 2 caratteri in $[A, \dots, Z]$, $|U| = 26^2 = 676$).

L'indirizzamento diretto è praticabile solo quando $|U|$ è piccolo. Se $n \ll |U|$ (poche chiavi usate su un universo enorme) si spreca molta memoria: si deve allocare un array di dimensione $|U|$ anche se si memorizzano solo n elementi.

9.2 Tabelle hash

Quando $|U|$ è troppo grande, si usa una tabella di dimensione $m \approx n$ (con $m \ll |U|$) e una **funzione hash**

$$h : U \rightarrow \{0, \dots, m-1\}$$

che mappa la chiave k nell'indice $h(k)$ (*valore hash* di k). Poiché $m \ll |U|$, h non può essere iniettiva: possono quindi verificarsi **collisioni**.

Definizione 9.2 (Funzione hash perfetta). Una funzione hash h è *perfetta* per un insieme di chiavi S se non produce collisioni sulle sole chiavi di S , ovvero se h è iniettiva su S .

9.3 Funzioni hash: scelta e qualità

Una funzione hash ideale determina ogni valore $h(k)$ in modo casuale e indipendente nell'insieme $\{0, \dots, m-1\}$, restituendo sempre lo stesso valore per la stessa chiave (*funzione di hash uniforme e indipendente* o *oracolo casuale*). Tale funzione è solo teorica; in pratica si usano funzioni che la approssimano.

9.3.1 Conversione verso gli interi

Quando le chiavi non sono interi, si converte prima la chiave in un intero:

- **Caratteri Unicode (Java):** base 65536 (16 bit per char):

$$k' = \sum_i c_i \cdot 65536^i$$

- **Caratteri Latin-1:** base 256.
- **Subset ristretto:** base ulteriormente ridotta riassegnando gli ordinali a partire da 0.

Nota

Per stringhe di anche soli 2 caratteri, la somma k' supera il limite di un `int` Java (32 bit). Il risultato viene troncato a $w = 32$ bit, equivalente a calcolare $k' \bmod 2^{32}$. La conversione perde biettività (possibili collisioni), ma questo è accettabile: anche h stessa non è iniettiva. L'obiettivo è semplicemente *mescolare* i caratteri in un intero prima di applicare h .

9.4 Hashing statico

9.4.1 Metodo della divisione

Definizione 9.3 (Metodo della divisione).

$$h(k) = k \bmod m$$

Richiede una sola operazione. La scelta di m è critica: si preferisce un **numero primo non vicino a potenze di 2**, oppure un numero con fattori primi > 20 (Lum, 1971).

Se $m = 2^p$, allora $h(k)$ dipende solo dagli p bit meno significativi di k , ignorando completamente la parte alta della chiave. Ad esempio, con $m = 2^{12} = 4096$:

$$h(40960) = h(45056) = h(49152) = h(53248) = 0$$

poiché questi valori hanno tutti i 12 bit meno significativi pari a zero.

9.4.2 Metodo della moltiplicazione

Definizione 9.4 (Metodo della moltiplicazione). Sia $0 < A < 1$ una costante. Il valore hash è:

$$h(k) = \lfloor m \cdot ((k \cdot A) \bmod 1) \rfloor$$

dove $(k \cdot A) \bmod 1$ è la parte frazionaria di $k \cdot A$. Il valore m non è critico; si usa tipicamente $m = 2^p$.

Nota

Donald Knuth suggerisce $A \approx \frac{\sqrt{5} - 1}{2} \approx 0.6180$ (sezione aurea).

9.4.3 Metodo moltiplica-e-scorri

Il metodo della moltiplicazione può essere reso più efficiente fissando $m = 2^l$ con $0 < l \leq w$ (w = numero di bit della parola macchina). Si pone $a = A \cdot 2^w$ (intero dispari), e si calcola:

$$h_a(k) = ((k \cdot a) \bmod 2^w) \ggg (w - l)$$

dove $\gg$ è lo scorrimento logico a destra (inserisce zeri a sinistra, a differenza di $\gg$ che mantiene il bit di segno). Si estraggono così gli l bit più significativi di $r_0 = (k \cdot a) \bmod 2^w$.

Esempio 9.1 (Moltiplica-e-scorri con $w = 8$, $m = 16$). Dati $w = 8$, $m = 2^4 = 16$ (dunque $l = 4$), $k = 202$ (11001010_2), $a = 173$ (10101101_2):

$$k \cdot a = 202 \cdot 173 = 34946, \quad r_0 = 34946 \bmod 256 = 130 = 10000010_2$$

$$h_a(k) = 10000010_2 \ggg 4 = 00001000_2 = 8$$

La chiave viene quindi assegnata alla cella $8 \in \{0, \dots, 15\}$.

9.5 Hashing randomizzato

Con una funzione hash statica, un avversario può costruire n chiavi che collidono tutte nella stessa cella, portando a $n - 1$ collisioni (caso peggiore sistematico). L'hashing randomizzato risolve questo problema scegliendo casualmente la funzione hash *all'inizio di ogni esecuzione* da una famiglia opportuna: nessun input può essere sistematicamente il caso peggiore.

Definizione 9.5 (Hashing universale). Sia $\mathcal{H}$ una famiglia finita di funzioni hash con dominio U e codominio $\{0, 1, \dots, m - 1\}$. $\mathcal{H}$ è *universale* se per ogni coppia di chiavi distinte $k_i, k_j \in U$:

$$\Pr_{h \in \mathcal{H}} \{h(k_i) = h(k_j)\} \leq \frac{1}{m}$$

Equivalentemente, il numero di funzioni $h \in \mathcal{H}$ che fanno collidere k_i e k_j è al più $|\mathcal{H}|/m$.

Definizione 9.6 (Classificazione delle famiglie). Sia $\mathcal{H}$ una famiglia finita di funzioni hash ($U \rightarrow \{0, \dots, m - 1\}$) e $h \in \mathcal{H}$ scelta casualmente:

- $\mathcal{H}$ è **uniforme** se $\forall k \in U$ e $\forall q \in \{0, \dots, m - 1\}$: $\Pr\{h(k) = q\} = 1/m$.
- $\mathcal{H}$ è **ε -universale** se $\forall k_i \neq k_j$: $\Pr\{h(k_i) = h(k_j)\} \leq \varepsilon$.
- $\mathcal{H}$ è **universale** se è ε -universale con $\varepsilon = 1/m$.

Una famiglia uniforme e indipendente è anche universale (ma non viceversa).

Famiglia basata sulla teoria dei numeri. Sia p un numero primo tale che ogni chiave $k \in U$ sia associabile a un intero in $[0, \dots, p - 1]$, con $p > m$. Siano $\mathbb{Z}_p = \{0, \dots, p - 1\}$ e $\mathbb{Z}_p^* = \{1, \dots, p - 1\}$. Per ogni $a \in \mathbb{Z}_p^*$ e $b \in \mathbb{Z}_p$ si definisce:

$$h_{ab}(k) = ((ak + b) \bmod p) \bmod m$$

La famiglia $\mathcal{H}_{pm} = \{h_{ab} \mid a \in \mathbb{Z}_p^*, b \in \mathbb{Z}_p\}$ è universale, con $|\mathcal{H}_{pm}| = p(p - 1)$ e m arbitrario.

Famiglia $2/m$ -universale basata su moltiplica-e-scorri. La famiglia:

$$\mathcal{H} = \{h_a(\cdot) \mid 1 \leq a < 2^w, a \text{ dispari}\}$$

è $2/m$ -universale (con $m = 2^l$). La velocità di calcolo di $h_a(k)$ (una moltiplicazione e uno scorrimento) compensa la probabilità di collisione $2/m$ invece di $1/m$.

Nella versione italiana di [CLRS] sono presenti due errori nella definizione di questa famiglia: verificare sempre la definizione originale (edizione inglese).

9.6 Gestione delle collisioni

9.6.1 Risoluzione mediante concatenamento

Nel concatenamento (*chaining*), ogni cella $T[j]$ della tabella è la testa di una lista concatenata. Tutti gli elementi con lo stesso valore hash finiscono nella stessa lista.

Le operazioni sono:

- $\text{INSERT}(o)$: inserisce o in testa alla lista $T[h(o.\text{key})] — \Theta(1)$.
- $\text{SEARCH}(k)$: scorre la lista $T[h(k)]$ cercando la chiave k .
- $\text{DELETE}(o)$: rimuove o dalla lista $T[h(o.\text{key})]$; poiché o contiene i puntatori **prev**/**next**, l'operazione è $\Theta(1)$.

Definizione 9.7 (Fattore di carico). Il *fattore di carico* (load factor) è:

$$\alpha = \frac{n}{m}$$

dove n è il numero di elementi memorizzati e m la dimensione della tabella. Nel concatenamento $\alpha > 1$ è accettabile (preferibilmente $\alpha \approx 1$).

Teorema 9.1 (Costo medio — ricerca senza successo). *In una tabella hash con concatenamento, sotto l'ipotesi di hashing uniforme e indipendente, una ricerca senza successo richiede in media tempo $\Theta(1 + \alpha)$.*

Dimostrazione. Ogni lista delle m celle ha lunghezza media α . La ricerca di una chiave non presente richiede $\Theta(1)$ per calcolare il valore hash e $\Theta(\alpha)$ per scorrere la lista. $\square$

Teorema 9.2 (Costo medio — ricerca con successo). *Sotto le stesse ipotesi, una ricerca con successo richiede in media tempo $\Theta(1 + \alpha)$.*

Dimostrazione. Sia x_i l' i -esimo elemento inserito. Per ogni coppia (i, j) con $j > i$ e ogni cella q , sia X_{ijq} la variabile indicatrice dell'evento $\{ \text{si cerca } x_i, h(k_i) = q \text{ e } h(k_j) = q \}$. Sotto hashing uniforme e indipendente:

$$\Pr\{X_{ijq} = 1\} = \frac{1}{n} \cdot \frac{1}{m^2} \quad \Rightarrow \quad E[X_{ijq}] = \frac{1}{nm^2}$$

Il numero atteso di elementi che precedono l'elemento cercato nella lista vale:

$$E[1 + Z] = 1 + \sum_{j=1}^n \sum_{q=0}^{m-1} \sum_{i=1}^{j-1} \frac{1}{nm^2} = 1 + \frac{\alpha}{2} - \frac{\alpha}{2n}$$

Aggiungendo il costo $\Theta(1)$ per il calcolo dell'hash, il costo totale è $\Theta\left(2 + \frac{\alpha}{2} - \frac{\alpha}{2n}\right) = \Theta(1 + \alpha)$. $\square$

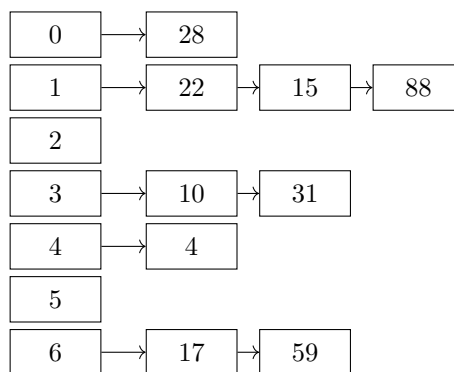
Tabella 1. Prestazioni della tabella hash con concatenamento

Operazione	Caso peggiore	Caso medio
INSERT	$\Theta(1)$	$\Theta(1)$
SEARCH	$\Theta(n)$	$\Theta(1 + \alpha)$
DELETE	$\Theta(1)$	$\Theta(1)$

Nota

Se $n = O(m)$ (il numero di elementi è proporzionale al numero di celle), allora $\alpha = n/m = O(1)$ e nel caso medio tutte le operazioni costano $O(1)$.

Esempio 9.2 (Tabella hash con concatenamento, $m = 7$). Funzione hash: $h(k) = k \bmod 7$. Chiavi da inserire: $\{10, 22, 31, 4, 15, 28, 17, 88, 59\}$.



Il **fattore di carico** è $\alpha = n/m = 9/7 \approx 1.3$. La ricerca di una chiave richiede in media $O(1 + \alpha) = O(1)$ se $m = \Theta(n)$.

9.6.2 Risoluzione mediante indirizzamento aperto

Nell'indirizzamento aperto (*open addressing*), tutti gli elementi sono memorizzati nella tabella stessa (niente liste esterne). In caso di collisione si cerca un'altra cella libera tramite una sequenza di *ispezioni* (*probing*). La funzione hash viene estesa come:

$$h : U \times \{0, 1, \dots, m-1\} \rightarrow \{0, 1, \dots, m-1\}$$

La funzione hash non estesa è denotata $h_0(\cdot)$. Poiché tutti gli slot sono nella tabella, il fattore di carico deve soddisfare $\alpha < 1$.

Algorithm 26: INSERT(T, o)**Input:** Tabella hash T ; oggetto o da inserire (non già presente).**Output:** Oggetto inserito, oppure errore “Overflow”.

```

1  $i \leftarrow 0$ ;
2 while  $i \neq m$  do
3    $j \leftarrow h(o.key, i)$ ;
4   if  $T[j] = \text{NULL}$  or  $T[j] = \text{DELETE}$  then
5      $T[j] \leftarrow o$ ;
6     return “Inserito”;
7    $i \leftarrow i + 1$ ;
8 return “Overflow”;

```

Algorithm 27: SEARCH(T, k)**Input:** Tabella hash T ; chiave k da cercare.**Output:** Indice di k in T se presente, NULL altrimenti.

```

1  $i \leftarrow 0$ ;
2 while  $i \neq m$  do
3    $j \leftarrow h(k, i)$ ;
4   if  $T[j] = \text{NULL}$  then return NULL;
5   if  $T[j] \neq \text{DELETE}$  and  $T[j].key = k$  then return  $j$ ;
6    $i \leftarrow i + 1$ ;
7 return NULL;

```

Algorithm 28: DELETE(T, i)**Input:** Tabella hash T ; indice i della cella da cancellare.

```

1  $T[i] \leftarrow \text{DELETE}$ ; // marcatore tombstone

```

Il marcatore DELETE (*tombstone*) è necessario per non interrompere le sequenze di ispezione. Se si sostituisse con NULL, la SEARCH si fermerebbe su quella cella producendo falsi negativi. SEARCH non si ferma su DELETE e continua l’ispezione; INSERT può riutilizzare una cella DELETE. Con molte cancellazioni le ispezioni peggiorano: si risolve con un **rehash** periodico (ricostruzione della tabella con i soli elementi presenti).

Analisi (ipotesi). L’analisi classica [CLRS] assume *hashing con permutazioni indipendenti e uniformi*: per ogni chiave k , la sequenza $\langle h(k, 0), \dots, h(k, m-1) \rangle$ è una permutazione casuale uniforme di $\{0, \dots, m-1\}$, e le sequenze per chiavi diverse sono indipendenti. Si assume inoltre assenza di cancellazioni (i marcatori DELETE degradano le prestazioni).

Esempio 9.3 (Ispezioni in funzione di α). Con $\alpha = 1/2$ (tabella riempita al 50%): ricerca con insuccesso $\rightarrow 1/(1 - 1/2) = 2$ ispezioni; ricerca con successo $\rightarrow 2 \ln 2 \approx 1.387$ ispezioni.

Tabella 2. Numero medio di ispezioni con indirizzamento aperto

Operazione	Caso medio
Ricerca con successo	$\frac{1}{\alpha} \ln \frac{1}{1-\alpha}$
Ricerca con insuccesso	$\frac{1}{1-\alpha}$

Con $\alpha = 0.75$ (tabella riempita al 75%): ricerca con insuccesso $\rightarrow 4$ ispezioni; ricerca con successo $\rightarrow \frac{4}{3} \ln 4 \approx 1.848$ ispezioni.

9.6.3 Ispezione lineare

Definizione 9.8 (Ispezione lineare).

$$h(k, i) = (h_0(k) + i) \bmod m, \quad i = 0, 1, \dots, m-1$$

In caso di collisione si scansionano le celle una ad una a partire da quella adiacente alla posizione originale.

L'ispezione lineare soffre del **problema del clustering primario**: le sequenze di celle adiacenti occupate (*cluster*) tendono a crescere rapidamente, vanificando la distribuzione uniforme. Il metodo genera al più m permutazioni distinte invece di $m!$, quindi non soddisfa l'ipotesi di hashing con permutazioni indipendenti. È comunque utile su architetture con memoria gerarchica (cache/RAM/disco) grazie alla località spaziale degli accessi.

9.6.4 Doppio hashing

Definizione 9.9 (Doppio hashing).

$$h(k, i) = (h_0(k) + i \cdot p(k)) \bmod m, \quad i = 0, 1, \dots, m-1$$

dove $p(k)$ è una seconda funzione hash (*funzione probe*). Affinché la sequenza copra tutte le m celle è sufficiente che $\gcd(p(k), m) = 1$.

Nota

Due costruzioni che garantiscono la copertura completa:

- m primo e $0 < p(k) < m$: ad esempio $m = 701$, $p(k) = 1 + (k \bmod 500)$.
- $m = 2^c$ e $p(k)$ dispari: ad esempio $m = 512$, $p(k) = 1 + 2(k \bmod (m/2))$.

Il doppio hashing non soffre del clustering primario ed è in generale migliore dell'ispezione lineare e quadratica. Pur non soddisfacendo l'ipotesi di hashing con permutazioni indipendenti, le permutazioni prodotte ne hanno molte caratteristiche.

10. Alberi Binari di Ricerca

Gli **alberi binari di ricerca** (ABR) sono strutture dati che possono essere usate come *dizionari* o *code di priorità*. Le operazioni di base hanno costo proporzionale all'altezza dell'albero, quindi è importante mantenerli il più possibile bilanciati (altezza $\approx \lg n$). Quando non è possibile costruire ABR bilanciati si usano varianti come gli **alberi rosso-neri**, che garantiscono altezza $O(\lg n)$ nel caso peggiore. Un altro tipo importante sono i **B-alberi**, adatti a gestire grandi quantità di dati nella memoria secondaria (in programma in "Basi di dati").

10.1 Definizione di ABR

Definizione 10.1 (Albero Binario di Ricerca). Un **albero binario di ricerca** (ABR) è un albero binario in cui ogni nodo x contiene:

- una chiave $x.key$ (che individua i dati satellite),
- i riferimenti $x.left$, $x.right$ e $x.p$ al figlio sinistro, destro e al genitore rispettivamente.

La radice $root$ è l'unico nodo con $x.p = \text{NULL}$.

Definizione 10.2 (Proprietà fondante). Le chiavi in un ABR soddisfano la seguente proprietà. Sia x un nodo:

- se y è un nodo del sottoalbero **sinistro** di x , allora $y.key < x.key$;
- se y è un nodo del sottoalbero **destro** di x , allora $y.key > x.key$.

La proprietà fondante vale per *tutti* i nodi del sottoalbero, non solo per i figli diretti. Lo stesso insieme di chiavi può essere rappresentato da ABR di forma diversa: la struttura dipende dall'ordine degli inserimenti.

Definizione 10.3 (Albero bilanciato in altezza). Un albero binario T è detto **bilanciato in altezza** (*height-balanced*) se

$$\forall t \in T, \quad |h(t_{sx}) - h(t_{dx})| \leq k,$$

dove t_{sx} e t_{dx} sono i sottoalberi sinistro e destro di t , e $h(t)$ è l'altezza del sottoalbero radicato in t . Solitamente $k = 1$. In generale, T è detto bilanciato quando la sua altezza è $O(\lg n)$, dove n è il numero di nodi.

10.2 Attraversamento

10.2.1 Attraversamento Simmetrico (in-order)

L'attraversamento **simmetrico** (in-order) sfrutta la proprietà fondante per visitare le chiavi in ordine crescente: si visita ricorsivamente il sottoalbero sinistro, poi la radice, poi il sottoalbero destro.

Algorithm 29: INORDERTREEWALK(x)

Input: Un nodo x di un ABR

```

1 if  $x = \text{NULL}$  then
2   return
3 INORDERTREEWALK( $x.\text{left}$ );
4 visit( $x.\text{key}$ ) ;                      // visit è una procedura esterna
5 INORDERTREEWALK( $x.\text{right}$ );
```

Nota

Su qualunque ABR che rappresenti l'insieme $\{2, 5, 5, 6, 7, 8\}$, INORDERTREEWALK produce sempre l'output ordinato $2, 5, 5, 6, 7, 8$, indipendentemente dalla forma (bilanciata o non) dell'albero.

Teorema 10.1 (Costo di INORDERTREEWALK). *Se x è la radice di un ABR di n nodi e il tempo di esecuzione di una singola chiamata a visit è $O(1)$, l'esecuzione di INORDERTREEWALK richiede tempo $\Theta(n)$.*

Dimostrazione. Sia $T(n)$ il tempo di esecuzione di INORDERTREEWALK su un albero di n nodi.

- $T(n) = \Omega(n)$: banale, si visitano n nodi.
- Si deve dimostrare $T(n) = O(n)$.
- $T(0) = c$ per il controllo che il nodo sia NULL.
- Per $n > 0$, con il sottoalbero sinistro di k nodi e quello destro di $n - k - 1$ nodi:

$$T(n) \leq T(k) + T(n - k - 1) + d,$$

dove d è un limite superiore al tempo costante del corpo della chiamata.

- Metodo di sostituzione: si assume $T(n) \leq (c + d)n + c$. Per $n = 0$ è banale. Per $n > 0$:

$$T(n) \leq [(c + d)k + c] + [(c + d)(n - k - 1) + c] + d = (c + d)n + c. \quad \square$$

10.2.2 Pre-ordine e Post-ordine

Gli attraversamenti in **pre-ordine** (radice $\rightarrow$ sx $\rightarrow$ dx) e in **post-ordine** (sx $\rightarrow$ dx $\rightarrow$ radice) non producono una sequenza ordinata come l'in-order.

Nota

Per esercizio: scrivere sia la versione ricorsiva sia quella iterativa di entrambi gli attraversamenti.

10.3 Interrogazione di un ABR

Gli ABR supportano le operazioni SEARCH, MINIMUM, MAXIMUM, SUCCESSOR e PREDECESSOR. Ciascuna richiede un tempo $O(h)$, dove h è l'altezza dell'ABR in input. Per i dettagli si veda [?].

10.3.1 Search**Algorithm 30:** SEARCH(x, k)

Input: Un nodo x di un ABR, una chiave k

Output: Un nodo che contiene k se esiste, NULL altrimenti

```

1 if  $x = \text{NULL}$  or  $k = x.\text{key}$  then
2   return  $x$ 
3 if  $k < x.\text{key}$  then
4   return SEARCH( $x.\text{left}, k$ )
5 return SEARCH( $x.\text{right}, k$ );
```

I nodi incontrati durante la ricerca formano un cammino verso il basso dell'albero, quindi il tempo di esecuzione è $O(h)$.

Esempio 10.1 (Ricerca di 13 in un ABR). Nell'ABR con radice 15 e nodi $\{2, 3, 4, 6, 7, 9, 13, 15, 17, 18, 20\}$, la ricerca di 13 esegue il percorso $15 \rightarrow 6 \rightarrow 7 \rightarrow 13$: tre chiamate ricorsive.

La versione **iterativa** è più efficiente in pratica (stessa complessità $O(h)$, ma costante nascosta minore poiché evita l'overhead dello stack delle chiamate):

Algorithm 31: ITERSEARCH(x, k)

Input: Un nodo x di un ABR, una chiave k

Output: Un nodo che contiene k se esiste, NULL altrimenti

```

1 while  $x \neq \text{NULL}$  and  $k \neq x.\text{key}$  do
2    $x \leftarrow (k < x.\text{key}) ? x.\text{left} : x.\text{right};$ 
3 return  $x$ ;
```

10.3.2 Minimum e Maximum

Per la proprietà fondante:

- la chiave **minima** si trova nel nodo più a sinistra (si scende sempre nel figlio sinistro fino a $x.\text{left} = \text{NULL}$);

- la chiave **massima** si trova nel nodo più a destra.

Entrambe le operazioni hanno costo $O(h)$.

Algorithm 32: MINIMUM(x)

Input: Un nodo x di un ABR

Output: Il nodo con la chiave minima nel sottoalbero radicato in x

1 **while** $x.left \neq NULL$ **do**

2 $x \leftarrow x.left$;

3 **return** x ;

Nota

La procedura MAXIMUM è simmetrica: si scende seguendo i figli destri. È lasciata per esercizio.

10.3.3 Successor e Predecessor

Definizione 10.4 (Successore). Il **successore** di un nodo x è il nodo con la più piccola chiave k' tale che $k' > x.key$ nella sequenza in-order. In un ABR si può trovare senza confrontare le chiavi:

- se x ha il sottoalbero destro, il successore è MINIMUM($x.right$);
- altrimenti, è il primo antenato y per il quale x appartiene al sottoalbero *sinistro* di y (si risale verso la radice finché si proviene da un figlio sinistro).

Algorithm 33: SUCCESSOR(x)

Input: Un nodo x di un ABR

Output: Il nodo successore di x nella sequenza ordinata dell'ABR

1 **if** $x.right \neq NULL$ **then**

2 **return** MINIMUM($x.right$)

3 $y \leftarrow x.p$;

4 **while** $y \neq NULL$ **and** $x = y.right$ **do**

5 $x \leftarrow y$;

6 $y \leftarrow y.p$;

7 **return** y ;

Nota

Nell'ABR di esempio: il successore di 4 è 6, il successore di 13 è 15. La procedura PREDECESSOR è simmetrica ed è lasciata per esercizio.

10.4 Inserimento e Cancellazione

Inserire o cancellare un elemento modifica la struttura dell'albero: occorre sempre garantire che la nuova struttura soddisfi ancora la proprietà fondante (ripristino). L'inserimento è semplice; la cancellazione richiede un ripristino più articolato.

10.4.1 Inserimento

INSERT percorre l'albero dalla radice verso il basso e inserisce z sempre come **foglia** nella posizione corretta. Non controlla la presenza di duplicati: può quindi inserire più copie dello stesso valore.

Algorithm 34: INSERT(t, z)

Input: Un ABR t e un nodo $z \notin t$ (con $z.left = z.right = \text{NULL}$)

```

1  $y \leftarrow \text{NULL};$ 
2  $x \leftarrow t.root;$ 
3 while  $x \neq \text{NULL}$  do
4    $y \leftarrow x;$ 
5    $x \leftarrow (z.key < x.key) ? x.left : x.right;$ 
6  $z.p \leftarrow y;$ 
7 if  $y = \text{NULL}$  then
8    $t.root \leftarrow z;$                                 //  $t$  era vuoto
9 else
10  if  $z.key < y.key$  then
11     $y.left \leftarrow z;$ 
12  else
13     $y.right \leftarrow z;$ 

```

10.4.2 Cancellazione

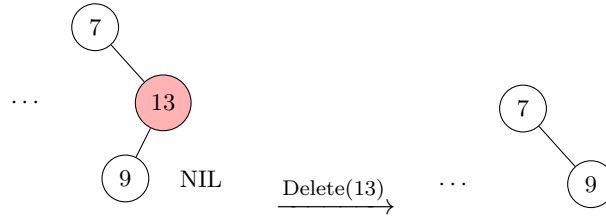
La cancellazione del nodo z gestisce i seguenti casi:

1. z **non ha figli**: si cancella direttamente z .
2. z **ha un solo figlio**: il figlio prende il posto di z .
3. z **ha entrambi i figli** e il suo successore y è figlio diretto di z (caso c): y prende semplicemente il posto di z .
4. z **ha entrambi i figli** e il successore y non è figlio diretto di z (caso d): si ricollocano y al posto di z e il figlio destro di y al posto di z . È il caso più complesso.

La soluzione CLRS utilizza la procedura ausiliaria

Esempio 10.2 (I quattro casi di DELETE). Consideriamo l'ABR con radice 15 e nodi $\{2, 3, 4, 6, 7, 9, 13, 15, 17, 18, 20\}$.

Caso (a) — z senza figlio sinistro (es. cancella 13).



TRANSPLANT($t, 13, 9$): il figlio destro (9) prende il posto di 13.

Caso (c) — entrambi i figli, successore figlio diretto (es. cancella 15). Il successore di 15 è 17 (figlio diretto del sottoalbero destro, senza figlio sinistro). TRANSPLANT($t, 15, 17$), poi si aggancia il sottoalbero sinistro di 15 come figlio sinistro di 17.

Caso (d) — entrambi i figli, successore non figlio diretto (es. cancella 6). Il successore di 6 è 7. Poiché $7.p \neq 6$, prima si esegue TRANSPLANT($t, 7, 13$) (il figlio destro di 7 prende il posto di 7), poi TRANSPLANT($t, 6, 7$) e si riagganciano i sottoalberi.

TRANSPLANT, che sostituisce il sottoalbero radicato in u con quello radicato in v :

Algorithm 35: TRANSPLANT(t, u, v)

Input: Un ABR t , due nodi $u, v \in t$ (v può essere NULL)

Output: In t , sostituisce il sottoalbero con radice u con quello con radice v

```

1 if  $u.p = \text{NULL}$  then
2    $t.\text{root} \leftarrow v$ ;
3 else
4   if  $u = u.p.\text{left}$  then
5      $u.p.\text{left} \leftarrow v$  ;                                //  $u$  è figlio sinistro
6   else
7      $u.p.\text{right} \leftarrow v$ ;
8 if  $v \neq \text{NULL}$  then
9    $v.p \leftarrow u.p$  ;                                    // Inserisci  $v$  in  $t$ 

```

TRANSPLANT ha complessità $O(1)$.

Algorithm 36: DELETE(t, z)**Input:** Un ABR t e un nodo $z \in t$ **Output:** In t , cancella z mantenendo t un ABR

```

1 if  $z.left = NULL$  then
2   TRANSPLANT( $t, z, z.right$ );
3   return ;                                     // caso (a)
4 if  $z.right = NULL$  then
5   TRANSPLANT( $t, z, z.left$ );
6   return ;                                     // caso (b)
7  $y \leftarrow \text{MINIMUM}(z.right)$ ;
8 if  $y.p \neq z$  then
9   TRANSPLANT( $t, y, y.right$ ) ; // caso (d): al posto di  $y$  metti suo figlio  $x$ 
10   $y.right \leftarrow z.right$  ;           // caso (d):  $r$  diventa figlio dx di  $y$ 
11   $y.right.p \leftarrow y$ ;
12 TRANSPLANT( $t, z, y$ ) ;                     // ultimo passo di (c) e (d)
13  $y.left \leftarrow z.left$ ;
14  $y.left.p \leftarrow y$ ;

```

DELETE ha complessità $O(h)$ perché al suo interno può eseguire MINIMUM. Non modificare la struttura prima di aver aggiornato tutti i puntatori: l'ordine delle operazioni in DELETE è cruciale.

10.5 Costruzione di un ABR

La complessità delle operazioni fondamentali è $O(h)$. Dalle proprietà degli alberi binari:

$$\lfloor \lg n \rfloor \leq h \leq n - 1,$$

dove n è il numero di nodi. Il caso peggiore ($h = n - 1$) si raggiunge inserendo le chiavi in ordine crescente (o decrescente): l'ABR degenera in una lista.

Teorema 10.2 (Altezza media di un ABR casuale). *L'altezza media di un ABR costruito in modo casuale con n chiavi **distinte** è $O(\lg n)$.*

Nota

Un ABR è *costruito in modo casuale* se, partendo dall'albero vuoto, le chiavi vengono inserite una alla volta scegliendole casualmente dall'insieme. Non sono note formule che stimano l'altezza media dopo una sequenza arbitraria di operazioni di inserimento e cancellazione.

10.6 Cenno agli ABR Rosso-Neri

Operazioni di inserimento e cancellazione possono sbilanciare un ABR fino a renderlo degenerare. Esistono varianti che mantengono il bilanciamento; la più nota è l'**ABR rosso-nero**.

Un ABR rosso-nero associa a ogni nodo un colore (rosso o nero) e rispetta le seguenti proprietà:

- ogni nodo è rosso o nero;
- la radice è nera; ogni riferimento a NULL è considerato nero;
- se un nodo è rosso, entrambi i figli sono neri;
- per ogni nodo, tutti i cammini semplici dal nodo alle foglie contengono lo stesso numero di nodi neri.

Teorema 10.3 (Altezza degli ABR rosso-neri). *L'altezza di un ABR rosso-nero con n nodi è al più $2 \lg(n + 1)$.*

Gli ABR rosso-neri **non sono in programma** per quest'anno: non si richiedono gli algoritmi di inserimento e cancellazione.

10.7 Cenno ai Radix Tree

Definizione 10.5 (Radix Tree). Un **radix tree** (o *radix trie*, *trie compresso*) è una struttura ad albero usata per rappresentare un insieme di stringhe binarie sfruttando i **prefissi comuni**:

- ogni arco è etichettato con un simbolo dell'alfabeto (0 oppure 1);
- il cammino dalla radice a un nodo rappresenta il prefisso ottenuto concatenando le etichette degli archi attraversati;
- una stringa s appartiene all'insieme se esiste un nodo marcato come **terminale** il cui cammino dalla radice è esattamente s ;
- stringhe con prefisso comune condividono la parte iniziale del cammino;
- ricerca, inserimento e visita richiedono tempo proporzionale alla lunghezza della stringa considerata.

Esempio 10.3 (Radix tree per $S = \{0, 011, 10, 1001011\}$). Le stringhe 0 e 011 condividono il prefisso 0; le stringhe 10 e 1001011 condividono il prefisso 10. I nodi terminali corrispondono esattamente agli elementi di S . La struttura ha altezza pari alla lunghezza della stringa più lunga dell'insieme (qui: 7).

10.8 Esercizi

1. Studiare il Capitolo 12 di [?].
2. Dimostrare la correttezza di `INORDERTREEWALK` per induzione.
Suggerimento: la base è un albero con un solo nodo (banale). Si suppone che la proprietà valga per alberi di dimensione $n - 1$ e si considera un albero di n nodi. Un albero di n nodi può essere visto come un nodo radice con due sottoalberi: uno di k nodi ($0 \leq k \leq n - 1$) e uno di $n - 1 - k$ nodi.
3. Scrivere le versioni ricorsiva e iterativa degli attraversamenti in pre-ordine e post-ordine.
4. Scrivere la procedura `MAXIMUM`.
5. Scrivere la procedura `PREDECESSOR`.
6. Risolvere gli esercizi 12.1-2, 12.1-3, 12.1-4, 12.2-2, 12.2-3, 12.2-4, 12.2-5, 12.3-1, 12.3-3 di [?].
Suggerimento per 12.1-3: la soluzione iterativa senza stack richiede di trasformare l'albero in un *albero threaded* (con fili). Si veda <https://www.techiedelight.com/inorder-tree-traversal-iterative-recursive>.

11. Programmazione Dinamica

La **programmazione dinamica** è un paradigma per la progettazione di algoritmi che risolvono problemi di ottimizzazione. Introdotta da Bellman (1957), si applica quando il problema presenta *sottostruttura ottima* e *sottoproblemi sovrapposti*: la soluzione ottima contiene le soluzioni ottime dei sottoproblemi, e un approccio ricorsivo naïve risolverebbe gli stessi sottoproblemi più volte, con costo esponenziale.

11.1 Idea generale

La tecnica si articola in **4 fasi**:

1. **Caratterizzazione** della struttura di una soluzione ottima.
2. **Definizione ricorsiva** del valore di una soluzione ottima.
3. **Calcolo** del valore secondo uno schema *bottom-up*.
4. **Costruzione** della soluzione ottima dalle informazioni calcolate.

In termini moderni, la programmazione dinamica descrive il problema come un insieme di **stati** (o sottoproblemi). Per ogni stato si definiscono:

- i *casi base*;
- le *transizioni*: come combinare stati più piccoli per ottenere il valore dello stato corrente;
- l'*ordine di calcolo*, che deve rispettare le dipendenze tra stati.

Il costo totale dell'algoritmo è spesso:

$$\# \text{stati} \times \text{costo per calcolare una transizione.}$$

La stessa ricorrenza può essere implementata con una tabella *bottom-up* (tabulazione) oppure con ricorsione e *memoization*.

Nota

Quando si usa la programmazione dinamica. Si usa quando:

- una soluzione ottima contiene al suo interno le soluzioni ottime dei sottoproblemi (**sottostruttura ottima**);
- un algoritmo ricorsivo richiederebbe di risolvere più volte gli stessi sottoproblemi (**sottoproblemi sovrapposti**);
- i sottoproblemi possono essere descritti da un numero limitato di parametri, cioè da uno **stato**.

11.1.1 Confronto con Divide et Impera

Proprietà	Tabulazione (bottom-up)	Divide et Impera
Tipo di risoluzione	bottom-up	top-down
Tipo di programma	iterativa	ricorsiva
Uso della memoria	esplicito per i risultati	implicito
Applicazione ideale	sottoistanze non disgiunte	sottoistanze disgiunte

Nota

La programmazione dinamica può essere implementata anche top-down tramite *memoization*, mantenendo la struttura ricorsiva del divide et impera ma riutilizzando i risultati già calcolati.

11.2 Esempi di applicazione: confronto di sequenze DNA

Una sequenza DNA si rappresenta come una stringa sull'alfabeto $\{A, C, G, T\}$. Il confronto tra due sequenze può rivelare quanto siano correlati i due organismi di origine. Esistono diversi criteri di confronto:

- verificare se una sequenza è sottostringa dell'altra;

- determinare il numero minimo di modifiche per trasformare una nell'altra (distanza di edit);
- trovare una *sottosequenza comune* di lunghezza massima.

11.3 Massima Sottosequenza Comune (LCS)

Definizione 11.1 (Sottosequenza). Data una sequenza $\langle a_1, \dots, a_n \rangle$, una **sottosequenza** è un sottoinsieme ordinato $\langle a_{i_1}, a_{i_2}, \dots, a_{i_k} \rangle$ dove la sequenza di indici $\langle i_1, i_2, \dots, i_k \rangle$ è strettamente crescente.

Esempio 11.1 (Sottosequenza). Data $X = \langle A, B, C, B, D, A, B \rangle$, la sequenza $Z = \langle B, C, D, B \rangle$ è sottosequenza di X perché i suoi indici in X formano la sequenza crescente $\langle 2, 3, 5, 7 \rangle$.

Definizione 11.2 (Sottosequenza comune — LCS). Date due sequenze X e Y , una sequenza Z è **sottosequenza comune** di X e Y se è sottosequenza di entrambe. La **Massima Sottosequenza Comune** (*Longest Common Subsequence*, LCS) è la sottosequenza comune di lunghezza massima.

Esempio 11.2 (LCS (CLRS)). Sia $X = \langle A, B, C, B, D, A, B \rangle$ e $Y = \langle B, D, C, A, B, A \rangle$. La sequenza $\langle B, C, A \rangle$ è una sottosequenza comune; la sottosequenza comune più lunga è $\langle B, C, B, A \rangle$ (lunghezza 4).

Problema formale. **Input:** due sequenze X e Y sullo stesso alfabeto. **Output:** la sottosequenza comune di lunghezza massima.

11.3.1 Fase 1 — Sottostruttura ottima

Sia $X_i = \langle x_1, \dots, x_i \rangle$ l' i -esimo prefisso di X .

Teorema 11.1 (Sottostruttura ottima della LCS). *Dati $X = \langle x_1, \dots, x_m \rangle$, $Y = \langle y_1, \dots, y_n \rangle$ e una LCS $Z = \langle z_1, \dots, z_k \rangle$:*

1. Se $x_m = y_n$, allora $z_k = x_m$ e Z_{k-1} è una LCS di X_{m-1} e Y_{n-1} .
2. Se $x_m \neq y_n$ e $z_k \neq x_m$, allora Z è una LCS di X_{m-1} e Y .
3. Se $x_m \neq y_n$ e $z_k \neq y_n$, allora Z è una LCS di X e Y_{n-1} .

Dimostrazione. Punto 1. Se $x_m = y_n$ ma Z non termina con tale carattere, si può sostituire l'ultimo elemento di Z con x_m ottenendo una LCS che termina con $x_m = y_n$. Tolto l'ultimo carattere, la parte rimanente è una LCS di X_{m-1} e Y_{n-1} .

Punto 2. Se $z_k \neq x_m$, allora Z è una LCS di X_{m-1} e Y : se esistesse una sottosequenza comune W di X_{m-1} e Y con $|W| > k$, allora W sarebbe anche sottosequenza comune di X e Y , contraddicendo l'ipotesi.

Punto 3. Dimostrazione simmetrica al punto 2. □

11.3.2 Fase 2 — Definizione ricorsiva del valore

Sia $c[i, j]$ la lunghezza di una LCS di X_i e Y_j . La soluzione del problema originale è $c[m, n]$. La ricorrenza è:

$$c[i, j] = \begin{cases} 0 & \text{se } i = 0 \text{ o } j = 0 \\ c[i - 1, j - 1] + 1 & \text{se } i, j > 0 \text{ e } x_i = y_j \\ \max\{c[i, j - 1], c[i - 1, j]\} & \text{se } i, j > 0 \text{ e } x_i \neq y_j \end{cases}$$

Perché non basta il Divide et Impera? Con $i = j = n$, il numero di chiamate ricorsive nel caso peggiore vale $T(i, j) = 2^{\binom{i+j}{i}} - 1$, valore **esponenziale** quando i e j crescono insieme. I sottoproblemi distinti sono invece soltanto $\Theta(nm)$, uno per ogni coppia di prefissi (X_i, Y_j) .

11.3.3 Fase 3 — Calcolo bottom-up: algoritmo LCS-Length

Algorithm 37: LCS-LENGTH(X, Y)

Input: $X = \langle x_1, \dots, x_m \rangle, Y = \langle y_1, \dots, y_n \rangle$

Output: Tabelle $c[0:m, 0:n]$ e $b[1:m, 1:n]$

```

1  Siano  $b[1:m, 1:n]$  e  $c[0:m, 0:n]$  due tabelle inizializzate a 0 ;           //  $b$  serve per
   ricostruire la LCS
2  for  $i \leftarrow 1$  to  $m$  do
3      for  $j \leftarrow 1$  to  $n$  do
4          if  $x_i = y_j$  then
5               $c[i, j] \leftarrow c[i - 1, j - 1] + 1$ ;
6               $b[i, j] \leftarrow \nwarrow$ ;
7          else
8              if  $c[i - 1, j] \geq c[i, j - 1]$  then
9                   $c[i, j] \leftarrow c[i - 1, j]$ ;
10                  $b[i, j] \leftarrow \uparrow$ ;
11             else
12                  $c[i, j] \leftarrow c[i, j - 1]$ ;
13                  $b[i, j] \leftarrow \leftarrow$ ;
14 return  $c, b$ 
```

Il tempo di computazione è $\Theta(nm)$.

11.3.4 Fase 4 — Ricostruzione: algoritmo Print-LCS

Algorithm 38: PRINT-LCS(b, X, i, j)

Input: Matrice b ; sequenza X ; indici correnti i, j

```

1 if  $i = 0$  and  $j = 0$  then
2   return
3 if  $b[i, j] = "\nwarrow"$  then
4   PRINT-LCS( $b, X, i - 1, j - 1$ );
5   stampa  $x_i$ ;
6 else
7   if  $b[i, j] = "\uparrow"$  then
8     PRINT-LCS( $b, X, i - 1, j$ );
9   else
10    PRINT-LCS( $b, X, i, j - 1$ );
```

Si invoca come PRINT-LCS($b, X, |X|, |Y|$). Il tempo di computazione è $O(n + m)$.

Esempio 11.3 (LCS su ABCBDAB / BDCABA). Per $X = \langle A, B, C, B, D, A, B \rangle$ e $Y = \langle B, D, C, A, B, A \rangle$, l'algoritmo restituisce la LCS $\langle B, C, B, A \rangle$ di lunghezza 4, leggendo le frecce $\nwarrow$ nella tabella b .

11.4 Massima Sottosequenza Crescente

Definizione 11.3 (Sottosequenza crescente). Una sottosequenza $a_{i_1}, a_{i_2}, \dots, a_{i_k}$ di interi è **crescente** se $a_{i_1} < a_{i_2} < \dots < a_{i_k}$.

Problema. **Input:** sequenza di interi $a_1, \dots, a_n$. **Output:** sottosequenza crescente di lunghezza massima.

Esempio 11.4 (MSC). La sequenza 5, 2, 8, 6, 3, 6, 9, 7 ha massima sottosequenza crescente 2, 3, 6, 9 (lunghezza 4).

11.4.1 Riduzione a cammino massimo in un DAG

Si costruisce un DAG $G = (V, E)$ dove esiste l'arco (i, j) se e solo se $i < j$ e $a_i < a_j$. Trovare la massima sottosequenza crescente è **equivalente** a trovare il cammino più lungo nel DAG.

11.4.2 Definizione ricorsiva

Sia $L(j)$ la lunghezza del cammino più lungo che termina nel nodo j :

$$L(j) = 1 + \max(\{0\} \cup \{L(i) \mid (i, j) \in E\}).$$

Il valore della soluzione ottima è $\max_j L(j)$.

11.4.3 Algoritmo bottom-up (versione valutativa)

Algorithm 39: MSCRESC-V($a_1, \dots, a_n$)

Input: Sequenza di interi $a_1, \dots, a_n$
Output: Lunghezza della massima sottosequenza crescente

```

1 Costruisci il DAG trasposto  $G^T = (V, E^T)$  da  $(a_1, \dots, a_n)$ ;
2  $L \leftarrow$  array di  $n$  interi ;                                // soluzioni parziali
3  $ottimo \leftarrow -\infty$ ;
4 for  $j \leftarrow 1$  to  $n$  do
5    $max \leftarrow 0$ ;
6   for ogni  $i$  tale che  $(j, i) \in E^T$  do
7     if  $L[i] > max$  then
8        $max \leftarrow L[i]$ 
9    $L[j] \leftarrow 1 + max$ ;
10  if  $L[j] > ottimo$  then
11     $ottimo \leftarrow L[j]$ 
12 return  $ottimo$ 

```

Complessità. La costruzione del grafo trasposto richiede $O(n^2)$. Il calcolo di tutti gli $L(j)$ richiede $O(|E|)$, con $|E| \leq \frac{n(n-1)}{2}$. La complessità totale è $O(n^2)$.

L'approccio Divide et Impera puro genera 2^{j-1} chiamate ricorsive per calcolare $L(j)$ nel caso peggiore (sequenza completamente crescente), rendendo l'algoritmo esponenziale. La programmazione dinamica risolve ciascuno degli n stati esattamente una volta.

11.5 String Matching Approssimato

Nella ricerca approssimata di un pattern P in un testo T si ammettono tre tipi di errore: *sostituzione* (caratteri corrispondenti diversi), *gap in P* (un carattere di T non allineato con nessun carattere di P), *gap in T* (un carattere di P non allineato con nessun carattere di T). Questo problema generalizza lo string matching esatto ed è strettamente correlato alla **distanza di edit** (Levenshtein).

Definizione 11.4 (Occorrenza k -approssimata). Data una stringa T (testo, lunghezza n) e una stringa P (pattern, lunghezza $m < n$), un'**occorrenza k -approssimata** di P in T è un allineamento di P con una sottostringa di T in cui compaiono esattamente k errori dei tipi: sostituzione, gap in P , gap in T .

Problema formale. **Input:** testo T di lunghezza n , pattern P di lunghezza $m < n$. **Output:** occorrenza k -approssimata minima (rispetto a k) di P in T .

11.5.1 Definizione ricorsiva

Sia $K[i, j]$ il minimo numero di errori nell'approssimare $P[1, \dots, i]$ con una sottostringa di $T[1, \dots, j]$ che termina in posizione j . La soluzione ottima è $k = \min_j K[m, j]$.

Equazione di ricorrenza unificata:

$$K[i, j] = \min\{ \text{diff}(i, j) + K[i-1, j-1], \ 1 + K[i-1, j], \ 1 + K[i, j-1] \}$$

dove $\text{diff}(i, j) = 0$ se $P[i] = T[j]$, altrimenti 1.

Casi base.

- $K[i, 0] = i$ (il pattern ha i caratteri in più della stringa vuota);
- $K[0, j] = 0$ (il pattern vuoto si allinea a costo 0 a qualunque prefisso di T).

11.5.2 Algoritmo SMA

Algorithm 40: SMA(T, P)

Input: Testo T , pattern P

Output: (numero minimo di errori; posizione ultimo carattere dell'occorrenza in T)

```

1  $n \leftarrow |T|$ ;
2  $m \leftarrow |P|$ ;
3 for  $i \leftarrow 0$  to  $n$  do
4    $K[0, i] \leftarrow 0$ 
5 for  $i \leftarrow 0$  to  $m$  do
6    $K[i, 0] \leftarrow i$ 
7 for  $i \leftarrow 1$  to  $m$  do
8   for  $j \leftarrow 1$  to  $n$  do
9      $\text{diff} \leftarrow (T[j] = P[i]) ? 0 : 1$ ;
10     $K[i, j] \leftarrow \min\{ \text{diff} + K[i-1, j-1], \ 1 + K[i-1, j], \ 1 + K[i, j-1] \}$ ;
11  $\text{Min} \leftarrow +\infty$ ;
12  $\text{pos} \leftarrow -1$ ;
13 for  $j \leftarrow 1$  to  $n$  do
14   if  $K[m, j] < \text{Min}$  then
15      $\text{Min} \leftarrow K[m, j]$ ;
16      $\text{pos} \leftarrow j$ ;
17 return ( $\text{Min}, \text{pos}$ )
```

La complessità è $\Theta(nm)$.

Esempio 11.5 (SMA su “It’s snowy” / “sunny”). Con $T = \text{It's snowy}$ e $P = \text{sunny}$, la tabella K produce $\min_j K[5, j] = 3$, confermando che il pattern si abbina al testo con 3 errori.

11.6 Problema dello Zaino (Knapsack 0-1)

Problema formale. **Input:** n oggetti con valore v_i e peso p_i (interi positivi); portata massima P . **Output:** vettore $x \in \{0, 1\}^n$ che massimizza $\sum_{i=1}^n x_i v_i$ con vincolo $\sum_{i=1}^n x_i p_i \leq P$.

Nota

Questa versione è chiamata **Zaino 0-1** perché ogni oggetto o si prende ($x_i = 1$) o si lascia ($x_i = 0$). Esistono anche le versioni *frazionaria* (risolvibile greedy) e *con ripetizione*.

11.6.1 Sottostruttura ottima

Si considera la soluzione ottima per i primi j oggetti con capacità w .

- Se l'oggetto j **non** è nella soluzione: la soluzione è ottima per i primi $j - 1$ oggetti con capacità w .
- Se l'oggetto j **è** nella soluzione: toltolo, rimane una soluzione ottima per i primi $j - 1$ oggetti con capacità $w - p_j$.

In entrambi i casi: *taglia e incolla* mostra che la soluzione ottima del problema originale contiene la soluzione ottima del sottoproblema.

11.6.2 Definizione ricorsiva

Sia $K[j, w]$ il massimo valore ottenibile usando uno zaino di capacità w e i primi j oggetti. La soluzione è $K[n, P]$.

$$K[j, w] = \begin{cases} 0 & \text{se } j = 0 \text{ o } w = 0 \\ K[j - 1, w] & \text{se } p_j > w \\ \max\{K[j - 1, w], K[j - 1, w - p_j] + v_j\} & \text{altrimenti} \end{cases}$$

11.6.3 Algoritmo Zaino

Algorithm 41: ZAINO(P, p, v)**Input:** Portata P ; array pesi p ; array valori v **Output:** Valore ottimo dello zaino

```

1  $n \leftarrow |v|$ ;
  ; // In Java le tabelle sono già inizializzate a 0
2 for  $j \leftarrow 1$  to  $n$  do
3   for  $w \leftarrow 1$  to  $P$  do
4     if  $p[j] > w$  then
5        $K[j, w] \leftarrow K[j - 1, w]$ ; // oggetto  $j$  troppo pesante
6     else
7        $K[j, w] \leftarrow \max(K[j - 1, w], K[j - 1, w - p[j]] + v[j])$ ;
8 return  $K[n, P]$ 

```

Complessità. La complessità è $\Theta(nP)$.

$\Theta(nP)$ è **pseudo-polinomiale**, non polinomiale nella dimensione dell'input. Se k è il numero di bit per rappresentare P , allora $k = \Theta(\log P)$, cioè $P = \Theta(2^k)$: il costo può essere esponenziale nella lunghezza della rappresentazione binaria di P . Il problema Zaino 0-1 è NP-difficile.

Esempio 11.6 (Zaino 0-1). Con $v = [6, 13, 4, 4]$, $p = [3, 13, 5, 3]$, $P = 13$, lo zaino ottimo contiene gli oggetti $\{x_1, x_3, x_4\}$ con valore totale $6 + 4 + 4 = 14$ e peso $3 + 5 + 3 = 11$.

11.7 Casi comuni di sottoproblemi

Nota

Caratterizzare correttamente la decomposizione in sottoproblemi richiede creatività ed esperienza. Quattro schemi ricorrono più frequentemente:

1. **Prefisso singolo.** Input $x_1, \dots, x_n$; il sottoproblema considera $x_1, \dots, x_i$. Numero di stati: $O(n)$.
2. **Doppio prefisso.** Input $x_1, \dots, x_m$ e $y_1, \dots, y_n$; il sottoproblema considera $x_1, \dots, x_i$ e $y_1, \dots, y_j$. Numero di stati: $O(nm)$.
3. **Intervallo.** Input $x_1, \dots, x_n$; il sottoproblema considera $x_i, \dots, x_j$. Numero di stati: $O(n^2)$ (tabella $n \times n$ triangolare superiore).
4. **Sottoalbero.** Input: albero radicato di n nodi; il sottoproblema è un sottoalbero radicato. Numero di stati: $O(n)$.

11.8 Memoization

Definizione 11.5 (Memoization). Tecnica (termine coniato da Donald Michie nel 1968, senza la “r” di *memorization*) che, in un programma ricorsivo, memorizza i risultati delle chiamate già eseguite in una struttura di supporto (hash table, array, matrice) indicizzata dai parametri, per riutilizzarli anziché ricalcolarli.

Confronto tra i tre approcci:

- **Tabulazione bottom-up:** risolve *tutte* le sottoistanze rispettando l'ordine di dipendenza. Iterativa, costanti moltiplicative ridotte.
- **Divide et impera:** risolve solo le sottoistanze necessarie ma ricalcola quelle comuni ogni volta.
- **Memoization:** mantiene la struttura ricorsiva del divide et impera ma riusa i risultati già incontrati. Può essere più efficiente della tabulazione se molte sottoistanze non sono necessarie; in generale ha overhead maggiore per le chiamate ricorsive.

11.8.1 Esempio: Zaino con memoization

Algorithm 42: ZAINO-MEMO(i, w)

Input: Indice oggetto i , capacità residua w ; P, v, p globali

Output: Valore ottimo per i primi i oggetti con capacità w

```

1 if  $K.CONTAINSKEY(i, w)$  then
2   return  $K.GET(i, w)$ 
3 if  $i = 0$  and  $w = 0$  then
4   return 0
5 if  $p[i] > w$  then
6    $k \leftarrow \text{ZAINO-MEMO}(i - 1, w)$  ; // troppo pesante
7 else
8    $k \leftarrow \max(\text{ZAINO-MEMO}(i - 1, w),$ 
9      $\text{ZAINO-MEMO}(i - 1, w - p[i]) + v[i])$ ;
10  $K.PUT((i, w), k)$ ;
11 return  $k$ 
```

Complessità: $O(nP)$, identica alla versione tabulare, ma con costanti moltiplicative più grandi per l'overhead delle chiamate ricorsive.

12. Algoritmi Avidi (Greedy)

12.1 Paradigma greedy

12.1.1 Definizione

Un algoritmo **greedy**, o algoritmo **avido**, risolve un problema di ottimizzazione costruendo la soluzione in modo incrementale. A ogni iterazione sceglie una componente che, secondo un criterio locale, risulta la migliore in quel momento; se tale componente può essere aggiunta senza violare i vincoli del problema, essa viene inserita nella soluzione.

Formalmente, dato un insieme di candidati

$$X = \{x_1, x_2, \dots, x_n\},$$

un criterio greedy ordina o seleziona i candidati in base alla loro appetibilità locale. L'algoritmo mantiene un insieme parziale $S \subseteq X$ e aggiunge un candidato x_i solo se $S \cup \{x_i\}$ resta ammissibile.

Un algoritmo greedy determina una soluzione ottima quando il problema soddisfa le due proprietà indicate nelle slide:

1. **proprietà della scelta greedy;**
2. **proprietà della sottostruttura ottima.**

12.1.2 Intuizione

Il punto centrale è che l'algoritmo non esplora tutte le soluzioni possibili. Si comporta invece come segue: guarda lo stato corrente, sceglie il candidato localmente migliore, decide se inserirlo, e non torna indietro.

Questa scelta è potente ma pericolosa. Una scelta localmente buona non è automaticamente una scelta globalmente buona. Per questo la correttezza di un algoritmo greedy non si dimostra mai dicendo soltanto che “sceglie il migliore”. Si deve dimostrare che esiste una soluzione ottima compatibile con quella scelta locale e che, dopo aver fissato tale scelta, resta un sottoproblema della stessa natura.

12.1.3 Motivazione

Molti problemi di ottimizzazione hanno uno spazio di soluzioni enorme. Per esempio, nel problema dell'albero di ricoprimento minimo, le soluzioni ammissibili sono alberi di ricoprimento; enumerarle tutte non è l'approccio seguito dal corso.

Il paradigma greedy serve quando si riesce a sostituire l'enumerazione esaustiva con una regola locale corretta. Se la regola è giustificata, l'algoritmo diventa semplice, efficiente e spesso implementabile con strutture dati come code di priorità.

12.1.4 Proprietà teoriche

Proprietà della scelta greedy. La scelta locale effettuata dall'algoritmo deve essere compatibile con almeno una soluzione ottima. In altre parole, se l'algoritmo sceglie un certo elemento x , bisogna poter dimostrare che esiste una soluzione ottima che contiene x .

Proprieta' della sottostruttura ottima. Dopo aver fissato una scelta greedy corretta, il problema residuo deve essere ancora un problema di ottimizzazione dello stesso tipo. Risolvendo ottimamente il sottoproblema residuo e aggiungendo la scelta fatta, si ottiene una soluzione ottima del problema originale.

Attenzione logica. Le due proprieta' sono condizioni da dimostrare nel problema specifico. Non sono conseguenze automatiche della parola "greedy".

12.1.5 Algoritmo

Lo schema generale riportato nelle slide e' il seguente.

Algorithm 43: GREEDY(X)

Input: $X = \{x_1, x_2, \dots, x_n\}$, criterio greedy di selezione, test di ammissibilita'

Output: Un sottoinsieme $S = \{x_{i_1}, \dots, x_{i_k}\} \subseteq X$ che ottimizza il valore obiettivo

```
1  $S \leftarrow \emptyset$ ;  
2 Ordina  $X$  secondo il criterio greedy di selezione;  
3 for  $i \leftarrow 1$  to  $n$  do  
4   if  $x_i$  puo' essere aggiunto a  $S$  then  
5      $S \leftarrow S \cup \{x_i\}$ ;  
6 return  $S$ ;
```

12.1.6 Esempio guidato

Si consideri un insieme astratto di candidati gia' ordinati per appetibilita':

$$x_4, x_2, x_5, x_1, x_3.$$

L'algoritmo procede cosi':

$$S_0 = \emptyset.$$

Se x_4 e' ammissibile, allora

$$S_1 = \{x_4\}.$$

Se x_2 e' ammissibile insieme a x_4 , allora

$$S_2 = \{x_4, x_2\}.$$

Se x_5 non e' ammissibile, viene scartato definitivamente:

$$S_3 = \{x_4, x_2\}.$$

L'algoritmo continua fino all'esaurimento dei candidati.

Il punto d'esame e' questo: lo scarto di x_5 e' definitivo. Quindi il test di ammissibilita' e la

dimostrazione di correttezza devono garantire che non servira' recuperarlo piu' avanti.

12.1.7 Grafico o diagramma

Candidati ordinati per appetibilita':

x4	x2	x5	x1	x3
si	si	no	si	no
v	v		v	

$S_0 = \{\}$

$S_1 = \{x_4\}$

$S_2 = \{x_4, x_2\}$

$S_3 = \{x_4, x_2\}$ x_5 scartato

$S_4 = \{x_4, x_2, x_1\}$

$S_5 = \{x_4, x_2, x_1\}$ x_3 scartato

12.1.8 Dimostrazione

Le slide non danno una dimostrazione generale del paradigma astratto. La correttezza viene dimostrata nei casi specifici:

- albero di ricoprimento minimo tramite tagli e archi leggeri;
- Dijkstra tramite invariante sui nodi gia' fissati;
- matroidi tramite proprieta' di scambio e sottostruttura ottima.

12.1.9 Analisi della complessita'

Per lo schema generale, se $n = |X|$, il costo dipende da:

- ordinamento dei candidati;
- costo del test di ammissibilita';
- costo dell'eventuale aggiornamento della soluzione parziale.

Se l'ordinamento costa $O(n \log n)$ e il test di ammissibilita' costa $f(n)$ per candidato, allora il costo e'

$$O(n \log n + nf(n)).$$

Le slide non fissano un costo unico per il paradigma astratto, perche' il costo dipende dal problema e dalla struttura dati usata.

12.1.10 Errori tipici d'esame

Errore grave: affermare che un algoritmo e' corretto perche' "a ogni passo prende il migliore". Questo non dimostra nulla. Bisogna dimostrare che la scelta locale e' sicura rispetto a una soluzione ottima.

Errore frequente: confondere ammissibilita' con ottimalita'. Un candidato ammissibile puo' essere aggiunto senza violare i vincoli, ma cio' non significa automaticamente che porti a una soluzione ottima.

12.1.11 Possibili domande d'esame

- Definire il paradigma greedy.
- Enunciare le due proprieta' che consentono a un algoritmo greedy di determinare una soluzione ottima.
- Spiegare perche' una scelta localmente ottima non e' necessariamente globalmente ottima.
- Scrivere lo schema generale di un algoritmo greedy.

12.1.12 Collegamenti teorici

Il paradigma greedy e' collegato a:

- alberi di ricoprimento minimo, tramite archi sicuri;
- cammini minimi da singola sorgente, tramite stime definitive;
- matroidi, tramite indipendenza e proprieta' di scambio;
- scheduling di attivita' unitarie con penalita', tramite massimizzazione delle penalita' evitate.

12.2 Albero di ricoprimento minimo e algoritmo di Prim

12.2.1 Definizione

Il problema dell'**albero di ricoprimento minimo** e' definito nelle slide come segue.

Input: un grafo

$$G = (V, E)$$

non orientato, connesso, di ordine n , e una funzione peso

$$w : E \rightarrow \mathbb{R}.$$

Output: un albero di ricoprimento minimo

$$T = (V, E_T)$$

tale che:

- T e' un albero con n nodi;
- $E_T \subseteq E$;
- il peso totale

$$w(T) = \sum_{e \in E_T} w(e)$$

e' minimo tra tutti gli alberi di ricoprimento di G .

Un **albero di ricoprimento** e' quindi un sottografo che contiene tutti i vertici di G , e' connesso ed e' aciclico.

12.2.2 Intuizione

L'algoritmo di Prim costruisce progressivamente un albero. Parte da un nodo r e, a ogni passo, aggiunge all'albero corrente un nuovo nodo tramite un arco di peso minimo tra quelli che collegano l'albero gia' costruito al resto del grafo.

L'intuizione corretta non e': "prendo l'arco piu' leggero del grafo". L'intuizione corretta e': "dato il taglio tra i nodi gia' raggiunti e quelli non ancora raggiunti, prendo un arco leggero che attraversa quel taglio". Questa distinzione e' essenziale.

12.2.3 Motivazione

Enumerare tutti gli alberi di ricoprimento non e' l'approccio usato nelle slide. Si cerca invece una regola locale che permetta di aggiungere archi senza distruggere la possibilita' di completare la soluzione in un albero di ricoprimento minimo.

Il Teorema dell'arco sicuro fornisce esattamente questa garanzia.

12.2.4 Proprieta' teoriche

Sia $A \subseteq E$ un insieme di archi che appartiene ad almeno un albero di ricoprimento minimo.

Taglio. Dato $S \subseteq V$, la coppia

$$(S, V \setminus S)$$

e' un **taglio** del grafo.

Arco che attraversa un taglio. Un arco $(u, v) \in E$ attraversa il taglio $(S, V \setminus S)$ se un suo estremo appartiene a S e l'altro appartiene a $V \setminus S$.

Taglio che rispetta A . Un taglio rispetta A se nessun arco di A attraversa il taglio.

Arco leggero. Un arco che attraversa un taglio e ha peso minimo tra tutti gli archi che attraversano quel taglio e' detto **arco leggero**.

Teorema 12.1 (Arco sicuro). *Sia $G = (V, E)$ un grafo pesato e connesso, sia w la funzione peso, e sia $A \subseteq E$ un insieme di archi che appartiene a un albero di ricoprimento minimo. Sia $(S, V \setminus S)$ un taglio di G che rispetta A e sia (u, v) un arco leggero del taglio. Allora (u, v) e' un arco sicuro per A : si puo' aggiungere ad A e $A \cup \{(u, v)\}$ rimane sottoinsieme di un albero di ricoprimento minimo.*

12.2.5 Algoritmo

Nell'algoritmo di Prim ogni nodo v ha:

- $v.key$, cioe' il peso minimo di un arco che collega v all'albero corrente;
- $v.\pi$, cioe' il predecessore di v nell'albero costruito.

Algorithm 44: PRIM(G, w, r)

Input: $G = (V, E)$, funzione peso w , nodo di partenza $r \in V$

Output: Un albero di ricoprimento minimo con radice r , definito tramite π

```

1 foreach  $v \in V$  do
2    $v.key \leftarrow \infty$ ;
3    $v.\pi \leftarrow \text{NULL}$ ;
4  $r.key \leftarrow 0$ ;
5  $Q \leftarrow V$ ;
6 BUILDMINPRIORITYQUEUE( $Q$ );           //  $Q$  e' ordinata rispetto a  $key$ 
7 while  $Q \neq \emptyset$  do
8    $u \leftarrow \text{EXTRACTMIN}(Q)$ ;
9   foreach  $v \in G.Adj[u]$  do
10    if  $v \in Q$  e  $w(u, v) < v.key$  then
11       $v.\pi \leftarrow u$ ;
12      DECREASEKEY( $Q, v, w(u, v)$ );      //  $v.key$  diventa  $w(u, v)$ 

```

12.2.6 Esempio guidato

Consideriamo il seguente grafo non orientato, pesato, e scegliamo A come radice.

$$V = \{A, B, C, D\}.$$

Arco	Peso
(A, B)	1
(A, C)	4
(B, D)	2
(C, D)	3

Stato iniziale:

$$A.key = 0, \quad B.key = C.key = D.key = \infty.$$

Passo	Nodo estratto	Aggiornamenti	Archi fissati tramite π
0	—	$A.key = 0$	—
1	A	$B.key = 1, B.\pi = A; C.key = 4, C.\pi = A$	—
2	B	$D.key = 2, D.\pi = B$	(A, B)
3	D	$C.key = 3, C.\pi = D$	$(A, B), (B, D)$
4	C	nessuno	$(A, B), (B, D), (D, C)$

L'albero finale e'

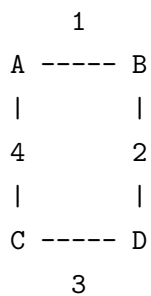
$$E_T = \{(A, B), (B, D), (D, C)\}$$

e ha peso totale

$$1 + 2 + 3 = 6.$$

12.2.7 Grafico o diagramma

Grafo:



Prim da A:

Q iniziale:

A:0, B:inf, C:inf, D:inf

Estrai A:

B:1 tramite A

C:4 tramite A

D:inf

Estrai B:

D:2 tramite B

C resta 4

Estrai D:

C:3 tramite D

Estrai C:
fine

MST:

A -- B -- D -- C

12.2.8 Dimostrazione

Dimostriamo il Teorema dell'arco sicuro secondo la dimostrazione “taglia-e-cuci” riportata nelle slide.

Sia T un albero di ricoprimento minimo che contiene A .

Se $(u, v) \in T$, allora non c'è nulla da dimostrare: l'arco è già contenuto in un MST che contiene A .

Supponiamo quindi che

$$(u, v) \notin T.$$

Poiché T è un albero, tra u e v esiste in T un unico cammino semplice. Aggiungendo l'arco (u, v) a T , si forma un ciclo.

Poiché u e v stanno su lati opposti del taglio $(S, V \setminus S)$, il cammino in T da u a v deve attraversare il taglio almeno una volta. Esiste quindi un arco (x, y) del cammino che attraversa il taglio.

Il taglio rispetta A , cioè nessun arco di A attraversa il taglio. Poiché (x, y) attraversa il taglio, segue che

$$(x, y) \notin A.$$

Rimuoviamo (x, y) da T e aggiungiamo (u, v) :

$$T' = T - \{(x, y)\} + \{(u, v)\}.$$

Togliere (x, y) spezza T in due componenti; aggiungere (u, v) le ricongiunge. Quindi T' è ancora un albero di ricoprimento.

Poiché (u, v) è un arco leggero del taglio,

$$w(u, v) \leq w(x, y).$$

Pertanto

$$w(T') = w(T) - w(x, y) + w(u, v) \leq w(T).$$

Ma T è un albero di ricoprimento minimo, quindi nessun albero di ricoprimento può avere peso minore di $w(T)$:

$$w(T) \leq w(T').$$

Dalle due disuguaglianze segue

$$w(T') = w(T).$$

Dunque T' e' anch'esso un albero di ricoprimento minimo.

Infine, $A \subseteq T$ per ipotesi e l'arco rimosso (x, y) non appartiene ad A ; quindi

$$A \subseteq T'.$$

Inoltre $(u, v) \in T'$. Dunque $A \cup \{(u, v)\}$ e' contenuto in un albero di ricoprimento minimo. L'arco (u, v) e' sicuro per A .

Correttezza di Prim. Durante Prim, l'insieme dei nodi estratti dalla coda definisce un taglio tra nodi gia' raggiunti e nodi non ancora raggiunti. L'arco scelto per collegare il prossimo nodo e' leggero rispetto a tale taglio, perche' la coda di min-priorita' seleziona il nodo con chiave minima. Per il Teorema dell'arco sicuro, ogni arco aggiunto mantiene la possibilita' di completare la soluzione in un MST. Quando tutti i nodi sono stati estratti, gli archi definiti dai predecessori π formano un albero di ricoprimento minimo.

12.2.9 Analisi della complessita'

Siano

$$n = |V|, \quad m = |E|.$$

Coda di priorita' semplice con ricerca lineare del minimo.

- Inizializzazione dei nodi: $O(n)$.
- Il ciclo `while` viene eseguito n volte.
- Ogni `EXTRACTMIN` costa $O(n)$.
- Tutti i cicli sugli adiacenti considerano complessivamente $2m$ occorrenze di archi.
- Il test $v \in Q$ puo' essere fatto in tempo costante mantenendo un attributo booleano o equivalente.
- Ogni `DECREASEKEY` costa $O(1)$.

Quindi:

$$O(n) + O(n^2) + O(m) = O(n^2 + m) = O(n^2),$$

poiche' in un grafo connesso $m \leq n^2$.

Coda di priorita' implementata con min-heap.

- Inizializzazione: $O(n)$.
- BUILDMINHEAP: $O(n)$.
- n estrazioni, ciascuna $O(\log n)$: totale $O(n \log n)$.
- Fino a $O(m)$ operazioni di aggiornamento, ciascuna $O(\log n)$: totale $O(m \log n)$.

Quindi:

$$O(n + n \log n + m \log n) = O(m \log n).$$

Le slide osservano che l'implementazione con heap conviene quando

$$m = o\left(\frac{n^2}{\log n}\right).$$

Spazio. Oltre al grafo in input, Prim mantiene key , π e la coda Q , quindi usa spazio aggiuntivo $O(n)$.

12.2.10 Errori tipici d'esame

Confondere Prim con una scelta globale dell'arco piu' leggero del grafo. Prim sceglie un arco leggero rispetto al taglio indotto dall'albero corrente.

Confondere $v.key$ con una distanza dalla radice. In Prim, $v.key$ e' il peso del miglior arco attualmente noto per collegare v all'albero corrente.

Dimenticare che il grafo in input deve essere non orientato e connesso, come indicato nella definizione del problema MST delle slide.

12.2.11 Possibili domande d'esame

- Definire taglio, arco che attraversa un taglio, taglio che rispetta A , arco leggero.
- Enunciare e dimostrare il Teorema dell'arco sicuro.
- Scrivere l'algoritmo di Prim.
- Analizzare Prim con coda semplice e con min-heap.
- Spiegare perche' l'arco scelto da Prim e' sicuro.

12.2.12 Collegamenti teorici

Prim e' un'applicazione diretta del paradigma greedy: la scelta locale e' l'arco leggero del taglio corrente; la correttezza e' garantita dal Teorema dell'arco sicuro.

Il problema MST e' anche collegato ai matroidi grafici: gli insiemi indipendenti sono foreste, e i massimali sono alberi di ricoprimento.

12.3 Cammini minimi da singola sorgente e algoritmo di Dijkstra

12.3.1 Definizione

Il problema dei **cammini minimi da singola sorgente** e' indicato nelle slide come SSSP, *Single-Source Shortest Paths*.

Input:

- un grafo orientato e connesso

$$G = (V, E)$$

di ordine n ;

- una funzione peso

$$w : E \rightarrow \mathbb{R}_{\geq 0},$$

nelle slide indicata come funzione a valori non negativi;

- un nodo sorgente $s \in V$.

Output: determinare i cammini minimi da s verso tutti gli altri nodi.

Indichiamo con

$$\delta(s, u)$$

la distanza minima da s a u .

12.3.2 Intuizione

Dijkstra mantiene, per ogni nodo v , una stima $v.d$ della distanza minima da s a v . Tale stima e' sempre un limite superiore: inizialmente e' $+\infty$ per tutti i nodi diversi da s e vale 0 per s .

A ogni passo l'algoritmo sceglie il nodo u non ancora fissato con stima minima. Con pesi non negativi, tale stima non potra' piu' essere migliorata in futuro, quindi diventa definitiva.

12.3.3 Motivazione

Il problema richiede tutti i cammini minimi da una sorgente. La strategia greedy evita di provare tutti i cammini: usa la non negativita' dei pesi per rendere definitiva, passo dopo passo, la migliore stima corrente.

La condizione sui pesi non negativi e' essenziale. Le slide affermano esplicitamente che l'algoritmo di Dijkstra funziona solo per grafi con pesi non negativi.

12.3.4 Proprieta' teoriche

Teorema 12.2 (Sottostruttura ottima dei cammini minimi). *Dato un cammino minimo*

$$p = [v_1, v_2, \dots, v_k]$$

dal vertice v_1 al vertice v_k , per qualsiasi i, j con

$$1 \leq i \leq j \leq k,$$

il sottocammino

$$p_{ij} = [v_i, v_{i+1}, \dots, v_j]$$

e' un cammino minimo da v_i a v_j .

Stima della distanza. Per ogni nodo v , l'attributo $v.d$ mantiene una stima del cammino minimo da s a v .

Rilassamento. Dato un arco $(u, v) \in E$, se passando per u si migliora la stima di v , cioe' se

$$v.d > u.d + w(u, v),$$

allora si aggiorna:

$$v.d \leftarrow u.d + w(u, v), \quad v.\pi \leftarrow u.$$

Insiemi usati dall'algoritmo.

- S contiene i nodi per cui la distanza minima e' gia' stata determinata.
- Q contiene i nodi non ancora fissati, organizzati in coda di min-priorita' rispetto a d .

12.3.5 Algoritmo

Algorithm 45: DIJKSTRA(G, s, w)

Input: $G = (V, E)$, sorgente $s \in V$, funzione peso w **Output:** Cammini minimi da s rappresentati dagli attributi d e π dei nodi

```

1 foreach  $v \in V$  do
2    $v.d \leftarrow \infty$ ;
3    $v.\pi \leftarrow \text{NULL}$ ;
4  $s.d \leftarrow 0$ ;
5  $Q \leftarrow V$ ;
6  $S \leftarrow \emptyset$ ;
7 BUILDMINPRIORITYQUEUE( $Q$ );                                // coda rispetto a  $d$ 
8 while  $S \subset V$  do
9    $u \leftarrow \text{EXTRACTMIN}(Q)$ ;
10   $S \leftarrow S \cup \{u\}$ ;
11  foreach  $v \in G.\text{Adj}[u]$  do
12    if  $v \in Q$  e  $v.d > u.d + w(u, v)$  then
13       $v.d \leftarrow u.d + w(u, v)$ ;
14      DECREASEKEY( $Q, v, v.d$ );
15       $v.\pi \leftarrow u$ ;

```

12.3.6 Esempio guidato

Consideriamo il grafo orientato con sorgente s :

$$(s, a) = 1, \quad (s, b) = 4, \quad (a, b) = 2, \quad (a, c) = 6, \quad (b, c) = 3.$$

Stato iniziale:

$$s.d = 0, \quad a.d = b.d = c.d = \infty.$$

Passo	Nodo estratto	Rilassamenti efficaci	Distanze dopo il passo
0	—	—	$d(s) = 0, d(a) = \infty, d(b) = \infty, d(c) = \infty$
1	s	$a.d = 1, b.d = 4$	$0, 1, 4, \infty$
2	a	$b.d = 3, c.d = 7$	$0, 1, 3, 7$
3	b	$c.d = 6$	$0, 1, 3, 6$
4	c	—	$0, 1, 3, 6$

I predecessori finali sono:

$$a.\pi = s, \quad b.\pi = a, \quad c.\pi = b.$$

Quindi il cammino minimo da s a c e'

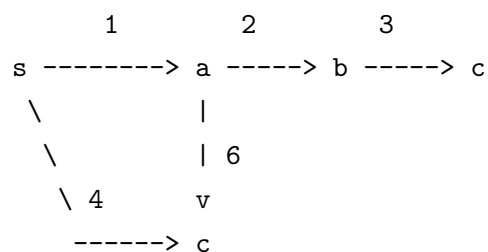
$$s \rightarrow a \rightarrow b \rightarrow c$$

di peso

$$1 + 2 + 3 = 6.$$

12.3.7 Grafico o diagramma

Grafo orientato:



Esecuzione:

inizio:

$S = \{s\}$

$Q = \{s:0, a:\text{inf}, b:\text{inf}, c:\text{inf}\}$

estrai s :

$S = \{s\}$

$Q = \{a:1, b:4, c:\text{inf}\}$

estrai a :

$S = \{s, a\}$

$Q = \{b:3, c:7\}$

estrai b :

$S = \{s, a, b\}$

$Q = \{c:6\}$

estrai c :

$S = \{s, a, b, c\}$

$Q = \{\}$

12.3.8 Dimostrazione

Teorema 12.3 (Correttezza di Dijkstra). *Dato un grafo $G = (V, E)$ con pesi non negativi w e sorgente s , l'algoritmo di Dijkstra termina e, per ogni $u \in V$, restituisce*

$$u.d = \delta(s, u).$$

La dimostrazione delle slide usa il seguente invariante di ciclo:

all'inizio di ogni ciclo **while**, $v.d = \delta(s, v)$ per ciascun nodo $v \in S$.

Inizializzazione. All'inizio

$$S = \emptyset.$$

L'invariante e' vero banalmente, perche' non esiste alcun nodo in S per cui verificarlo.

Mantenimento. Assumiamo che all'inizio di una certa iterazione valga

$$v.d = \delta(s, v) \quad \forall v \in S.$$

Sia u il nodo estratto da Q e aggiunto a S in questa iterazione. Dobbiamo dimostrare che

$$u.d = \delta(s, u).$$

Sia p un cammino minimo da s a u . Sia y il primo nodo di p che non appartiene a S e sia x il suo predecessore in p . Allora $x \in S$.

Poiche' y precede u lungo un cammino minimo e tutti i pesi sono non negativi,

$$\delta(s, y) \leq \delta(s, u).$$

Poiche' u viene scelto da EXTRACTMIN, esso ha stima minima tra i nodi in Q ; dato che $y \in Q$, vale

$$u.d \leq y.d.$$

Inoltre $u.d$ e' una stima ottenuta tramite cammini effettivamente scoperti da s a u , quindi non puo' essere minore della distanza minima:

$$\delta(s, u) \leq u.d.$$

Per ipotesi induttiva,

$$x.d = \delta(s, x).$$

Quando x e' stato aggiunto a S , l'arco (x, y) e' stato rilassato. Poiche' il sottocammino da s a y

lungo p e' minimo, il rilassamento porta a

$$y.d = \delta(s, y).$$

Mettendo insieme le disuguaglianze:

$$\delta(s, y) \leq \delta(s, u) \leq u.d \leq y.d = \delta(s, y).$$

La prima e l'ultima quantita' sono uguali, quindi tutte le quantita' intermedie sono uguali:

$$u.d = \delta(s, u).$$

L'invariante rimane vero dopo l'inserimento di u in S .

Conclusione. Quando l'algoritmo termina, tutti i vertici appartengono a S . Per l'invariante, per ogni $u \in V$ vale

$$u.d = \delta(s, u).$$

Quindi l'algoritmo restituisce le distanze minime corrette.

12.3.9 Analisi della complessita'

Siano

$$n = |V|, \quad m = |E|.$$

L'analisi e' analoga a quella di Prim.

- Con coda di priorita' come lista semplice:

$$O(n^2).$$

- Con coda di priorita' implementata tramite min-heap:

$$O(m \log n).$$

- Con Fibonacci heap:

$$O(m + n \log n).$$

Lo spazio aggiuntivo, oltre alla rappresentazione del grafo, e' $O(n)$ per d , π , S e Q .

12.3.10 Errori tipici d'esame

Dimenticare l'ipotesi dei pesi non negativi. La catena di disuguaglianze nella dimostrazione usa il fatto che, se y precede u su un cammino minimo, allora $\delta(s, y) \leq \delta(s, u)$.

Confondere S e Q . S contiene i nodi già definitivi; Q contiene i nodi ancora da fissare.

Pensare che il rilassamento renda sempre definitiva una distanza. Il rilassamento migliora una stima; la stima diventa definitiva solo quando il nodo viene estratto come minimo e inserito in S .

12.3.11 Possibili domande d'esame

- Definire il problema SSSP.
- Spiegare il rilassamento di un arco.
- Scrivere l'algoritmo di Dijkstra.
- Dimostrare la correttezza di Dijkstra tramite l'invariante su S .
- Spiegare perché l'algoritmo richiede pesi non negativi.
- Confrontare le complessità con lista semplice, min-heap e Fibonacci heap.

12.3.12 Collegamenti teorici

Dijkstra è greedy perché sceglie a ogni iterazione il nodo non ancora fissato con stima minima. La correttezza dipende dalla sottostruttura ottima dei cammini minimi e dalla non negatività dei pesi.

La struttura dati centrale è la coda di min-priorità, come in Prim; tuttavia il significato della chiave è diverso:

Algoritmo	Significato della chiave
Prim	peso minimo per collegare il nodo all'albero corrente
Dijkstra	stima della distanza minima dalla sorgente

12.4 Matroidi

12.4.1 Definizione

Un **matroide** è una coppia ordinata

$$M = (S, \mathcal{I})$$

che soddisfa le seguenti condizioni:

1. S e' un insieme finito;
2. $\mathcal{I}$ e' un insieme non vuoto di sottoinsiemi di S tale che

$$B \in \mathcal{I} \wedge A \subseteq B \implies A \in \mathcal{I};$$

questa proprieta' e' detta **ereditaria**, e gli elementi di $\mathcal{I}$ sono detti **sottoinsiemi indipendenti**;

3. vale la **proprieta' di scambio**:

$$A \in \mathcal{I}, B \in \mathcal{I}, |A| < |B| \implies \exists x \in B \setminus A : A \cup \{x\} \in \mathcal{I}.$$

12.4.2 Intuizione

Un matroide astrae il concetto di indipendenza. In un grafo, per esempio, un insieme di archi e' indipendente se non crea cicli. In altri problemi, l'indipendenza puo' significare che un insieme di attivita' puo' essere schedulato senza ritardi.

La proprieta' di scambio e' il cuore della teoria: se un insieme indipendente B e' piu' grande di un altro insieme indipendente A , allora esiste almeno un elemento di B che puo' essere aggiunto ad A senza perdere indipendenza.

12.4.3 Motivazione

Le slide introducono i matroidi per stabilire condizioni sufficienti affinche' una soluzione greedy sia ottima. Se un problema puo' essere modellato come ricerca di un sottoinsieme ottimo in un matroide pesato, allora la procedura greedy ordinata per peso e' corretta.

12.4.4 Proprieta' teoriche

Estensione. Un elemento $x \notin A$ e' un'**estensione** di $A \in \mathcal{I}$ se puo' essere aggiunto preservando l'indipendenza:

$$A \cup \{x\} \in \mathcal{I}.$$

Sottoinsieme massimale. Un sottoinsieme $A \in \mathcal{I}$ e' **massimale** se non ammette estensioni.

Cardinalita' dei massimali. Tutti i sottoinsiemi massimali di un matroide hanno la stessa dimensione. Infatti, se A e B fossero massimali con $|A| < |B|$, la proprieta' di scambio garantirebbe l'esistenza di un elemento $x \in B \setminus A$ tale che $A \cup \{x\} \in \mathcal{I}$, contraddicendo la massimalita' di A .

Matroide pesato. Se un matroide ha una funzione peso

$$w : S \rightarrow \mathbb{R}_{>0},$$

allora e' detto **matroide pesato**. Il peso di un sottoinsieme $A \subseteq S$ e'

$$w(A) = \sum_{x \in A} w(x).$$

Matroide grafico. Dato un grafo non orientato

$$G = (V, E),$$

il matroide grafico

$$M_G = (S_G, \mathcal{I}_G)$$

e' definito da:

$$S_G = E,$$

e, per $A \subseteq E$,

$$A \in \mathcal{I}_G \iff A \text{ e' aciclico.}$$

In altre parole, $\mathcal{I}_G$ contiene tutti i sottoinsiemi di archi che inducono una foresta.

12.4.5 Algoritmo

Il problema del **massimo di un matroide pesato** e' il seguente.

Input: un matroide pesato

$$M = (S, \mathcal{I}).$$

Output: un sottoinsieme indipendente

$$A \in \mathcal{I}$$

di peso massimo, detto anche sottoinsieme ottimo.

La procedura greedy delle slide e' la seguente.

Algorithm 46: GREEDY(M, w)

Input: Matroide pesato $M = (S, \mathcal{I})$, funzione peso w

Output: Sottoinsieme ottimo $A \in \mathcal{I}$

```
1  $A \leftarrow \emptyset;$  //  $A$  e' indipendente
2 Ordina  $S$  in ordine decrescente rispetto a  $w$ ;
3 for  $i \leftarrow 1$  to  $|S|$  do
4   if  $A \cup \{s_i\} \in \mathcal{I}$  then
5      $A \leftarrow A \cup \{s_i\};$ 
6 return  $A;$ 
```

L'invariante immediato e':

$$A \in \mathcal{I}$$

all'inizio di ogni iterazione.

12.4.6 Esempio guidato

Consideriamo il grafo triangolo con archi

$$e_1 = (a, b), \quad e_2 = (b, c), \quad e_3 = (a, c).$$

Nel matroide grafico, gli insiemi indipendenti sono quelli aciclici:

$$\emptyset, \{e_1\}, \{e_2\}, \{e_3\}, \{e_1, e_2\}, \{e_1, e_3\}, \{e_2, e_3\}.$$

L'insieme

$$\{e_1, e_2, e_3\}$$

non e' indipendente, perche' forma un ciclo.

Supponiamo

$$w(e_1) = 5, \quad w(e_2) = 4, \quad w(e_3) = 3.$$

L'ordine greedy e'

$$e_1, e_2, e_3.$$

L'algoritmo costruisce:

$$A_0 = \emptyset, \quad A_1 = \{e_1\}, \quad A_2 = \{e_1, e_2\}.$$

Quando considera e_3 , l'insieme

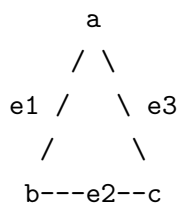
$$\{e_1, e_2, e_3\}$$

forma un ciclo, quindi e_3 viene scartato. Il risultato e'

$$A = \{e_1, e_2\}.$$

12.4.7 Grafico o diagramma

Matroide grafico su un triangolo:



Indipendenti:

{ } si

$\{e_1\}, \{e_2\}, \{e_3\}$	si
$\{e_1, e_2\}$	si
$\{e_1, e_3\}$	si
$\{e_2, e_3\}$	si
$\{e_1, e_2, e_3\}$	no: ciclo

12.4.8 Dimostrazione

Il matroide grafico e' un matroide. Verifichiamo le tre proprieta' indicate nella definizione.

Primo, $S_G = E$ e' finito perche' si considerano grafi finiti.

Secondo, $\mathcal{I}_G$ e' ereditario: ogni sottoinsieme di una foresta e' ancora una foresta. Infatti, togliere archi non puo' creare un ciclo.

Terzo, dimostriamo la proprieta' di scambio. Siano

$$A, B \in \mathcal{I}_G$$

con

$$|A| < |B|.$$

Allora

$$G_A = (V, A), \quad G_B = (V, B)$$

sono foreste. Una foresta con $|V|$ nodi e k archi contiene esattamente $|V| - k$ alberi. Poiche'

$$|A| < |B|,$$

la foresta G_B ha meno componenti connesse della foresta G_A .

Quindi esiste un albero T di G_B che contiene vertici appartenenti a due alberi distinti di G_A . Consideriamo un arco (u, v) di T tale che u e v appartengano a due alberi distinti di G_A .

Aggiungere (u, v) ad A fonde due alberi distinti di G_A e non crea alcun ciclo. Pertanto

$$A \cup \{(u, v)\} \in \mathcal{I}_G.$$

La proprieta' di scambio e' verificata.

Lemma 1: unicity dell'estensione. Se un elemento $x \in S$ non e' un'estensione di $\emptyset$, allora x non e' un'estensione di nessun sottoinsieme indipendente A di S .

Dimostriamo la contronominale, come suggerito dalle slide. Supponiamo che x sia estensione di qualche insieme indipendente A . Allora

$$A \cup \{x\} \in \mathcal{I}.$$

Poiche' $\{x\} \subseteq A \cup \{x\}$ e $\mathcal{I}$ e' ereditaria, segue

$$\{x\} \in \mathcal{I}.$$

Dunque x e' estensione di $\emptyset$. La contronominale e' vera, quindi e' vero il lemma.

Lemma 2: elemento massimo in una soluzione ottima. Sia S ordinato in modo decrescente rispetto al peso e sia x il primo elemento con $\{x\}$ indipendente. Allora esiste un sottoinsieme ottimo che contiene x .

Sia B un sottoinsieme ottimo tale che $x \notin B$. Si costruisce un insieme indipendente A partendo da

$$A = \{x\}$$

e usando la proprieta' di scambio con B fino a ottenere un insieme della stessa cardinalita' di B . Alla fine, per qualche $y \in B$,

$$A = B \setminus \{y\} \cup \{x\}.$$

Poiche' x e' il primo elemento aggiungibile nell'ordine decrescente dei pesi,

$$w(y) \leq w(x).$$

Quindi

$$w(A) = w(B) - w(y) + w(x) \geq w(B).$$

Ma B e' ottimo, quindi anche A e' ottimo e contiene x .

Teorema: sottostruttura ottima dei matroidi. Sia x il primo elemento scelto dalla procedura greedy su

$$M = (S, \mathcal{I}).$$

Il problema di trovare un sottoinsieme di peso massimo che contiene x si riduce a trovare un sottoinsieme di peso massimo nel sottomatroide

$$M' = (S', \mathcal{I}'),$$

dove

$$S' = \{y \in S \mid \{x, y\} \in \mathcal{I}\}$$

e

$$\mathcal{I}' = \{B \subseteq S' \mid B \cup \{x\} \in \mathcal{I}\}.$$

Infatti, se A e' un sottoinsieme indipendente di M che contiene x , allora

$$A' = A \setminus \{x\}$$

e' indipendente in M' . Viceversa, se A' e' indipendente in M' , allora

$$A' \cup \{x\}$$

e' indipendente in M . Inoltre

$$w(A) = w(A') + w(x).$$

Poiche' $w(x)$ e' costante, massimizzare $w(A)$ tra gli insiemi che contengono x equivale a massimizzare $w(A')$ nel sottomatroide.

Correttezza dell'algoritmo greedy sui matroidi. Per il Lemma 1, quando un elemento non puo' essere aggiunto, non diventera' aggiungibile piu' avanti. Per il Lemma 2, il primo elemento aggiungibile di peso massimo appartiene ad almeno una soluzione ottima. Dopo averlo scelto, il Teorema di sottostruttura ottima riduce il problema a un sottomatroide. Ripetendo lo stesso ragionamento a ogni iterazione, l'algoritmo greedy determina un sottoinsieme ottimo.

12.4.9 Analisi della complessita'

Sia

$$n = |S|.$$

Il costo della procedura e':

- ordinamento di S in ordine decrescente:

$$O(n \log n);$$

- n verifiche di indipendenza.

Se il costo di una verifica

$$A \cup \{s_i\} \in \mathcal{I}$$

e' $f(n)$, allora il costo totale e'

$$O(n \log n + nf(n)).$$

Lo spazio aggiuntivo e' almeno $O(n)$ per rappresentare l'insieme corrente A e l'ordine degli elementi, oltre alla rappresentazione del matroide e del test di indipendenza.

12.4.10 Errori tipici d'esame

Applicare la proprieta' di scambio senza controllare l'ipotesi $|A| < |B|$. Senza questa disuguaglianza, la proprieta' non dice nulla.

Confondere massimale con massimo. Un insieme massimale non puo' essere esteso; un insieme massimo ha peso ottimo. Nei matroidi pesati, l'algoritmo greedy cerca un massimale di peso massimo.

Nel matroide grafico, dire che un insieme di archi e' indipendente perche' e' connesso e' sbagliato. L'indipendenza corrisponde all'assenza di cicli.

12.4.11 Possibili domande d'esame

- Dare la definizione formale di matroide.
- Definire la proprieta' ereditaria e la proprieta' di scambio.
- Dimostrare che il matroide grafico e' un matroide.
- Definire estensione e sottoinsieme massimale.
- Scrivere e analizzare la procedura greedy per un matroide pesato.
- Dimostrare la correttezza dell'algoritmo greedy sui matroidi.

12.4.12 Collegamenti teorici

I matroidi spiegano in forma generale perche' alcuni algoritmi greedy sono corretti. Il matroide grafico collega questa teoria agli alberi di ricoprimento: i sottoinsiemi indipendenti sono foreste e i sottoinsiemi massimali sono alberi di ricoprimento.

Per problemi di minimizzazione su insiemi massimali, le slide propongono un esercizio sulla modifica della funzione peso. Una trasformazione deve convertire il confronto di minimo in un confronto di massimo preservando l'ordine tra insiemi massimali, che hanno tutti la stessa cardinalita'.

12.5 Programmazione di attivita' unitarie con penalita'

12.5.1 Definizione

Il problema ufficiale delle slide e' la **programmazione di attivita' di durata unitaria su singolo processore**.

Input:

- un insieme di attivita'

$$S = \{a_1, a_2, \dots, a_n\};$$

- scadenze

$$D = \{d_1, \dots, d_n\}, \quad 1 \leq d_i \leq n;$$

- penalita'

$$W = \{w_1, \dots, w_n\}, \quad w_i \geq 0.$$

Ogni attivita' ha durata unitaria. Se l'attivita' a_i non termina entro il tempo d_i , si paga una penalita' w_i .

Output: determinare un piano di programmazione su singolo processore che minimizza la somma delle penalita' pagate.

12.5.2 Intuizione

Un'attivita' puo' essere:

- **in anticipo**, se termina entro la propria scadenza;
- **in ritardo**, se termina dopo la propria scadenza.

Minimizzare le penalita' delle attivita' in ritardo equivale a massimizzare la somma delle penalita' delle attivita' in anticipo. Infatti, la somma totale delle penalita' e' fissa; scegliere quali attivita' salvare dal ritardo massimizza la penalita' non pagata.

12.5.3 Motivazione

Questo problema e' l'applicazione nelle slide della teoria dei matroidi. Si definisce un insieme indipendente come un insieme di attivita' che puo' essere schedulato senza ritardi. Poi si dimostra che tali insiemi indipendenti formano un matroide. A quel punto il greedy su matroide pesato fornisce una soluzione ottima.

12.5.4 Proprieta' teoriche

Forma canonica. Un piano generico puo' essere riordinato mettendo prima le attivita' in anticipo e poi quelle in ritardo.

Se le attivita' in anticipo sono ordinate per scadenza crescente, la programmazione e' in **forma canonica**.

Perche' lo scambio e' lecito? Se due attivita' in anticipo a_i e a_j terminano agli istanti k e $k+1$, e se $d_j < d_i$, allora a_j puo' essere messa prima di a_i senza creare ritardi: poiche' a_j era in anticipo a tempo $k+1$, vale $k+1 \leq d_j$, quindi anche a_i , spostata a $k+1$, resta in anticipo perche' $d_j < d_i$.

Indipendenza. Un insieme di attivita' e' **indipendente** se nessuna attivita' dell'insieme e' in ritardo in una programmazione opportuna.

Sia $\mathcal{I}$ l'insieme dei sottoinsiemi indipendenti.

Funzione $N_t(A)$. Per $A \subseteq S$ e $t = 1, \dots, n$, sia

$$N_t(A) = |\{a_i \in A \mid d_i \leq t\}|.$$

Cioe' $N_t(A)$ conta quante attivita' di A hanno scadenza al piu' t .

Le slide affermano:

$$A \text{ indipendente} \iff N_t(A) \leq t \quad \forall t = 1, \dots, n.$$

Teorema 12.4. *Sia S l'insieme delle attivita' unitarie con penalita' e sia $\mathcal{I}$ l'insieme di tutti gli insiemi indipendenti di attivita'. Allora*

$$M = (S, \mathcal{I})$$

e' un matroide.

12.5.5 Algoritmo

La soluzione applica il greedy su matroide pesato.

Algorithm 47: UNITTASKGREEDY(S, D, W)

Input: Attivita' unitarie S , scadenze D , penalita' W

Output: Insieme di attivita' in anticipo di penalita' massima

```

1  $A \leftarrow \emptyset$ ;
2 Ordina le attivita' in ordine decrescente rispetto alla penalita'  $w_i$ ;
3 foreach attivita'  $a_i$  in tale ordine do
4   if  $A \cup \{a_i\}$  e' indipendente then
5      $A \leftarrow A \cup \{a_i\}$ ;
6 return  $A$ ;
```

Una volta trovato A , si costruisce il piano canonico:

- prima le attivita' in A , ordinate per scadenza crescente;
- poi le attivita' non in A , che saranno in ritardo.

12.5.6 Esempio guidato

L'esempio delle slide usa attivita' gia' ordinate per penalita' decrescente:

a_i	1	2	3	4	5	6	7
d_i	4	2	4	3	1	4	6
w_i	70	60	50	40	30	20	10

Si mantiene un array s di n elementi, inizializzato con posizioni vuote. A partire da a_1 , si analizza un'attivita' alla volta. L'attivita' a_i viene collocata nella prima posizione libera di s partendo da d_i e scendendo verso 1. Se non c'e' posizione libera, l'attivita' viene scartata.

Attivita'	Scadenza	Decisione	Stato degli slot utili
a_1	4	inserita in 4	$[-, -, -, a_1, -, -, -]$
a_2	2	inserita in 2	$[-, a_2, -, a_1, -, -, -]$
a_3	4	inserita in 3	$[-, a_2, a_3, a_1, -, -, -]$
a_4	3	inserita in 1	$[a_4, a_2, a_3, a_1, -, -, -]$
a_5	1	scartata	$[a_4, a_2, a_3, a_1, -, -, -]$
a_6	4	scartata	$[a_4, a_2, a_3, a_1, -, -, -]$
a_7	6	inserita in 6	$[a_4, a_2, a_3, a_1, -, a_7, -]$

Il risultato riportato nelle slide e'

$$s = [a_4, a_2, a_3, a_1, -, a_7, -].$$

Poi si ordinano le attivita' in anticipo rispetto alla scadenza e si aggiungono le attivita' rimanenti per ottenere la soluzione canonica.

12.5.7 Grafico o diagramma

Slot temporali:

1	2	3	4	5	6	7

a4	a2	a3	a1	-	a7	-

Attivita' in anticipo:

a4, a2, a3, a1, a7

Attivita' in ritardo:

a5, a6

12.5.8 Dimostrazione

Criterio $N_t(A) \leq t$. Se A e' indipendente, allora le attivita' di A possono essere schedulate senza ritardi. Entro il tempo t esistono solo t slot disponibili. Tutte le attivita' con scadenza al piu' t devono essere completate entro tali t slot. Quindi necessariamente

$$N_t(A) \leq t.$$

Viceversa, se

$$N_t(A) \leq t \quad \forall t = 1, \dots, n,$$

ordinando le attivita' di A per scadenza crescente, la k -esima attivita' nell'ordine ha scadenza almeno k . Infatti, se avesse scadenza minore di k , esisterebbero piu' di d attivita' con scadenza

al piu' d , violando $N_d(A) \leq d$. Dunque ogni attivita' termina entro la propria scadenza e A e' indipendente.

Proprieta' di scambio. Le slide dimostrano la proprieta' di scambio. Siano B, A due sottoinsiemi indipendenti con

$$|B| > |A|.$$

Sia k il piu' grande indice tale che

$$N_k(B) \leq N_k(A).$$

Poiche'

$$N_n(B) = |B|, \quad N_n(A) = |A|$$

e $|B| > |A|$, vale $k < n$ e, per ogni j con $k+1 \leq j \leq n$,

$$N_j(B) > N_j(A).$$

Quindi $B \setminus A$ contiene almeno un'attivita' con scadenza $k+1$; sia tale attivita' a_i .

Definiamo

$$A' = A \cup \{a_i\}.$$

Per $k < t \leq n$ vale

$$N_t(A') \leq N_t(B) \leq t,$$

perche' B e' indipendente. Per $t \leq k$, l'aggiunta di a_i , che ha scadenza $k+1$, non modifica $N_t(A)$.

Quindi anche per $t \leq k$ resta

$$N_t(A') = N_t(A) \leq t.$$

Pertanto

$$A' \in \mathcal{I}.$$

La proprieta' di scambio e' dimostrata.

Poiche' le altre proprieta' del matroide sono immediate:

- S e' finito;
- ogni sottoinsieme di un insieme schedulabile senza ritardi e' schedulabile senza ritardi;

segue che $M = (S, \mathcal{I})$ e' un matroide.

12.5.9 Analisi della complessita'

Le slide danno il costo:

$$O(n^2).$$

In dettaglio:

- ordinare le attivita' per penalita' decrescente costa

$$O(n \log n);$$

- ci sono n cicli;
- in ciascun ciclo si decide se aggiungere l'attivita' alla soluzione;
- la verifica di indipendenza costa $O(|A|)$.

Nel caso peggiore:

$$\sum_{j=1}^n O(j) = O(n^2),$$

quindi il costo complessivo e'

$$O(n \log n + n^2) = O(n^2).$$

Nell'implementazione con array di slot descritta nell'esempio, cercare una posizione libera partendo dalla scadenza puo' costare $O(d_i)$ nel caso peggiore, e quindi ancora $O(n)$ per attivita', cioe' $O(n^2)$ totale.

Lo spazio aggiuntivo e' $O(n)$ per l'array degli slot e per rappresentare l'insieme scelto.

12.5.10 Errori tipici d'esame

Minimizzare direttamente le penalita' in ritardo senza osservare l'equivalenza con la massimizzazione delle penalita' delle attivita' in anticipo. Il collegamento con il matroide passa da questa equivalenza.

Credere che la forma canonica cambi quali attivita' sono in ritardo. La forma canonica riordina il piano senza peggiorare la puntualita' delle attivita' in anticipo.

Usare il criterio $N_t(A) \leq t$ solo per un valore di t . La condizione deve valere per ogni $t = 1, \dots, n$.

12.5.11 Possibili domande d'esame

- Definire il problema di scheduling di attivita' unitarie con penalita'.
- Spiegare la forma canonica.
- Dimostrare che A e' indipendente se e solo se $N_t(A) \leq t$ per ogni t .
- Dimostrare la proprieta' di scambio del Teorema 6.
- Applicare l'algoritmo all'esempio delle slide.
- Analizzare il costo $O(n^2)$.

12.5.12 Collegamenti teorici

Questo problema mostra l'uso completo della catena teorica:

problema di ottimizzazione $\longrightarrow$ insiemi indipendenti $\longrightarrow$ matroide $\longrightarrow$ greedy corretto.

12.6 Altri problemi greedy citati nelle slide

12.6.1 Definizione

Le slide elencano altri problemi importanti che ammettono un buon algoritmo greedy di risoluzione:

Problema	Costo greedy indicato
Albero minimo di ricoprimento	Prim: $O(n^2)$
Schedulazione su macchina singola con minimizzazione del numero di attività in ritardo	Moore-Hodgson
Compressione di file	Codice di Huffman
Soddisfacibilità delle clausole di Horn	$O(n)$

12.6.2 Intuizione

La tabella serve a chiarire che la teoria dei matroidi non esaurisce tutti i problemi risolvibili con greedy. Esistono problemi greedy corretti la cui struttura non è rappresentabile come matroide.

12.6.3 Motivazione

Questa osservazione evita un errore concettuale: i matroidi sono una condizione sufficiente potente per la correttezza greedy, ma non sono una caratterizzazione completa di tutti gli algoritmi greedy corretti trattabili.

12.6.4 Proprietà teoriche

Le slide dichiarano esplicitamente che:

- i problemi sui grafi visti precedentemente sono problemi su matroidi grafici pesati;
- esistono problemi la cui struttura non è rappresentabile da un matroide ma che ammettono soluzioni greedy;
- l'esempio più interessante citato è la compressione di file con algoritmo di Huffman.

12.6.5 Algoritmo

Per Moore-Hodgson, Huffman e soddisfacibilità delle clausole di Horn, in questa slide sono riportati soltanto i nomi dei problemi e i costi. Lo pseudocodice dettagliato non è presente nel materiale ufficiale di questa lezione.

Contenuto non presente nel materiale ufficiale del corso.

12.6.6 Esempio guidato

La slide non contiene un'esecuzione guidata di Moore-Hodgson, Huffman o Horn-SAT.

Contenuto non presente nel materiale ufficiale del corso.

12.6.7 Grafico o diagramma

Problemi greedy

```
|  
+-- modellabili con matroidi  
| |  
| +-- matroide grafico  
| +-- scheduling attivita' unitarie con penalita'  
|  
+-- non necessariamente modellabili con matroidi  
|  
| +-- Huffman  
| +-- Moore-Hodgson  
| +-- clausole di Horn
```

12.6.8 Dimostrazione

Le dimostrazioni di correttezza di Moore-Hodgson, Huffman e dell'algoritmo per clausole di Horn non sono presenti nella slide Greedy.

Contenuto non presente nel materiale ufficiale del corso.

12.6.9 Analisi della complessita'

Le complessita' riportate ufficialmente sono:

Algoritmo/problema	Costo
Prim	$O(n^2)$
Moore-Hodgson	$O(n \log n)$
Huffman	$O(n \log n)$
Horn-SAT	$O(n)$

12.6.10 Errori tipici d'esame

Affermare che ogni algoritmo greedy corretto derivi necessariamente da un matroide. Le slide affermano esplicitamente che esistono problemi greedy non rappresentabili da matroidi.

12.6.11 Possibili domande d'esame

- Spiegare perché i matroidi non esauriscono tutti i problemi greedy.
- Riportare gli esempi della tabella finale delle slide.
- Distinguere tra problemi greedy trattati in dettaglio e problemi solo citati.

12.6.12 Collegamenti teorici

Questa sezione collega la teoria generale dei matroidi ai limiti della teoria stessa: una struttura può non essere un matroide e tuttavia ammettere un algoritmo greedy corretto.

12.7 Esercizi ufficiali della lezione

12.7.1 Definizione

Le slide finali propongono quattro esercizi:

1. completare Dijkstra in modo che restituisca anche i cammini minimi;
2. modificare la funzione peso di un matroide per trovare una soluzione di peso minimo tra gli insiemi massimali;
3. dimostrare il criterio A indipendente se e solo se $N_t(A) \leq t$ e verificare l'indipendenza in tempo $O(|A|)$;
4. studiare il problema del resto in monete.

12.7.2 Intuizione

Gli esercizi chiedono di applicare le idee centrali della lezione:

- i predecessori π in Dijkstra codificano i cammini;
- un problema di minimo può essere trasformato in un problema di massimo quando tutti i massimali hanno la stessa cardinalità;
- lo scheduling unitario si controlla contando le scadenze;
- il resto in monete mostra che il greedy può essere corretto per alcuni sistemi di monete e scorretto per altri.

12.7.3 Motivazione

Questi esercizi sono utili per l'esame perché obbligano a distinguere tra:

- scrivere un algoritmo;
- dimostrarne la correttezza;
- riconoscere quando una scelta greedy non basta.

12.7.4 Proprietà teoriche

Dijkstra e cammini. Gli attributi π permettono di ricostruire un cammino minimo risalendo dal nodo destinazione alla sorgente.

Minimizzazione in un matroide. Poiché tutti gli insiemi massimali di un matroide hanno la stessa cardinalità, una trasformazione affine dei pesi che inverte l'ordine dei singoli elementi può trasformare la minimizzazione in massimizzazione sui massimali.

Scheduling unitario. Il criterio

$$N_t(A) \leq t \quad \forall t$$

è necessario e sufficiente per l'indipendenza.

Resto in monete. Il problema consiste nel formare il resto di n centesimi usando il minor numero possibile di monete, assumendo valori interi delle monete.

12.7.5 Algoritmo

Ricostruzione dei cammini in Dijkstra. Dopo l'esecuzione di Dijkstra, per stampare il cammino da s a v si usa il predecessore.

Algorithm 48: PRINTPATH(s, v)

Input: Sorgente s , vertice v , predecessori π calcolati da Dijkstra

Output: Cammino da s a v

```
1 if  $v = s$  then
2   | stampa  $s$ ;
3 else
4   | if  $v.\pi = \text{NULL}$  then
5     | stampa "nessun cammino";
6   | else
7     | PRINTPATH( $s, v.\pi$ );
8     | stampa  $v$ ;
```

Minimizzazione su matroide. Se si vuole trovare un massimale di peso minimo e i pesi sono $w(x)$, si può usare una funzione trasformata

$$w'(x) = C - w(x),$$

dove C è una costante maggiore del massimo peso degli elementi. Poiché tutti i massimali hanno la stessa cardinalità, massimizzare

$$\sum_{x \in A} w'(x)$$

equivale a minimizzare

$$\sum_{x \in A} w(x).$$

Verifica di indipendenza nello scheduling. Per verificare se A è indipendente:

1. ordina le attività di A per scadenza crescente;
2. controlla che la i -esima attività abbia scadenza almeno i .

Questo è equivalente al controllo $N_t(A) \leq t$ per ogni t .

Greedy per il resto in monete. Dato un insieme di monete ordinato per valore decrescente, l'algoritmo greedy prende sempre il massimo numero possibile di monete del valore corrente e poi passa al valore successivo.

12.7.6 Esempio guidato

Resto con monete $\{1, 5, 10, 20\}$. Per formare $n = 38$ centesimi, il greedy sceglie:

$$20, 10, 5, 1, 1, 1.$$

Totale:

$$20 + 10 + 5 + 1 + 1 + 1 = 38,$$

con 6 monete.

Sistema in cui il greedy fallisce. Con monete

$$\{1, 3, 4\}$$

e resto

$$n = 6,$$

il greedy sceglie

$$4 + 1 + 1,$$

cioè 3 monete. La soluzione ottima è

$$3 + 3,$$

cioè 2 monete. Quindi il greedy non è corretto per ogni insieme di monete.

12.7.7 Grafico o diagramma

Esempio di fallimento del greedy:

monete = {1, 3, 4}

resto = 6

greedy:

6 → prende 4, resta 2

2 → prende 1, resta 1

1 → prende 1, resta 0

soluzione greedy: 4 + 1 + 1 (3 monete)

ottimo:

6 = 3 + 3 (2 monete)

12.7.8 Dimostrazione

Monete {1, 5, 10, 20}. In una soluzione ottima:

- non possono comparire cinque monete da 1, perché si sostituirebbero con una moneta da 5;
- non possono comparire due monete da 5, perché si sostituirebbero con una moneta da 10;
- non possono comparire due monete da 10, perché si sostituirebbero con una moneta da 20.

Quindi una soluzione ottima senza monete da 20 può valere al massimo

$$10 + 5 + 4 \cdot 1 = 19.$$

Se il resto è almeno 20, ogni soluzione ottima deve contenere una moneta da 20; dunque la scelta greedy della moneta da 20 è sicura. Ripetendo l'argomento sul resto residuo si ottiene la correttezza del greedy.

Monete $1, c, c^2, \dots, c^k$ con $c > 1$. In una soluzione ottima non possono comparire c monete di valore c^i , perché esse possono essere sostituite da una moneta di valore

$$c \cdot c^i = c^{i+1},$$

riducendo il numero di monete da c a 1.

Quindi, per ogni denominazione inferiore a c^j , una soluzione ottima contiene al più $c - 1$ monete di ciascun valore. Il massimo valore ottenibile usando solo monete inferiori a c^j è

$$(c - 1)(1 + c + c^2 + \dots + c^{j-1}) = (c - 1) \frac{c^j - 1}{c - 1} = c^j - 1.$$

Pertanto, se il valore residuo è almeno c^j , una soluzione ottima deve usare una moneta da c^j . La scelta greedy è quindi sicura a ogni passo.

Fallimento generale. L'esempio $\{1, 3, 4\}$ con $n = 6$ dimostra che non basta la presenza della moneta da 1 per garantire la correttezza del greedy. La moneta da 1 garantisce l'esistenza di una soluzione, non l'ottimalità della soluzione greedy.

12.7.9 Analisi della complessità

PrintPath. La procedura visita al più un predecessore per ogni vertice sul cammino. Nel caso peggiore il costo è $O(n)$ e lo spazio di stack ricorsivo è $O(n)$.

Verifica di indipendenza. Se le attività sono già mantenute ordinate per scadenza, la verifica richiede $O(|A|)$. In caso contrario, l'ordinamento aggiungerebbe un costo non indicato nell'esercizio.

Resto in monete. Se il numero di tipi di monete è $k + 1$, l'algoritmo greedy scorre le denominazioni una volta e costa $O(k)$, assumendo operazioni aritmetiche elementari a costo costante nel modello RAM.

12.7.10 Errori tipici d'esame

Nel resto in monete, confondere esistenza della soluzione con ottimalità. La moneta da 1 assicura che un resto si possa sempre formare, ma non assicura che il greedy usi il numero minimo di monete.

Nella minimizzazione su matroide, dimenticare che la trasformazione dei pesi funziona sugli insiemi massimali perché hanno tutti la stessa cardinalità.

In Dijkstra, restituire soltanto le distanze d e non i predecessori π quando viene richiesto anche il cammino.

12.7.11 Possibili domande d'esame

- Scrivere la procedura per ricostruire un cammino minimo usando π .

- Spiegare come trasformare un problema di minimo su massimali di un matroide in un problema di massimo.
- Dimostrare il criterio $N_t(A) \leq t$ per lo scheduling unitario.
- Dimostrare la correttezza del greedy per le monete $\{1, 5, 10, 20\}$.
- Dimostrare la correttezza del greedy per monete $1, c, c^2, \dots, c^k$.
- Fornire un insieme di monete per cui il greedy fallisce.

12.7.12 Collegamenti teorici

Gli esercizi collegano tutti i temi della lezione:

- Dijkstra mostra un greedy con coda di priorit ;
- i matroidi danno una struttura generale di correttezza;
- lo scheduling unitario e' un'applicazione dei matroidi;
- il resto in monete mostra che un algoritmo greedy richiede sempre una dimostrazione specifica.

12.8 Contenuti richiesti ma non presenti nella slide Greedy

12.8.1 Definizione

Alcuni argomenti spesso associati agli algoritmi greedy non sono sviluppati nella slide ufficiale Greedy usata per questo capitolo.

12.8.2 Intuizione

Questa sezione serve a evitare inserimenti non autorizzati rispetto alla fonte ufficiale.

12.8.3 Motivazione

Il vincolo della dispensa e' che le slide prevalgano su ogni altra fonte. Quindi un argomento non presente nelle slide non deve essere ricostruito da materiale esterno.

12.8.4 Proprieta' teoriche

- Algoritmo di Kruskal: contenuto non presente nel materiale ufficiale della slide Greedy.
- Dimostrazione dettagliata di Moore-Hodgson: contenuto non presente nel materiale ufficiale della slide Greedy.
- Pseudocodice dettagliato di Huffman: contenuto non presente nel materiale ufficiale della slide Greedy.

12.8.5 Algoritmo

Contenuto non presente nel materiale ufficiale del corso.

12.8.6 Esempio guidato

Contenuto non presente nel materiale ufficiale del corso.

12.8.7 Grafico o diagramma

Nel capitolo Greedy ufficiale:

MST -> Prim

SSSP -> Dijkstra

Matroidi -> greedy su matroide pesato

Scheduling unitario -> applicazione dei matroidi

Non sviluppati nella slide Greedy:

Kruskal

pseudocodice Huffman

dimostrazione Moore-Hodgson

12.8.8 Dimostrazione

Contenuto non presente nel materiale ufficiale del corso.

12.8.9 Analisi della complessita'

Per gli argomenti non sviluppati nella slide Greedy non viene fornita un'analisi dettagliata, salvo i costi riportati nella tabella ufficiale per Moore-Hodgson, Huffman e Horn-SAT.

12.8.10 Errori tipici d'esame

Attribuire alle slide un algoritmo non presente. Se una domanda chiede specificamente il materiale della lezione Greedy, bisogna attenersi a Prim, Dijkstra, matroidi e scheduling unitario, oltre agli esercizi finali.

12.8.11 Possibili domande d'esame

- Quali algoritmi greedy sono sviluppati esplicitamente nella slide Greedy?
- Quali problemi sono solo citati nella tabella finale?
- Perche' non si puo' sostituire Prim con Kruskal se Kruskal non e' stato introdotto nel materiale ufficiale?

12.8.12 Collegamenti teorici

Gli argomenti non presenti in questa slide possono appartenere ad altri corsi o ad altre trattazioni, ma non vengono usati come fonte teorica per questo capitolo.

13. Grafi

13.1 Rappresentazione dei grafi

13.1.1 Definizione

In questa parte del corso si assume che i concetti matematici di base sui grafi siano già noti. Si richiamano però le due rappresentazioni fondamentali usate dagli algoritmi:

- **matrice di adiacenza;**
- **liste di adiacenza.**

Sia

$$G = (V, E)$$

un grafo, orientato o non orientato. Indichiamo con

$$n = |V|, \quad m = |E|$$

il numero di vertici e di archi.

Matrice di adiacenza. Dato un grafo di n nodi, la matrice di adiacenza è una matrice $n \times n$ in cui righe e colonne sono etichettate con i nomi dei nodi. La cella di coordinate (i, j) indica se esiste l'arco dal nodo i al nodo j :

$$A[i, j] = \begin{cases} 1 & \text{se } (i, j) \in E, \\ 0 & \text{altrimenti.} \end{cases}$$

Nel caso di grafo non orientato, la matrice è simmetrica.

Liste di adiacenza. La rappresentazione con liste di adiacenza mantiene una lista principale dei nodi e, per ciascun nodo v , una lista

$$Adj[v]$$

contenente tutti i vertici u tali che

$$(v, u) \in E.$$

Se il numero di nodi e' fisso, la lista principale puo' essere implementata come un array di riferimenti a liste.

13.1.2 Intuizione

La matrice di adiacenza risponde molto velocemente alla domanda:

“l'arco (u, v) esiste?”

perche' basta leggere una cella. Tuttavia usa sempre spazio quadratico, anche se il grafo ha pochi archi.

Le liste di adiacenza, invece, memorizzano soltanto gli archi presenti. Sono quindi piu' adatte per grafi sparsi, cioe' grafi in cui m e' molto piu' piccolo di n^2 .

13.1.3 Motivazione

La scelta della rappresentazione incide direttamente sulla complessita' degli algoritmi. BFS, DFS, ordinamento topologico e componenti fortemente connesse vengono analizzati nelle slide con liste di adiacenza, perche' in tale rappresentazione l'esplorazione di tutti gli archi costa

$$\Theta(|V| + |E|).$$

13.1.4 Proprieta' teoriche

Operazione	Matrice di adiacenza	Liste di adiacenza
Memoria	$\Theta(n^2)$	$\Theta(n + m)$
Verificare se $(u, v) \in E$	$\Theta(1)$	$\Theta(d(u))$
Accedere agli adiacenti di u	$\Theta(n)$	$\Theta(d(u))$
Identificare tutti gli archi	$\Theta(n^2)$	$\Theta(n + m)$

Nel caso di grafi non orientati, ogni arco $\{u, v\}$ compare due volte nelle liste di adiacenza: una volta in $Adj[u]$ e una volta in $Adj[v]$.

13.1.5 Algoritmo

Per costruire una matrice di adiacenza da una lista di archi:

Algorithm 49: COSTRUISCIMATRICEADIACENZA

Input: Insieme di vertici $V = \{1, \dots, n\}$, insieme

Output: Matrice di adiacenza A

```

1 for  $i \leftarrow 1$  to  $n$  do
2   for  $j \leftarrow 1$  to  $n$  do
3      $A[i, j] \leftarrow 0$ ;
4 foreach  $(u, v) \in E$  do
5    $A[u, v] \leftarrow 1$ ;
6 return  $A$ ;
```

Per un grafo non orientato, quando si incontra l'arco $\{u, v\}$ si impostano entrambe le celle:

$$A[u, v] \leftarrow 1, \quad A[v, u] \leftarrow 1.$$

13.1.6 Esempio guidato

Consideriamo il grafo non orientato

$$V = \{1, 2, 4, 5\}, \quad E = \{\{1, 2\}, \{1, 4\}, \{2, 4\}, \{4, 5\}\}.$$

Con ordine dei vertici 1, 2, 4, 5, la matrice di adiacenza è:

	1	2	4	5
1	0	1	1	0
2	1	0	1	0
4	1	1	0	1
5	0	0	1	0

La stessa informazione con liste di adiacenza è:

1	2, 4
2	1, 4
4	1, 2, 5
5	4

13.1.7 Grafico o diagramma

Grafo non orientato:

```

1 ----- 2
 \       /
```

\ /
4 ----- 5

Liste di adiacenza:

Adj[1] = <2,4>

Adj[2] = <1,4>

Adj[4] = <1,2,5>

Adj[5] = <4>

13.1.8 Dimostrazione

Le slide non presentano una dimostrazione formale delle rappresentazioni; forniscono definizioni e costi. I costi seguono direttamente dalla struttura dati:

- una matrice $n \times n$ contiene sempre n^2 celle;
- una lista di adiacenza contiene un riferimento per ogni nodo e un elemento di lista per ogni arco orientato;
- in un grafo non orientato ogni arco compare in due liste.

13.1.9 Analisi della complessita'

La matrice richiede memoria $\Theta(n^2)$, indipendentemente dal numero di archi.

Le liste richiedono memoria $\Theta(n + m)$, perche' servono n riferimenti principali e spazio proporzionale agli archi memorizzati.

Se si accetta di usare piu' memoria del necessario, e' possibile implementare le liste di adiacenza in modo da ridurre quasi a tempo costante la verifica di esistenza di un nodo o di un arco. Le slide pongono questa osservazione come domanda; il contenuto implementativo dettagliato non viene sviluppato nel materiale ufficiale.

13.1.10 Errori tipici d'esame

Confondere il costo di verificare un arco con il costo di scandire tutti gli adiacenti. Nella matrice il primo e' $\Theta(1)$, il secondo e' $\Theta(n)$.

Dimenticare che, nei grafi non orientati rappresentati con liste di adiacenza, ogni arco compare due volte.

13.1.11 Possibili domande d'esame

- Definire matrice di adiacenza e liste di adiacenza.

- Confrontare i costi delle due rappresentazioni.
- Spiegare perché la matrice di un grafo non orientato è simmetrica.
- Dire quale rappresentazione conviene per grafi sparsi.

13.1.12 Collegamenti teorici

BFS, DFS, ordinamento topologico e componenti fortemente connesse sono analizzati sulle liste di adiacenza. Per questo la complessità tipica sarà

$$\Theta(|V| + |E|).$$

13.2 Visita in ampiezza

13.2.1 Definizione

La **visita in ampiezza**, o **BFS** (*breadth-first search*), è un algoritmo di ricerca su grafi.

Dato un grafo

$$G = (V, E)$$

e un nodo sorgente

$$s \in V,$$

BFS ispeziona gli archi di G per scoprire tutti i nodi raggiungibili da s .

La visita procede per livelli: per

$$k = 1, 2, \dots, n - 1,$$

scopre i nodi a distanza k a partire dai nodi a distanza $k - 1$.

L'algoritmo:

- calcola la distanza minima da s a ciascun nodo raggiungibile;
- genera un albero, detto **albero BF** o **breadth-first tree**, con radice s e contenente tutti i nodi raggiungibili da s .

13.2.2 Intuizione

BFS usa una coda FIFO. La coda contiene i nodi scoperti ma non ancora completamente esaminati. Questo garantisce che i nodi scoperti prima vengano espansi prima.

Di conseguenza, prima vengono visitati tutti i nodi a distanza 0, poi quelli a distanza 1, poi quelli a distanza 2, e così via. Questa è la ragione per cui BFS calcola cammini minimi nei grafi non pesati.

13.2.3 Motivazione

BFS è uno schema di base per molti algoritmi importanti sui grafi. Le slide citano esplicitamente:

- algoritmo di Prim per l'albero di connessione minimo;
- algoritmo di Dijkstra per i cammini minimi da sorgente unica.

In questa lezione BFS viene usata soprattutto per mostrare come si esplora un grafo per livelli e come si dimostra formalmente la correttezza delle distanze calcolate.

13.2.4 Proprieta' teoriche

Ogni nodo ha tre attributi:

$$u.color, \quad u.d, \quad u.\pi.$$

- $u.color$ indica lo stato del nodo.
- $u.d$ e' la distanza stimata da s .
- $u.\pi$ e' il predecessore di u nel cammino da s a u .

I colori hanno il seguente significato:

Colore	Significato
WHITE	nodo non ancora scoperto
GRAY	nodo scoperto ma non ancora completato
BLACK	nodo completato: tutti i suoi vicini sono stati scoperti

L'invariante di ciclo indicato nelle slide per il ciclo `while` e':

quando viene eseguito il test del ciclo `while`, la coda Q contiene solo nodi grigi.

13.2.5 Algoritmo

Algorithm 50: BFS(G, s)

Input: $G = (V, E)$, sorgente $s \in V$

```

1 foreach  $u \in V \setminus \{s\}$  do
2    $u.color \leftarrow \text{WHITE};$ 
3    $u.d \leftarrow \infty;$  //  $d$  e' la distanza da  $s$ 
4    $u.\pi \leftarrow \text{NULL};$  //  $\pi$  e' il predecessore
5  $s.color \leftarrow \text{GRAY};$ 
6  $s.d \leftarrow 0;$ 
7  $s.\pi \leftarrow \text{NULL};$ 
8  $Q \leftarrow \emptyset;$  // coda FIFO dei nodi grigi
9  $Q.\text{ENQUEUE}(s);$ 
10 while  $Q \neq \emptyset$  do
11    $u \leftarrow Q.\text{DEQUEUE}();$ 
12   foreach  $v \in G.\text{Adj}[u]$  do
13     if  $v.color = \text{WHITE}$  then
14        $v.color \leftarrow \text{GRAY};$ 
15        $v.d \leftarrow u.d + 1;$ 
16        $v.\pi \leftarrow u;$ 
17        $Q.\text{ENQUEUE}(v);$ 
18    $u.color \leftarrow \text{BLACK};$ 

```

13.2.6 Esempio guidato

Usiamo l'esempio coerente con la visita mostrata nelle slide, con sorgente 2. Le liste di adiacenza rilevanti sono:

1	2, 4
2	1, 4
3	6
4	2, 1, 5
5	4
6	3

La componente raggiungibile da 2 contiene i nodi 2, 1, 4, 5. I nodi 3 e 6 non sono raggiungibili da 2.

Passo	Operazione	Coda Q	Distanze finite	Predecessori nuovi
0	inizializza $s = 2$	$\langle 2 \rangle$	$2.d = 0$	–
1	esamina 2, scopre 1	$\langle 1 \rangle$	$1.d = 1$	$1.\pi = 2$
2	esamina 2, scopre 4	$\langle 1, 4 \rangle$	$4.d = 1$	$4.\pi = 2$
3	completa 2	$\langle 1, 4 \rangle$	–	–
4	esamina 1	$\langle 4 \rangle$	–	–
5	esamina 4, scopre 5	$\langle 5 \rangle$	$5.d = 2$	$5.\pi = 4$
6	completa 5	$\langle \rangle$	–	–

Al termine:

$$2.d = 0, \quad 1.d = 1, \quad 4.d = 1, \quad 5.d = 2,$$

mentre

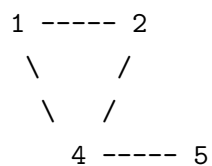
$$3.d = 6.d = \infty.$$

L'albero BF e':

$$2 \rightarrow 1, \quad 2 \rightarrow 4, \quad 4 \rightarrow 5.$$

13.2.7 Grafico o diagramma

Grafo dell'esempio:



3 ----- 6

BFS($G, 2$):

inizio:

$Q = \langle 2 \rangle$

colori: 2 grigio, altri bianchi

d: 2=0

dopo scansione di 2:

$Q = \langle 1, 4 \rangle$

colori: 1,4 grigi; 2 nero

d: 1=1, 4=1

pi: 1<-2, 4<-2

dopo scansione di 1:

Q = <4>

nessun nuovo nodo

dopo scansione di 4:

Q = <5>

d: 5=2

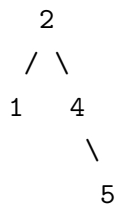
pi: 5<-4

dopo scansione di 5:

Q = <>

fine

Albero BF:



13.2.8 Dimostrazione

La correttezza di BFS rispetto ai cammini minimi nei grafi non pesati viene dimostrata nelle slide tramite tre lemmi, un corollario e un teorema.

Definizione 13.1 (Grafo pesato). Un grafo $G = (V, E)$ si dice pesato quando esiste una funzione peso

$$\omega : E \rightarrow \mathbb{R}$$

che associa a ogni arco un valore detto peso o valore dell'arco. Un grafo non pesato puo' essere visto come un grafo pesato in cui

$$\omega(e) = 1 \quad \forall e \in E.$$

Definizione 13.2 (Distanza di cammino minimo). Si definisce distanza di cammino minimo da s a v il valore

$$\delta(s, v) = \min_{P: s \rightsquigarrow v} \sum_{e \in P} \omega(e).$$

Se non esiste un cammino da s a v , allora

$$\delta(s, v) = \infty.$$

Un cammino da s a v di lunghezza $\delta(s, v)$ e' detto cammino minimo.

Lemma 13.1. Se $G = (V, E)$ e' un grafo non pesato e $s \in V$, allora per qualsiasi arco $(u, v) \in E$ vale

$$\delta(s, v) \leq \delta(s, u) + 1.$$

Dimostrazione. Se u e' raggiungibile da s , sia

$$s \xrightarrow{P'} u$$

un cammino minimo da s a u . Allora concatenando P' con l'arco (u, v) si ottiene un cammino da s a v di lunghezza

$$\delta(s, u) + 1.$$

Il cammino minimo da s a v non puo' essere piu' lungo di un cammino specifico da s a v , quindi

$$\delta(s, v) \leq \delta(s, u) + 1.$$

Se u non e' raggiungibile da s , allora $\delta(s, u) = \infty$ e la disuguaglianza e' ancora vera. $\square$

Lemma 13.2. Sia $G = (V, E)$ un grafo non pesato e sia $s \in V$. Se si esegue $BFS(G, s)$, al termine vale

$$v.d \geq \delta(s, v) \quad \forall v \in V.$$

Dimostrazione. La dimostrazione e' per induzione sul numero di operazioni ENQUEUE.

Caso base. All'inizio solo s e' inserito in Q :

$$s.d = 0 = \delta(s, s).$$

Per ogni altro nodo v ,

$$v.d = \infty \geq \delta(s, v).$$

Passo induttivo. Supponiamo che la proprieta' valga prima di una nuova operazione ENQUEUE.

Un nodo bianco v viene scoperto durante la visita di un nodo u . Per ipotesi induttiva:

$$u.d \geq \delta(s, u).$$

L'algoritmo assegna:

$$v.d \leftarrow u.d + 1.$$

Quindi:

$$v.d = u.d + 1 \geq \delta(s, u) + 1 \geq \delta(s, v),$$

dove l'ultima disuguaglianza segue dal lemma precedente. Dopo questa assegnazione, $v.d$ non viene piu' modificato, perche' v non sara' piu' bianco. $\square$

Lemma 13.3. *Si assuma che durante BFS la coda contenga*

$$Q = \langle v_1, v_2, \dots, v_r \rangle.$$

Allora:

$$v_r.d \leq v_1.d + 1$$

e

$$v_i.d \leq v_{i+1}.d \quad \text{per } i = 1, 2, \dots, r-1.$$

Dimostrazione. La dimostrazione e' per induzione sulle operazioni possibili sulla coda Q .

Base. Quando

$$Q = \langle s \rangle,$$

la tesi e' immediata.

Passo induttivo: estrazione. Se si estrae v_1 , la coda diventa

$$\langle v_2, \dots, v_r \rangle.$$

Per ipotesi induttiva valevano

$$v_r.d \leq v_1.d + 1$$

e

$$v_1.d \leq v_2.d.$$

Da queste due relazioni segue

$$v_r.d \leq v_2.d + 1.$$

L'ordine non decrescente delle distanze nella coda rimane quindi valido.

Passo induttivo: inserimento. Se durante l'esame di $u = v_1$ si inserisce un nuovo nodo v in coda, allora

$$v.d = u.d + 1 = v_1.d + 1.$$

Dopo l'estrazione di u , il nuovo primo elemento della coda ha distanza almeno $v_1.d$. Pertanto il nuovo ultimo elemento soddisfa ancora la proprieta' di stare a distanza al piu' uno dal primo elemento della coda. $\square$

Corollario 13.4. *Se v_i e v_j sono inseriti in coda durante l'esecuzione di BFS, e v_i viene inserito prima di v_j , allora*

$$v_i.d \leq v_j.d.$$

Teorema 13.5 (Correttezza di BFS). *BFS scopre tutti i nodi $v \in V$ raggiungibili da s e, alla fine,*

$$v.d = \delta(s, v) \quad \forall v \in V.$$

Inoltre, per qualsiasi nodo v raggiungibile da s , un cammino minimo da s a v si ottiene prendendo un cammino minimo da s a $v.\pi$ e aggiungendo l'arco $(v.\pi, v)$.

Dimostrazione. La dimostrazione procede per assurdo, come nelle slide.

Supponiamo che esista almeno un vertice raggiungibile per cui la distanza calcolata non e' minima. Tra tutti questi vertici, scegliamo v con distanza minima $\delta(s, v)$.

Dal lemma precedente sappiamo che

$$v.d \geq \delta(s, v).$$

Poiche' per ipotesi $v.d \neq \delta(s, v)$, deve valere

$$v.d > \delta(s, v).$$

Il nodo v e' diverso da s , perche'

$$s.d = 0 = \delta(s, s).$$

Sia u il vertice che precede immediatamente v in un cammino minimo

$$s \overset{P}{\rightsquigarrow} v.$$

Allora:

$$\delta(s, v) = \delta(s, u) + 1.$$

Poiche' u precede v nel cammino minimo, vale

$$\delta(s, u) < \delta(s, v).$$

Per la scelta di v come vertice errato di minima distanza, u non puo' essere errato. Quindi:

$$u.d = \delta(s, u).$$

Ne segue:

$$v.d > \delta(s, v) = \delta(s, u) + 1 = u.d + 1.$$

Consideriamo il momento in cui BFS esamina v come adiacente di u .

Caso 1: v e' bianco. L'algoritmo assegna

$$v.d \leftarrow u.d + 1,$$

in contraddizione con

$$v.d > u.d + 1.$$

Caso 2: v e' nero. Allora v e' gia' stato rimosso dalla coda prima di u . Per il corollario

sull'ordine di inserimento e sulle distanze:

$$v.d \leq u.d.$$

Quindi:

$$v.d \leq u.d < u.d + 1 < v.d,$$

contraddizione.

Caso 3: v e' grigio. Allora v e' stato colorato grigio durante la visita di un nodo w , rimosso dalla coda prima di u , e

$$v.d = w.d + 1.$$

Per il corollario:

$$w.d \leq u.d.$$

Quindi:

$$v.d = w.d + 1 \leq u.d + 1,$$

in contraddizione con

$$v.d > u.d + 1.$$

Tutti i casi portano a contraddizione. Dunque per ogni nodo raggiungibile v vale

$$v.d = \delta(s, v).$$

Quando un nodo v viene scoperto, l'algoritmo assegna $v.\pi = u$ e $v.d = u.d + 1$. Per quanto dimostrato, $u.d = \delta(s, u)$ e $v.d = \delta(s, v)$, quindi il cammino minimo verso v si ottiene ricorsivamente dal cammino minimo verso $v.\pi$ seguito dall'arco $(v.\pi, v)$. $\square$

13.2.9 Analisi della complessita'

Il primo ciclo **for** e le inizializzazioni della sorgente preparano i nodi:

$$O(|V|).$$

Le slide precisano che si scrive O e non necessariamente Θ , perche' l'inizializzazione puo' essere effettuata in modo pigro in $O(1)$.

Il ciclo **while** continua finche' ci sono nodi grigi. Ogni nodo diventa grigio una sola volta, quindi viene inserito in coda una sola volta e rimosso una sola volta. Le operazioni **ENQUEUE** e **DEQUEUE** costano $O(1)$.

Per ciascun nodo u , il ciclo sugli adiacenti esamina tutta la lista $Adj[u]$. Sommando su tutti i nodi:

$$\sum_{u \in V} |Adj[u]| = \Theta(|E|)$$

nei grafi orientati; nei grafi non orientati ogni arco e' contato due volte, ma il costo resta $\Theta(|E|)$.
La complessita' finale e':

$$O(|V| + |E|).$$

Lo spazio aggiuntivo e' $O(|V|)$ per colori, distanze, predecessori e coda.

13.2.10 Errori tipici d'esame

Dire che BFS calcola cammini minimi in qualunque grafo pesato. Nelle slide la correttezza dei cammini minimi e' dimostrata per grafi non pesati, equivalenti a grafi con peso unitario su ogni arco.

Confondere nodo grigio e nodo nero. Un nodo grigio e' stato scoperto ma non completato; un nodo nero ha gia' avuto tutti i vicini esaminati.

Dimenticare che la coda e' FIFO. Senza FIFO, l'esplorazione per livelli e la dimostrazione sulle distanze non valgono.

13.2.11 Possibili domande d'esame

- Scrivere lo pseudocodice di BFS.
- Spiegare il ruolo dei colori.
- Dimostrare l'invariante sulla coda dei nodi grigi.
- Dimostrare che BFS calcola le distanze minime nei grafi non pesati.
- Simulare BFS mostrando coda, colori, distanze e predecessori.
- Analizzare la complessita' $O(|V| + |E|)$.

13.2.12 Collegamenti teorici

BFS e' la base per ragionare su:

- cammini minimi in grafi non pesati;
- costruzione di alberi di visita;
- algoritmi che usano code o code di priorita' per espandere nodi in un ordine controllato.

13.3 Visita in profondita'

13.3.1 Definizione

La **visita in profondita'**, o **DFS** (*depth-first search*), visita un grafo andando il piu' possibile in profondita'.

Dato un grafo

$$G = (V, E),$$

DFS:

- ispeziona gli archi a partire dall'ultimo nodo scoperto che ha ancora archi non ispezionati;
- quando tutti gli archi di un nodo v sono stati ispezionati, ritorna al nodo da cui v e' stato scoperto;
- continua finche' esistono nodi da ispezionare.

La visita puo' generare piu' alberi: il risultato e' una **foresta DF**, cioe' una *depth-first forest*.

13.3.2 Intuizione

DFS si comporta come una pila ricorsiva. Appena scopre un nuovo nodo, sospende temporaneamente l'esame del nodo corrente e scende sul nuovo nodo. Solo quando non puo' piu' scendere, torna indietro.

Questa differenza e' centrale:

BFS	DFS
esplora per livelli	esplora andando in profondita'
usa una coda FIFO	usa ricorsione, cioe' una pila implicita
calcola distanze minime non pesate	calcola tempi di scoperta e completamento

13.3.3 Motivazione

DFS e' usata nelle slide come base per:

- classificazione degli archi;
- proprieta' a parentesi dei tempi;
- ordinamento topologico;
- componenti fortemente connesse.

13.3.4 Proprieta' teoriche

Per ogni nodo v , DFS mantiene:

$$v.color, \quad v.\pi, \quad v.d, \quad v.f.$$

- $v.d$ e' l'istante in cui v viene scoperto, cioe' diventa grigio.
- $v.f$ e' l'istante in cui v viene completato, cioe' diventa nero.

- $v.\pi$ e' il predecessore di v nella foresta DF.

L'algoritmo usa una variabile globale

time

che viene incrementata ogni volta che un nodo cambia colore per scoperta o completamento.

Classificazione degli archi. Le slide riportano la seguente classificazione degli archi ispezionati da DFS:

Tipo	Condizione
T (tree)	l'arco (u, v) e' ispezionato e v e' bianco
B (backward)	l'arco (u, v) e' ispezionato e v e' grigio
F (forward)	l'arco (u, v) e' ispezionato, v e' nero e $u.d < v.d$
C (cross)	l'arco (u, v) e' ispezionato, v e' nero e $u.d > v.f$

Per un arco backward, il nodo v e' un antenato di u . Per un arco forward, v e' un discendente di u . Per un arco cross, u e v non sono in relazione di discendenza, oppure appartengono a due alberi distinti della foresta DF.

13.3.5 Algoritmo

Algorithm 51: DFS(G)

```

Input:  $G = (V, E)$ 
1 foreach  $u \in V$  do
2    $u.color \leftarrow \text{WHITE};$ 
3    $u.\pi \leftarrow \text{NULL};$ 
4  $time \leftarrow 0;$                                      // variabile globale
5 foreach  $u \in V$  do
6   if  $u.color = \text{WHITE}$  then
7     DFS-VISIT( $G, u$ );

```

Algorithm 52: DFS-VISIT(G, u)

Input: $G = (V, E)$, nodo $u \in V$

```

1   $time \leftarrow time + 1$ ;
2   $u.d \leftarrow time$ ;
3   $u.color \leftarrow \text{GRAY}$ ;
4  foreach  $v \in G.Adj[u]$  do
5      if  $v.color = \text{WHITE}$  then
6           $v.\pi \leftarrow u$ ;
7          DFS-VISIT( $G, v$ );
8   $time \leftarrow time + 1$ ;
9   $u.f \leftarrow time$ ;
10  $u.color \leftarrow \text{BLACK}$ ;
```

13.3.6 Esempio guidato

L'esecuzione mostrata nelle slide produce la foresta:

$$2 \rightarrow 1 \rightarrow 4 \rightarrow 5, \quad 3 \rightarrow 6.$$

I tempi sono:

Nodo	d	f
2	1	8
1	2	7
4	3	6
5	4	5
3	9	12
6	10	11

Evoluzione della pila ricorsiva:

Evento	<i>time</i>	Stack chiamate DFS-Visit
scopri 2	1	$\langle 2 \rangle$
scopri 1	2	$\langle 1, 2 \rangle$
scopri 4	3	$\langle 4, 1, 2 \rangle$
scopri 5	4	$\langle 5, 4, 1, 2 \rangle$
completa 5	5	$\langle 5, 4, 1, 2 \rangle$
completa 4	6	$\langle 4, 1, 2 \rangle$
completa 1	7	$\langle 1, 2 \rangle$
completa 2	8	$\langle 2 \rangle$
scopri 3	9	$\langle 3 \rangle$
scopri 6	10	$\langle 6, 3 \rangle$
completa 6	11	$\langle 6, 3 \rangle$
completa 3	12	$\langle 3 \rangle$

13.3.7 Grafico o diagramma

Foresta DF dell'esempio:

```

2
|
1
|
4
|
5

3
|
6

```

Tempi:

```

2: d=1, f=8
1: d=2, f=7
4: d=3, f=6
5: d=4, f=5
3: d=9, f=12
6: d=10, f=11

```

Espressione a parentesi:

(2 (1 (4 (5 5) 4) 1) 2)(3 (6 6) 3)

13.3.8 Dimostrazione

Le slide non richiedono le dimostrazioni dei teoremi 20.7, 20.9 e 20.10. Si esplicita pero' la proprieta' a parentesi indicata nel materiale.

Struttura a parentesi. La scoperta di un nodo u si rappresenta con una parentesi aperta (u , mentre il completamento di u si rappresenta con una parentesi chiusa u).

Ogni chiamata ricorsiva DFS-VISIT(G, u):

- apre la parentesi di u quando assegna $u.d$;
- completa interamente tutte le chiamate ricorsive dei discendenti;
- chiude la parentesi di u quando assegna $u.f$.

Quindi non e' possibile chiudere un nodo prima di aver chiuso tutti i suoi discendenti. L'espressione prodotta e' ben formata.

Nell'esempio delle slide:

(2 (1 (4 (5 5) 4) 1) 2)(3 (6 6) 3)

mostra esattamente che l'intervallo temporale di un discendente e' contenuto nell'intervallo temporale del suo antenato.

13.3.9 Analisi della complessita'

L'algoritmo DFS richiede tempo $\Theta(|V|)$ escluso il tempo delle chiamate a DFS-VISIT.

DFS-VISIT viene eseguito esattamente una volta per ciascun nodo, quindi ci sono $|V|$ chiamate.

In ogni chiamata DFS-VISIT(G, u), il ciclo **foreach** viene eseguito

$$|Adj[u]|$$

volte. Sommando su tutti i nodi:

$$\sum_{u \in V} |Adj[u]| = \Theta(|E|).$$

Il costo totale e':

$$\Theta(|V| + |E|).$$

Lo spazio aggiuntivo e' $O(|V|)$ per colori, predecessori, tempi e stack ricorsivo.

13.3.10 Errori tipici d'esame

Confondere $v.d$ con una distanza. In DFS, $v.d$ e' un tempo di scoperta, non una distanza dalla sorgente.

Pensare che DFS produca sempre un solo albero. Se il grafo non e' tutto raggiungibile dal primo nodo esaminato, DFS produce una foresta.

Dimenticare che l'ordine di ispezione dei vertici puo' cambiare la foresta DF e i tempi, anche se le applicazioni dell'algoritmo restano corrette.

13.3.11 Possibili domande d'esame

- Scrivere DFS e DFS-VISIT.
- Simulare DFS mostrando stack ricorsivo, colori, tempi d e f .
- Spiegare la differenza tra BFS e DFS.
- Definire archi tree, backward, forward e cross.
- Spiegare la struttura a parentesi.
- Analizzare la complessita' $\Theta(|V| + |E|)$.

13.3.12 Collegamenti teorici

DFS e' la base per:

- ordinamento topologico dei DAG;
- componenti fortemente connesse;
- ragionamenti sui tempi di scoperta e completamento.

13.4 Ordinamento topologico

13.4.1 Definizione

Un **DAG** e' un grafo diretto aciclico, cioe' un *directed acyclic graph*.

Un **ordinamento topologico** di un DAG

$$G = (V, E)$$

e' un ordinamento lineare dei nodi tale che, per ogni arco

$$(u, v) \in E,$$

il nodo u compare prima del nodo v nell'ordinamento.

13.4.2 Intuizione

Un ordinamento topologico dispone i nodi su una linea in modo che tutti gli archi vadano da sinistra verso destra.

Se un arco (u, v) rappresenta il vincolo “ u deve avvenire prima di v ”, allora un ordinamento topologico e' una lista che rispetta tutte le precedenze.

13.4.3 Motivazione

Le slide indicano che gli ordinamenti topologici sono usati in molti contesti per rappresentare precedenze fra eventi. L'algoritmo proposto usa i tempi di completamento calcolati da DFS.

13.4.4 Proprieta' teoriche

La proprieta' fondamentale riportata nelle slide e':

$$\text{in un DAG, per ogni arco } (u, v) \in E, \quad u.f > v.f.$$

Quindi, se i nodi sono inseriti in testa a una lista al momento del completamento, il nodo con tempo di completamento maggiore finisce prima nella lista.

Poiche' un DAG non contiene cicli, durante DFS non possono comparire archi all'indietro che chiuderebbero un ciclo.

13.4.5 Algoritmo

Algorithm 53: TOPOLOGICAL-SORT(G)

Input: DAG $G = (V, E)$ **Output:** Un ordinamento topologico dei nodi

- 1 $lista \leftarrow \langle \rangle$;
 - 2 Esegui DFS(G) e, ogni volta che un valore $u.f$ viene calcolato, inserisci u in testa a $lista$;
 - 3 **return** $lista$;
-

13.4.6 Esempio guidato

Consideriamo il DAG:

$$a \rightarrow b, \quad a \rightarrow c, \quad b \rightarrow d, \quad c \rightarrow d.$$

Un'esecuzione DFS puo' completare i nodi nell'ordine:

$$d, b, c, a.$$

Inserendo ogni nodo in testa alla lista al momento del completamento:

Nodo completato	Lista
d	$\langle d \rangle$
b	$\langle b, d \rangle$
c	$\langle c, b, d \rangle$
a	$\langle a, c, b, d \rangle$

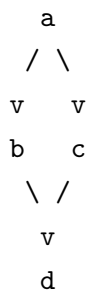
La lista

$$\langle a, c, b, d \rangle$$

e' un ordinamento topologico: a precede b e c , e sia b sia c precedono d .

13.4.7 Grafico o diagramma

DAG:



Ordinamento topologico possibile:

a c b d

Tutti gli archi vanno da sinistra verso destra.

13.4.8 Dimostrazione

La correttezza si basa sul fatto indicato nelle slide:

$$(u, v) \in E \implies u.f > v.f$$

in un DAG.

Consideriamo un arco (u, v) durante DFS.

Se v e' bianco quando l'arco viene esaminato, allora v diventa discendente di u nella foresta DFS.

Un discendente viene completato prima dell'antenato, quindi

$$v.f < u.f.$$

Se v e' nero, allora v e' gia' stato completato prima del completamento di u , quindi ancora

$$v.f < u.f.$$

Il caso in cui v sia grigio corrisponderebbe a un arco all'indietro verso un antenato. Questo produrrebbe un ciclo, impossibile in un DAG.

Quindi per ogni arco (u, v) vale

$$u.f > v.f.$$

L'algoritmo inserisce ogni nodo in testa alla lista quando viene completato. Di conseguenza, i nodi con tempo di completamento maggiore compaiono prima. Pertanto, per ogni arco (u, v) , u compare prima di v . Questa e' esattamente la definizione di ordinamento topologico.

13.4.9 Analisi della complessita'

L'algoritmo esegue DFS e inserisce ogni nodo una volta in lista. Quindi:

$$\Theta(|V| + |E|)$$

per la DFS, piu' $O(|V|)$ per gli inserimenti in testa.

La complessita' finale e':

$$\Theta(|V| + |E|).$$

Lo spazio aggiuntivo e' $O(|V|)$ per la lista e per gli attributi di DFS.

13.4.10 Errori tipici d'esame

Applicare l'ordinamento topologico a un grafo diretto con cicli. La definizione usata nelle slide richiede un DAG.

Ordinare i nodi per tempo di scoperta invece che per tempo di completamento. L'algoritmo usa i tempi f , non i tempi d .

Inserire il nodo in coda invece che in testa al completamento. L'inserimento in testa realizza l'ordine decrescente dei tempi di completamento.

13.4.11 Possibili domande d'esame

- Definire DAG e ordinamento topologico.
- Scrivere l'algoritmo TOPOLOGICAL-SORT.
- Dimostrare che in un DAG, per ogni arco (u, v) , vale $u.f > v.f$.
- Simulare l'algoritmo su un piccolo DAG.

- Analizzare la complessità.

13.4.12 Collegamenti teorici

L'ordinamento topologico è un'applicazione diretta di DFS. Usa i tempi di completamento e il fatto che un DAG non contiene archi all'indietro.

13.5 Componenti fortemente connesse

13.5.1 Definizione

In un grafo diretto

$$G = (V, E),$$

una **componente fortemente connessa** è un insieme massimale di nodi

$$C \subseteq V$$

tale che, per ogni coppia $u, v \in C$, vale:

$$u \rightsquigarrow v \quad \text{e} \quad v \rightsquigarrow u.$$

Il termine **massimale** significa che non è possibile aggiungere a C un altro nodo preservando la proprietà di mutua raggiungibilità.

Grafo trasposto. Dato un grafo diretto

$$G = (V, E),$$

il suo grafo trasposto è

$$G^T = (V, E^T),$$

dove

$$E^T = \{(v, u) \mid (u, v) \in E\}.$$

Il grafo trasposto inverte il verso di tutti gli archi.

13.5.2 Intuizione

Una componente fortemente connessa è una regione del grafo in cui tutti i nodi sono raggiungibili reciprocamente. Se si contraggono le componenti in singoli nodi, si ottiene una struttura senza cicli tra componenti.

L'algoritmo delle slide usa due visite DFS:

1. una DFS su G per calcolare i tempi di completamento;

2. una DFS su G^T , scegliendo i nodi in ordine decrescente rispetto ai tempi di completamento calcolati in G .

13.5.3 Motivazione

DFS da sola non basta a separare direttamente le componenti fortemente connesse in un grafo diretto arbitrario. I tempi di completamento della prima DFS indicano l'ordine corretto in cui esplorare il grafo trasposto.

13.5.4 Proprieta' teoriche

Stesse componenti in G e G^T . I grafi G e G^T hanno le stesse componenti fortemente connesse. Infatti, se in G esiste un cammino $u \rightsquigarrow v$, allora in G^T esiste il cammino inverso $v \rightsquigarrow u$. La mutua raggiungibilita' e' quindi preservata invertendo tutti gli archi.

Tempi di componente. Per una componente U , definiamo:

$$d(U) = \min\{u.d \mid u \in U\},$$

cioe' il primo tempo di scoperta di un vertice della componente, e

$$f(U) = \max\{u.f \mid u \in U\},$$

cioe' l'ultimo tempo di completamento di un vertice della componente.

Lemma 13.6. *Si considerino due componenti fortemente connesse distinte C e C' di $G = (V, E)$. Se*

$$(u, v) \in E, \quad u \in C', \quad v \in C,$$

allora

$$f(C') > f(C).$$

Corollario 13.7. *Si considerino due componenti fortemente connesse distinte C e C' di $G = (V, E)$. Se*

$$f(C') < f(C),$$

allora E^T non contiene nessun arco (v, u) tale che

$$v \in C, \quad u \in C'.$$

13.5.5 Algoritmo

Algorithm 54: STRONGLY-CONNECTED-COMPONENTS(G)

Input: $G = (V, E)$ **Output:** Componenti fortemente connesse di G

```

1 DFS( $G$ );                                // determina i tempi di completamento  $u.f$ 
2 Determina  $G^T$ ;
3 Esegui DFS( $G^T$ ) esaminando i nodi in ordine decrescente rispetto al tempo di
  completamento calcolato in  $G$ ;
4  $C \leftarrow []$ ;
5  $i \leftarrow 0$ ;
6 foreach albero DFS  $T$  in  $G^T$  do
7    $C[i] \leftarrow$  sottografo indotto dai nodi dell'albero  $T$ ;
8    $i \leftarrow i + 1$ ;
9 return  $C$ ;
```

13.5.6 Esempio guidato

Consideriamo il grafo diretto:

$$a \rightarrow b, \quad b \rightarrow a, \quad b \rightarrow c, \quad c \rightarrow d, \quad d \rightarrow c.$$

Le componenti fortemente connesse sono:

$$C_1 = \{a, b\}, \quad C_2 = \{c, d\}.$$

Infatti a e b si raggiungono reciprocamente, e anche c e d si raggiungono reciprocamente. Inoltre esiste un arco da C_1 a C_2 , ma non un cammino di ritorno da C_2 a C_1 .

La prima DFS su G calcola i tempi f . Poi si costruisce G^T , invertendo tutti gli archi:

$$b \rightarrow a, \quad a \rightarrow b, \quad c \rightarrow b, \quad d \rightarrow c, \quad c \rightarrow d.$$

La seconda DFS su G^T , usando l'ordine decrescente dei tempi f della prima DFS, produce alberi DFS che coincidono con le componenti fortemente connesse.

13.5.7 Grafico o diagramma

G:

a <-----> b -----> c <-----> d

Componenti fortemente connesse:

$\{a,b\} \text{ ----> } \{c,d\}$

G^T :

$a \text{ <----> } b \text{ <---- } c \text{ <----> } d$

La seconda DFS su G^T , nell'ordine corretto,
estrae un albero per ciascuna componente.

13.5.8 Dimostrazione

Dimostrazione del lemma. Siano C e C' due componenti fortemente connesse distinte e sia

$$(u, v) \in E$$

con

$$u \in C', \quad v \in C.$$

Ci sono due casi.

Caso 1: $d(C') < d(C)$. Sia $y \in C'$ tale che

$$y.d = d(C').$$

Al tempo $y.d$, i nodi di C' e di C rilevanti per il cammino sono ancora bianchi, e da y si possono raggiungere:

- tutti i nodi di C' , per forte connessione;
- il nodo u , quindi tramite l'arco (u, v) il nodo v ;
- tutti i nodi di C , per forte connessione di C .

Quindi i nodi di C' e C diventano discendenti di y nella DFS. Ne segue:

$$y.f = f(C') > f(C).$$

Caso 2: $d(C') > d(C)$. Sia $y \in C$ tale che

$$y.d = d(C).$$

All'inizio della visita da y , i nodi di C' sono ancora bianchi. Non puo' esistere un cammino da y a un nodo di C' , perche' altrimenti, usando l'arco da C' a C e la forte connessione interna delle due componenti, C e C' non sarebbero due componenti distinte.

Dunque la DFS partita da y completa i nodi di C senza scoprire i nodi di C' . Al tempo

$$y.f = f(C),$$

i nodi di C' sono ancora bianchi. Essi saranno scoperti e completati solo dopo. Quindi:

$$f(C') > f(C).$$

In entrambi i casi vale

$$f(C') > f(C).$$

Dimostrazione del corollario. La contronominale del lemma dice: se

$$f(C') < f(C),$$

allora non esiste alcun arco

$$(u, v) \in E$$

con

$$u \in C', \quad v \in C.$$

Nel grafo trasposto, un arco (u, v) di G diventa (v, u) in G^T . Quindi G^T non contiene archi da C verso C' .

Correttezza dell'algoritmo SCC. La prima DFS su G calcola i tempi di completamento. La seconda DFS viene eseguita su G^T in ordine decrescente di tali tempi.

Sia x il primo nodo scelto nella seconda DFS, cioè quello con tempo $x.f$ massimo. Sia C la componente fortemente connessa che contiene x . Allora $f(C)$ è massimo tra le componenti.

Per il corollario, in G^T non esistono archi da C verso componenti con tempo di completamento minore ancora bianche. Quindi la visita DFS da x in G^T non può uscire da C verso una nuova componente bianca. Poiché G e G^T hanno le stesse componenti fortemente connesse, la DFS da x raggiunge tutti e soli i nodi di C . Il primo albero DFS della seconda visita determina quindi una componente fortemente connessa.

Nel passo successivo, il ciclo principale di DFS su G^T sceglie tra i nodi ancora bianchi un nodo y con tempo di completamento massimo. Lo stesso ragionamento vale per la componente di y : eventuali archi verso componenti già trovate portano a nodi neri, mentre non esistono archi verso componenti bianche con tempo minore che possano far uscire la visita dalla componente corrente.

Ripetendo il ragionamento, ogni albero DFS della seconda visita identifica esattamente una componente fortemente connessa. Quindi l'algoritmo calcola correttamente tutte le componenti fortemente connesse di un grafo orientato.

13.5.9 Analisi della complessita'

L'algoritmo esegue:

- una DFS su G :

$$\Theta(|V| + |E|);$$

- la costruzione di G^T :

$$\Theta(|V| + |E|)$$

se G e' rappresentato con liste di adiacenza;

- una DFS su G^T :

$$\Theta(|V| + |E|).$$

La complessita' totale e':

$$\Theta(|V| + |E|).$$

Le slide osservano che ciascun $C[i]$ puo' essere determinato durante la seconda visita DFS; il ciclo che raccoglie gli alberi e' scritto nell'algoritmo per chiarezza.

13.5.10 Errori tipici d'esame

Eseguire la seconda DFS su G invece che su G^T . L'inversione degli archi e' essenziale.

Usare nella seconda DFS un ordine qualunque dei nodi. L'ordine deve essere decrescente rispetto ai tempi di completamento calcolati nella prima DFS su G .

Confondere componente connessa e componente fortemente connessa. Nei grafi diretti serve raggiungibilita' in entrambe le direzioni per ogni coppia di nodi della componente.

13.5.11 Possibili domande d'esame

- Definire una componente fortemente connessa.
- Definire il grafo trasposto.
- Dimostrare che G e G^T hanno le stesse componenti fortemente connesse.
- Enunciare e dimostrare il lemma sui tempi $f(C') > f(C)$.
- Scrivere l'algoritmo STRONGLY-CONNECTED-COMPONENTS.
- Dimostrare la correttezza dell'algoritmo.
- Analizzare la complessita' $\Theta(|V| + |E|)$.

13.5.12 Collegamenti teorici

Le componenti fortemente connesse sono un'applicazione avanzata di DFS. L'algoritmo usa:

- tempi di completamento;
- grafo trasposto;
- seconda DFS ordinata per tempi decrescenti.

13.6 Procedura per stampare un cammino minimo

13.6.1 Definizione

Tra gli esercizi ufficiali delle slide compare la richiesta di scrivere la procedura che stampa il cammino minimo dalla sorgente a un nodo dato, se esiste.

Dopo BFS, per ogni nodo raggiungibile v , il predecessore $v.\pi$ permette di risalire da v verso la sorgente s lungo l'albero BF.

13.6.2 Intuizione

Il cammino viene ricostruito all'indietro:

$$v, v.\pi, v.\pi.\pi, \dots, s.$$

Per stamparlo nell'ordine corretto da s a v , si usa una procedura ricorsiva: prima stampa il cammino fino al predecessore, poi stampa v .

13.6.3 Motivazione

BFS non restituisce solo le distanze. Gli attributi π codificano anche un cammino minimo effettivo, come affermato dal teorema di correttezza.

13.6.4 Proprieta' teoriche

Se v e' raggiungibile da s , allora la catena dei predecessori termina in s :

$$v, v.\pi, v.\pi.\pi, \dots, s.$$

Se v non e' raggiungibile, allora

$$v.\pi = \text{NULL}$$

e $v \neq s$.

13.6.5 Algoritmo

Algorithm 55: PRINT-PATH(s, v)

Input: Sorgente s , nodo destinazione v , predecessori π calcolati da BFS**Output:** Cammino minimo da s a v , se esiste

```
1 if  $v = s$  then
2   | stampa  $s$ ;
3 else
4   | if  $v.\pi = \text{NULL}$  then
5     | stampa “nessun cammino da  $s$  a  $v$ ”;
6   | else
7     | PRINT-PATH( $s, v.\pi$ );
8     | stampa  $v$ ;
```

13.6.6 Esempio guidato

Nell'esempio BFS con sorgente 2:

$$5.\pi = 4, \quad 4.\pi = 2, \quad 2.\pi = \text{NULL}.$$

La chiamata

PRINT-PATH(2, 5)

esegue:

PRINT-PATH(2, 4),

che esegue:

PRINT-PATH(2, 2).

Stampa quindi:

2, 4, 5.

13.6.7 Grafico o diagramma

Catena dei predecessori:

5.pi = 4
4.pi = 2
2.pi = NULL

Risalita:

5 -> 4 -> 2

Stampa ricorsiva:

2 -> 4 -> 5

13.6.8 Dimostrazione

La procedura e' corretta per induzione sulla lunghezza del cammino.

Caso base. Se $v = s$, il cammino da s a s contiene solo s , e la procedura stampa s .

Passo induttivo. Se $v \neq s$ e $v.\pi \neq \text{NULL}$, per il teorema di correttezza di BFS un cammino minimo da s a v e' formato da un cammino minimo da s a $v.\pi$ seguito dall'arco $(v.\pi, v)$. Per ipotesi induttiva, la chiamata ricorsiva stampa correttamente il cammino da s a $v.\pi$; stampando poi v , si ottiene il cammino da s a v .

Se $v.\pi = \text{NULL}$ e $v \neq s$, non esiste cammino da s a v registrato da BFS, quindi la procedura segnala correttamente l'assenza del cammino.

13.6.9 Analisi della complessita'

La procedura segue al piu' un predecessore per ogni nodo del cammino. Nel caso peggiore:

$$O(|V|)$$

tempo e

$$O(|V|)$$

spazio sullo stack ricorsivo.

13.6.10 Errori tipici d'esame

Stampare prima v e poi il predecessore produce il cammino al contrario. La procedura ricorsiva deve stampare prima il cammino verso $v.\pi$ e poi v .

Non gestire il caso $v.\pi = \text{NULL}$ quando $v \neq s$. In quel caso il cammino non esiste.

13.6.11 Possibili domande d'esame

- Scrivere PRINT-PATH.
- Spiegare perche' usa i predecessori π .
- Dimostrare la correttezza per induzione.
- Analizzare tempo e spazio.

13.6.12 Collegamenti teorici

La procedura e' collegata direttamente al teorema di correttezza di BFS: le distanze d danno la lunghezza minima, mentre i predecessori π permettono di ricostruire un cammino minimo.

14. Può tornare utile

Questa sezione contiene materiale di riferimento (appendici) e domande d'esame.

14.1 Appendice A (Sommatorie)

14.1.1 Formule fondamentali

$$\begin{aligned}\sum_{i=1}^n 1 &= n \\ \sum_{i=1}^n i &= \frac{n(n+1)}{2} = \Theta(n^2) \\ \sum_{i=1}^n i^2 &= \frac{n(n+1)(2n+1)}{6} = \Theta(n^3) \\ \sum_{i=1}^n i^k &= \Theta(n^{k+1}) \quad \text{per } k \geq 0 \\ \sum_{i=0}^n x^i &= \frac{x^{n+1} - 1}{x - 1} \quad \text{per } x \neq 1 \\ \sum_{i=0}^{\infty} x^i &= \frac{1}{1 - x} \quad \text{per } |x| < 1\end{aligned}$$

14.1.2 Serie geometrica

Nota

La serie geometrica è fondamentale nell'analisi degli algoritmi ricorsivi.

Formula generale:

$$\sum_{i=0}^n ar^i = a \cdot \frac{r^{n+1} - 1}{r - 1}$$

Caso particolare importante ($r = 2$):

$$\sum_{i=0}^n 2^i = 2^{n+1} - 1$$

Serie infinita convergente ($|r| < 1$):

$$\sum_{i=0}^{\infty} ar^i = \frac{a}{1-r}$$

14.1.3 Serie armonica

$$H_n = \sum_{i=1}^n \frac{1}{i} = 1 + \frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} = \Theta(\log n)$$

Nota

La serie armonica cresce **logaritmicamente**. Utile per analizzare:

- Quicksort (numero medio di confronti)
- Hashing (lunghezza catene)
- Alberi bilanciati (altezza media)

Approssimazione: $H_n \approx \ln n + \gamma$ dove $\gamma \approx 0.5772$ (costante di Eulero-Mascheroni)

14.1.4 Tecniche di manipolazione

1. Separazione dei termini:

$$\sum_{i=1}^n (a_i + b_i) = \sum_{i=1}^n a_i + \sum_{i=1}^n b_i$$

2. Estrazione di costanti:

$$\sum_{i=1}^n c \cdot a_i = c \sum_{i=1}^n a_i$$

3. Telescoping (sommatoria telescopica):

$$\sum_{i=1}^n (a_i - a_{i-1}) = a_n - a_0$$

Esempio 14.1 (Telescoping applicato).

$$\sum_{i=1}^n \frac{1}{i(i+1)} = \sum_{i=1}^n \left(\frac{1}{i} - \frac{1}{i+1} \right) = 1 - \frac{1}{n+1} = \frac{n}{n+1}$$

4. Cambio di variabile:

Se $j = n - i$, allora:

$$\sum_{i=0}^n f(i) = \sum_{j=n}^0 f(n-j) = \sum_{j=0}^n f(n-j)$$

14.1.5 Approssimazione con integrali

Per f crescente:

$$\int_1^n f(x) dx \leq \sum_{i=1}^n f(i) \leq f(n) + \int_1^n f(x) dx$$

Esempio 14.2 (Somma di potenze).

$$\begin{aligned} \sum_{i=1}^n i^k &\approx \int_1^n x^k dx = \frac{n^{k+1}}{k+1} \\ &\Rightarrow \sum_{i=1}^n i^k = \Theta(n^{k+1}) \end{aligned}$$

14.2 Appendice B (Insiemi, relazioni, funzioni, grafi, alberi)

14.2.1 Insiemi

Definizione 14.1 (Insieme). Collezione non ordinata di elementi distinti. Notazione: $A = \{1, 2, 3\}$

Operazioni fondamentali:

- **Unione:** $A \cup B = \{x : x \in A \text{ oppure } x \in B\}$
- **Intersezione:** $A \cap B = \{x : x \in A \text{ e } x \in B\}$
- **Differenza:** $A \setminus B = \{x : x \in A \text{ e } x \notin B\}$
- **Complemento:** $\overline{A} = \{x : x \notin A\}$ (rispetto a un universo)

Nota

Cardinalità: $|A|$ = numero di elementi in A

Insieme delle parti: $\mathcal{P}(A)$ = insieme di tutti i sottoinsiemi di A

Se $|A| = n$ allora $|\mathcal{P}(A)| = 2^n$

14.2.2 Relazioni

Definizione 14.2 (Relazione). Una relazione R da A a B è un sottoinsieme di $A \times B$ (prodotto cartesiano).

Scriviamo aRb se $(a, b) \in R$.

Proprietà di relazioni su A (relazione binaria):

- **Riflessiva:** $\forall a \in A : aRa$
- **Simmetrica:** $\forall a, b \in A : aRb \Rightarrow bRa$
- **Transitiva:** $\forall a, b, c \in A : (aRb \wedge bRc) \Rightarrow aRc$
- **Antisimmetrica:** $\forall a, b \in A : (aRb \wedge bRa) \Rightarrow a = b$

Definizione 14.3 (Relazione di equivalenza). Una relazione è di **equivalenza** se è riflessiva, simmetrica e transitiva.

Esempio: uguaglianza ($=$), congruenza modulo n

Definizione 14.4 (Ordine parziale). Una relazione è di **ordine parziale** se è riflessiva, antisimmetrica e transitiva.

Esempio: $\leq$ sui numeri reali, $\subseteq$ sugli insiemi

14.2.3 Funzioni

Definizione 14.5 (Funzione). Una funzione $f : A \rightarrow B$ assegna a ogni elemento di A esattamente un elemento di B .

- $A =$ **dominio**
- $B =$ **codominio**
- $f(A) = \{f(a) : a \in A\} =$ **immagine**

Tipi di funzioni:

- **Iniettiva** (uno-a-uno): $f(a_1) = f(a_2) \Rightarrow a_1 = a_2$
- **Suriettiva** (sopra): $\forall b \in B, \exists a \in A : f(a) = b$
- **Biettiva** (biunivoca): iniettiva e suriettiva

Nota

Una funzione biettiva ha **funzione inversa** $f^{-1} : B \rightarrow A$ tale che:

$$f^{-1}(f(a)) = a \quad \text{e} \quad f(f^{-1}(b)) = b$$

14.2.4 Grafi

Definizione 14.6 (Grafo). Un grafo $G = (V, E)$ consiste di:

- $V =$ insieme di **vertici** (o nodi)
- $E \subseteq V \times V =$ insieme di **archi**

Tipi di grafi:

- **Grafo non orientato:** $(u, v) \in E \Leftrightarrow (v, u) \in E$
- **Grafo orientato** (digrafo): gli archi hanno una direzione
- **Grafo pesato:** ogni arco ha un peso $w : E \rightarrow \mathbb{R}$

Terminologia:

- **Grado** di v : numero di archi incidenti
 - Grafo orientato: *grado entrante* + *grado uscente*
- **Cammino:** sequenza di vertici $v_1, v_2, \dots, v_k$ con $(v_i, v_{i+1}) \in E$
- **Ciclo:** cammino con $v_1 = v_k$
- **Connesso:** esiste un cammino tra ogni coppia di vertici

Rappresentazioni:

1. **Matrice di adiacenza** ($n \times n$):

$$A[i][j] = \begin{cases} 1 & \text{se } (i, j) \in E \\ 0 & \text{altrimenti} \end{cases}$$

Spazio: $\Theta(n^2)$, accesso arco: $O(1)$

2. **Liste di adiacenza:** Array di n liste, $\text{Adj}[u]$ contiene i vicini di u

Spazio: $\Theta(n + m)$, iterare vicini: $O(\deg(u))$

14.2.5 Alberi

Definizione 14.7 (Albero). Un albero è un grafo **connesso** e **aciclico**.

Equivalentemente:

- Grafo connesso con n nodi e $n - 1$ archi
- Esiste un unico cammino tra ogni coppia di nodi

Albero radicato: albero con un nodo speciale detto **radice**.

- **Padre/Figlio:** u è padre di v se $(u, v) \in E$ e u è più vicino alla radice
- **Foglia:** nodo senza figli

- **Altezza:** lunghezza del cammino più lungo dalla radice a una foglia
- **Profondità** di v : distanza dalla radice a v

Definizione 14.8 (Albero binario). Albero radicato in cui ogni nodo ha al più 2 figli (sinistro e destro).

Albero binario completo: tutte le foglie allo stesso livello, ogni nodo interno ha 2 figli.

Un albero binario completo di altezza h ha $2^{h+1} - 1$ nodi.

Visite di alberi binari:

- **Preorder:** radice $\rightarrow$ sinistro $\rightarrow$ destro
- **Inorder:** sinistro $\rightarrow$ radice $\rightarrow$ destro
- **Postorder:** sinistro $\rightarrow$ destro $\rightarrow$ radice

14.3 Appendice C (Calcolo combinatorio e probabilità)

14.3.1 Principi di conteggio

1. Principio del prodotto:

Se un evento può avvenire in m modi e un altro in n modi, allora entrambi possono avvenire in $m \times n$ modi.

Esempio 14.3. Quante stringhe binarie di lunghezza 3?

Ogni posizione: 2 scelte $\Rightarrow$ totale: $2 \times 2 \times 2 = 8$

2. Principio della somma:

Se un evento può avvenire in m modi disgiunti o in n modi disgiunti, può avvenire in $m + n$ modi.

14.3.2 Permutazioni e combinazioni

Definizione 14.9 (Permutazione). Numero di modi per ordinare n oggetti distinti:

$$P(n) = n! = n \times (n-1) \times \cdots \times 2 \times 1$$

Definizione 14.10 (Permutazione con ripetizioni). Se tra n oggetti ci sono gruppi di $n_1, n_2, \dots, n_k$ oggetti identici:

$$\frac{n!}{n_1! \cdot n_2! \cdot \dots \cdot n_k!}$$

Esempio 14.4. Permutazioni della parola “MISSISSIPPI” (11 lettere: 4 I, 4 S, 2 P, 1 M):

$$\frac{11!}{4! \cdot 4! \cdot 2! \cdot 1!} = 34650$$

Definizione 14.11 (Disposizioni). Numero di modi per scegliere k oggetti da n tenendo conto dell'ordine:

$$D(n, k) = \frac{n!}{(n-k)!} = n \times (n-1) \times \cdots \times (n-k+1)$$

Definizione 14.12 (Combinazioni (coefficiente binomiale)). Numero di modi per scegliere k oggetti da n **senza** considerare l'ordine:

$$C(n, k) = \binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Proprietà dei coefficienti binomiali:

$$\begin{aligned} \binom{n}{0} &= \binom{n}{n} = 1 \\ \binom{n}{k} &= \binom{n}{n-k} \quad (\text{simmetria}) \\ \binom{n}{k} &= \binom{n-1}{k-1} + \binom{n-1}{k} \quad (\text{ricorsione - Pascal}) \\ \sum_{k=0}^n \binom{n}{k} &= 2^n \end{aligned}$$

14.3.3 Probabilità

Definizione 14.13 (Spazio campionario). Insieme Ω di tutti i possibili risultati di un esperimento.

Definizione 14.14 (Evento). Sottoinsieme $A \subseteq \Omega$ dello spazio campionario.

Definizione 14.15 (Probabilità). Funzione $P : \mathcal{P}(\Omega) \rightarrow [0, 1]$ tale che:

- $P(\Omega) = 1$
- $P(A \cup B) = P(A) + P(B)$ se $A \cap B = \emptyset$ (additività)

Caso equiprobabile:

$$P(A) = \frac{|A|}{|\Omega|} = \frac{\text{casi favorevoli}}{\text{casi possibili}}$$

Regole fondamentali:

- $P(\emptyset) = 0$
- $P(\overline{A}) = 1 - P(A)$ (complemento)
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$ (unione)

14.3.4 Probabilità condizionata e indipendenza

Definizione 14.16 (Probabilità condizionata). La probabilità di A dato B (con $P(B) > 0$):

$$P(A | B) = \frac{P(A \cap B)}{P(B)}$$

Teorema 14.1 (Regola del prodotto).

$$P(A \cap B) = P(A | B) \cdot P(B) = P(B | A) \cdot P(A)$$

Definizione 14.17 (Eventi indipendenti). A e B sono indipendenti se:

$$P(A \cap B) = P(A) \cdot P(B)$$

Equivalentemente: $P(A | B) = P(A)$

Teorema 14.2 (Teorema di Bayes).

$$P(A | B) = \frac{P(B | A) \cdot P(A)}{P(B)}$$

14.3.5 Variabili casuali e valore atteso

Definizione 14.18 (Variabile casuale). Funzione $X : \Omega \rightarrow \mathbb{R}$ che assegna un valore numerico a ogni risultato.

Definizione 14.19 (Valore atteso). Media pesata dei valori:

$$E[X] = \sum_x x \cdot P(X = x)$$

Proprietà del valore atteso:

- $E[aX + b] = aE[X] + b$ (linearità)
- $E[X + Y] = E[X] + E[Y]$ (sempre vera)
- $E[XY] = E[X] \cdot E[Y]$ (solo se X e Y indipendenti)

Definizione 14.20 (Variabile indicatrice). Variabile casuale che vale 1 se un evento si verifica, 0 altrimenti.

$$I_A = \begin{cases} 1 & \text{se } A \text{ si verifica} \\ 0 & \text{altrimenti} \end{cases}$$

Proprietà fondamentale: $E[I_A] = P(A)$

Esempio 14.5 (Numero atteso di confronti in QuickSort). Usando variabili indicatrici si dimostra che il numero atteso di confronti è $\Theta(n \log n)$.

14.4 Appendice D (Matrici)

14.4.1 Definizioni base

Definizione 14.21 (Matrice). Una matrice A di dimensione $m \times n$ è una tabella di m righe e n colonne:

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

Notazione: $A = (a_{ij})$ dove a_{ij} è l'elemento in posizione (i, j)

Tipi speciali:

- **Matrice quadrata:** $m = n$
- **Matrice identità** I_n : diagonale di 1, resto di 0

$$I_3 = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

- **Matrice diagonale:** solo la diagonale può essere non-zero
- **Matrice triangolare superiore:** $a_{ij} = 0$ per $i > j$
- **Matrice simmetrica:** $A = A^T$ (vale $a_{ij} = a_{ji}$)

14.4.2 Operazioni fondamentali

1. Somma: (solo se A e B hanno stesse dimensioni)

$$(A + B)_{ij} = a_{ij} + b_{ij}$$

Complessità: $\Theta(mn)$

2. Prodotto per scalare:

$$(cA)_{ij} = c \cdot a_{ij}$$

Complessità: $\Theta(mn)$

3. Trasposizione: scambia righe con colonne

$$(A^T)_{ij} = a_{ji}$$

Se A è $m \times n$, allora A^T è $n \times m$

Complessità: $\Theta(mn)$

4. Prodotto: A è $m \times n$, B è $n \times p$

$$(AB)_{ij} = \sum_{k=1}^n a_{ik} b_{kj}$$

AB è $m \times p$

Complessità (algoritmo naive): $\Theta(mnp)$

Per matrici quadrate $n \times n$: $\Theta(n^3)$

14.4.3 Proprietà del prodotto

- **Associativa:** $(AB)C = A(BC)$
- **Distributiva:** $A(B + C) = AB + AC$
- **NON commutativa:** in generale $AB \neq BA$
- $(AB)^T = B^T A^T$ (inversione ordine)
- $IA = AI = A$ (elemento neutro)

14.4.4 Determinante e matrice inversa

Definizione 14.22 (Determinante). Per matrice 2×2 :

$$\det \begin{bmatrix} a & b \\ c & d \end{bmatrix} = ad - bc$$

Per matrici $n \times n$: sviluppo di Laplace (ricorsivo)

Proprietà:

- $\det(AB) = \det(A) \cdot \det(B)$
- $\det(A^T) = \det(A)$
- $\det(I) = 1$
- $\det(cA) = c^n \det(A)$ per matrice $n \times n$

Definizione 14.23 (Matrice inversa). A è invertibile se esiste A^{-1} tale che:

$$AA^{-1} = A^{-1}A = I$$

Condizione necessaria e sufficiente: $\det(A) \neq 0$

Proprietà della matrice inversa:

- $(A^{-1})^{-1} = A$
- $(AB)^{-1} = B^{-1}A^{-1}$ (inversione ordine)
- $(A^T)^{-1} = (A^{-1})^T$

14.4.5 Applicazioni agli algoritmi

1. **Grafi:** matrice di adiacenza

$A^k[i][j]$ = numero di cammini di lunghezza k da i a j

2. **Programmazione dinamica:** Floyd-Warshall usa prodotto matriciale generalizzato

3. **Complessità prodotto matriciale:**

- Algoritmo standard: $\Theta(n^3)$
- Algoritmo di Strassen: $\Theta(n^{\log_2 7}) \approx \Theta(n^{2.81})$
- Algoritmi avanzati: $O(n^{2.37})$ (teorici, non pratici)

14.5 Esercizi proposti in aula

14.5.1 Confronto tra BST e Min-Heap

Definizione 14.24 (Proprietà BST). In un **albero binario di ricerca** (BST), per ogni nodo v vale:

$$\text{key}(\text{sottoalbero sinistro di } v) \leq \text{key}(v) \leq \text{key}(\text{sottoalbero destro di } v)$$

Questa proprietà garantisce un *ordinamento totale e globale* tra tutti i nodi dell'albero.

Definizione 14.25 (Proprietà Min-Heap). In un **min-heap**, per ogni nodo v con figli u_1, u_2 vale:

$$\text{key}(v) \leq \text{key}(u_1), \quad \text{key}(v) \leq \text{key}(u_2)$$

Questa proprietà garantisce solo un *ordinamento parziale e locale* tra padre e figli diretti. Non esiste alcuna relazione d'ordine imposta tra il figlio sinistro e il figlio destro.

Nota

[Differenza chiave] Nel BST la struttura permette di navigare efficientemente verso sinistra o destra per cercare un valore; nel min-heap sappiamo solo che la radice è il minimo globale, ma non abbiamo informazioni sull'ordinamento relativo tra i due sottoalberi.

14.6 Elencare ordinatamente le chiavi: BST vs Min-Heap

14.6.1 BST: visita in-order in $O(n)$

Nel BST è possibile elencare tutte le n chiavi in ordine crescente in tempo $O(n)$ tramite la **visita in-order** (sinistra $\rightarrow$ radice $\rightarrow$ destra), che visita ogni nodo esattamente una volta.

Algorithm 56: INORDER(v)**Input:** Radice v di un BST**Output:** Chiavi stampate in ordine crescente

```

1 if  $v \neq \text{NIL}$  then
2   INORDER(figlio sinistro di  $v$ );
3   stampa  $\text{key}(v)$ ;
4   INORDER(figlio destro di  $v$ );

```

14.6.2 Min-Heap: impossibile in $O(n)$

Non è possibile elencare ordinatamente le chiavi di un min-heap in tempo $O(n)$.

Dimostrazione per assurdo. Supponiamo per assurdo che esista un algoritmo A che, dato un min-heap di n elementi, ne elenca le chiavi in ordine in tempo $O(n)$.

1. La costruzione di un min-heap a partire da n elementi arbitrari richiede tempo $O(n)$ (algoritmo di Floyd).
2. Per ipotesi, A elenca le chiavi ordinate in $O(n)$.
3. Concatenando le due operazioni otterremmo un algoritmo di **ordinamento in tempo**:

$$O(n) + O(n) = O(n)$$

4. Ma questo contraddice il **limite inferiore** $\Omega(n \log n)$ dimostrato per qualsiasi algoritmo di ordinamento basato su confronti.

Quindi A non può esistere. $\square$

In parole semplici: se potessimo estrarre tutte le chiavi in ordine da un heap in $O(n)$, avremmo un algoritmo di sorting lineare, cosa dimostrata impossibile nel modello a confronti. La proprietà del min-heap è *troppo debole* per supportare un'enumerazione ordinata efficiente.

Costo effettivo. Estrarre tutte le n chiavi ordinate da un min-heap richiede n operazioni di EXTRACTMIN, ciascuna di costo $O(\log n)$, per un totale di $O(n \log n)$ — esattamente il costo dell'**HeapSort**.

14.7 Domande d'esame (Appello 2)

Queste sono domande reali dell'esame.³

Definizione di algoritmo

D: Definizione sufficiente di algoritmo?

R: Procedura ben definita che produce output da input in tempo finito.

Problema computazionale

D: Cos'è un problema computazionale?

R: Relazione input-output desiderata. Il problema dice *cosa*, l'algoritmo dice *come*.

Efficienza

D: Cosa si intende per efficienza?

R: Come crescono le risorse al crescere di n (non il tempo in secondi, dipendente dall'hardware).

InsertionSort vs MergeSort

D: $\text{InsSort} = 10n^2$, $\text{MergeSort} = 32n \log_2 n$. Quando è più veloce InsSort?

R: Per n piccoli: $10n^2 < 32n \log_2 n$ solo inizialmente.

$100n^2$ vs 2^n

D: Minimo n tale che $100n^2 < 2^n$?

R: $n \geq 15$.

Invariante di ciclo

D: Cos'è un'invariante di ciclo?

R: Proprietà vera all'inizio di ogni iterazione. Si dimostra per inizializzazione, conservazione e conclusione.

Vale $\max(f, g) = \Theta(f + g)$?

R: Sì: $\max(f, g) \leq f + g \leq 2 \max(f, g)$, con $c_1 = 1$, $c_2 = 2$.

2^{n+1} e 2^{2n} vs $O(2^n)$

R: $2^{n+1} = O(2^n)$ (costante 2). $2^{2n} = 4^n \neq O(2^n)$ (richiederebbe $2^n \leq c$, impossibile).

Costo costante in RAM

R: Quando la parola di memoria ha dimensione fissa (es. 64 bit).

³Se si studiano solo queste l'esame si passa solo se si è colpiti da un fulmine mentre si fa un salto mortale carpiato suonando il flauto

Prestazioni su istanze note

R: Precomputare le soluzioni e restituirle direttamente quando quell'istanza viene ricevuta.

$$T(n) = T(n-1) + n^c$$

R: $T(n) = \Theta(n^{c+1})$. Dim.: $\sum_{k=1}^n k^c \approx \int_1^n x^c dx = \frac{n^{c+1}}{c+1}$.

$$T(n) = 2T(n/3) + 1$$

R: Master Theorem, caso 1: $T(n) = \Theta(n^{\log_3 2})$ ($\log_3 2 \approx 0.63$).

Divide et impera: classe di problemi

R: Problemi divisibili in 2+ sotto-istanze con combinazione a costo al più polinomiale.

Analisi probabilistica

R: Uso del calcolo della probabilità nell'analisi degli algoritmi (non solo caso medio).

permute-without-identity

R: No: con $n = 3$ produce solo 3 permutazioni invece delle 5 attese.

Hire-Assistant: probabilità

R: Esattamente 1 assunzione: $1/n$. Esattamente n assunzioni: $1/n!$.

Generatore casuale

R: Valore intero in un intervallo con probabilità uniforme.

Algoritmo randomizzato

R: Il comportamento dipende dall'input e da valori casuali.

Ricorrenza con frazioni

D: $T(n) \leq T(10n/11) + T(n/11) + O(n)$?

R: $T(n) = O(n \log n)$. Ogni livello costa $O(n)$, profondità $O(\log n)$.

Partition su array

D: Partition su $[13, 19, 9, 5, 12, 8, 7, 4, 21, 2, 6, 11]$, pivot $A[11]$?

R: $[9, 5, 8, 7, 4, 2, 6, 11, 21, 13, 19, 12]$, pivot in pos. 7.

Quicksort con elementi uguali

R: $O(n^2)$: ogni Partition genera partizione $(n-1)$ vs 0.

Partition elementi uguali: fix

R: Restituisce indice massimo. Fix: flag booleano; se tutti uguali, restituire $\lfloor (p+r)/2 \rfloor$.

Quicksort non crescente

R: Invertire il confronto in Partition: da $\leq$ a $\geq$.

Rand-Select: caso peggiore

R: Il pivot è sempre il massimo: dimensione scende di 1 a ogni passo, $O(n^2)$.

IV statistica d'ordine

D: IV statistica di $[8, 4, 8, 3, 4, 2, 3, 2]$?

R: Ordinato: $[2, 2, 3, 3, 4, 4, 8, 8]$. IV elemento = 3.

Eliminare duplicati

D: Eliminare duplicati in $\Theta(n \log n)$?

R: Ordinare, poi scorrere linearmente rimuovendo consecutivi uguali.

Build-Max-Heap: indice decrescente

R: Le foglie sono già heap. MaxHeapify richiede figli sistemati: si parte dal basso.

Visita inorder con pila

R: Visita in ordine (inorder), $O(n)$. Pattern: push-sinistra, pop-visita-destra.

LCS senza tabella b

```
PRINT-LCS(c, X, Y, i, j)
if c[i,j] == 0 then return
if X[i] == Y[j]
  PRINT-LCS(c, X, Y, i-1, j-1)
  print X[i]
else if c[i-1,j] >= c[i,j-1]
  PRINT-LCS(c, X, Y, i-1, j)
else
  PRINT-LCS(c, X, Y, i, j-1)
```

Costo $O(m + n)$: ogni chiamata riduce i o j di 1.

Cammino minimo vs TSP

R: Cammino minimo: sorgente $\rightarrow$ destinazione, polinomiale (Dijkstra). TSP: circuito visitando ogni nodo una volta, NP-difficile.

Prima e ultima cifra (Java)

```
1  int last  = n % 10;
2  int first = n / ((int) Math.pow(10, (int) (Math.log10(n))));
```

T extends Comparable

R: La ricerca binaria usa confronti: serve `compareTo()`, disponibile solo se T estende `Comparable`.

Ereditarietà e attributo omonimo

R: `super.getIdA()` per `A.idA`, `this.getIdA()` per `B.idA`. Le variabili private non sono ereditate.

Errore in makeCombination

R: `values = numbers` copia solo il riferimento locale. Fix: copia elemento per elemento.

Eccezione in try-with-resources

R: L'eccezione di `close()` è soppressa (JLS 14.20.3): diventa *suppressed exception* della prima.

Downcasting Java

D: `A value = new B(); B value1 = (A) value; value = null;?`

R: `value` → `null`; `value1` punta all'oggetto `B` ancora in memoria.

Stack con due code

Push i -esimo: enqueue in ausiliaria, svuota corrente nell'ausiliaria, scambia. Costo: $O(i)$. Totale n push: $O(n^2)$.

Pop: dequeue. Costo: $O(1)$.

Ricerca in matrice destrorsa

R: Ricerca binaria su ogni riga. Complessità: $O(n \log n)$.

```

1  public static <T extends Comparable<T>>
2  int[] search(T[][] M, T x) {
3      int n = M.length;
4      for (int i = 0; i < n; i++) {
5          int lo = 0, hi = n - 1;
6          while (lo <= hi) {
7              int mid = (lo + hi) / 2;
8              int cmp = M[i][mid].compareTo(x);
9              if (cmp == 0) return new int[]{i, mid};
10             else if (cmp < 0) lo = mid + 1;
11             else hi = mid - 1;
12         }
13     }
14     return null;
15 }
```